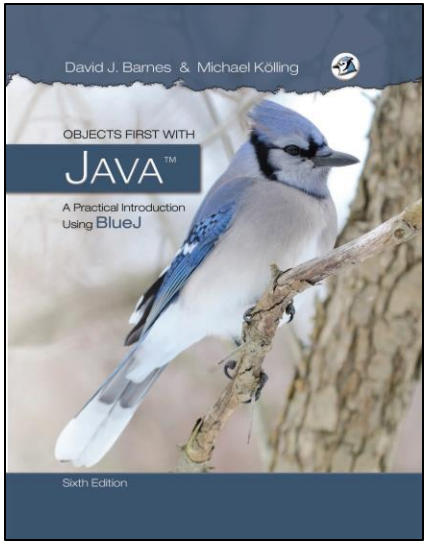
Unidad 5

Herencia



Esta presentación es una traducción y adaptación de las presentaciones proporcionadas por la editorial Pearson de los capítulos 10, 11 y 12 del libro “Objects First with Java: A Practical Introduction Using BlueJ, 6th Edition” escrito por David J. Barnes y Michael Kolling” © 2017

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales

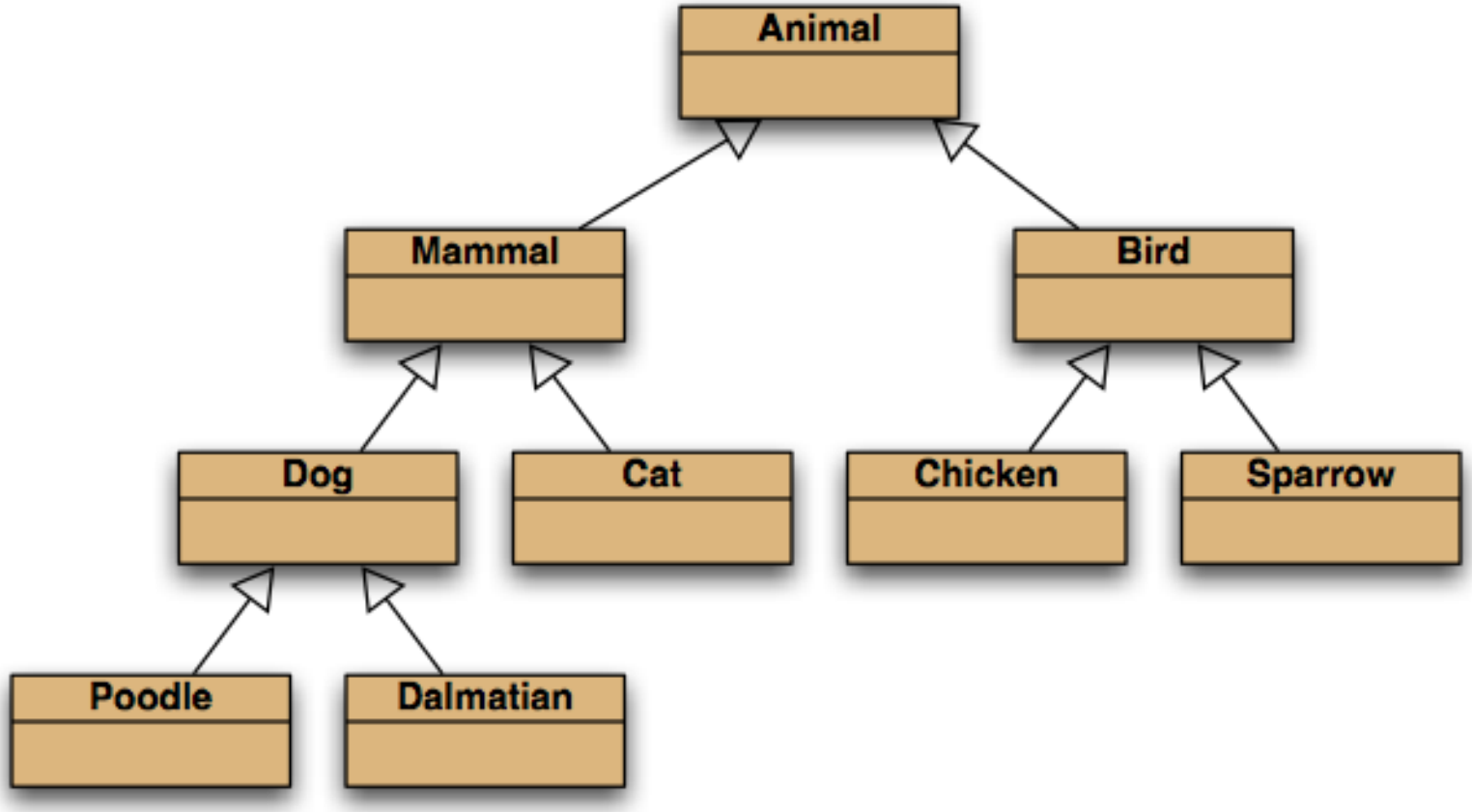
- Herencia
  - Constructores
- Polimorfismo
  - Subtipos y Sustitución
  - La Clase Object
  - Colecciones Polimórficas
  - Casting
  - Tipo Estático y Dinámico
  - Sobreescritura
  - En Métodos
- StringBuilder
- Clases y métodos abstractos
- Métodos de la clase Object
  - toString
  - equals
  - hashCode
- Extendiendo jerarquías de clases
- Herencia múltiple con Interfaces
- Acceso Protegido (protected)
- La clase “Class”

Conceptos fundamentales

- Herencia
  - Constructores
- Polimorfismo
  - Subtipos y Sustitución
  - La Clase Object
  - Colecciones Polimórficas
  - Casting
  - Tipo Estático y Dinámico
  - Sobreescritura
  - En Métodos
- StringBuilder
- Clases y métodos abstractos
- Métodos de la clase Object
  - toString
  - equals
  - hashCode
- Extendiendo jerarquías de clases
- Herencia múltiple con Interfaces
- Acceso Protegido (protected)
- La clase “Class”

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Herencia

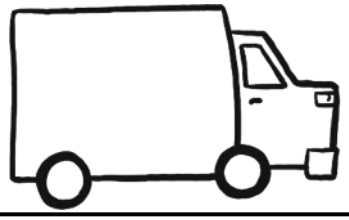


Según la **RAE**:
Rasgo o **rasgos** morales, científicos, ideológicos, etc., que,
habiendo caracterizado a alguien,
continúan advirtiéndose en sus descendientes o continuadores.

Herencia

- Truck y Car tienen muchas similitudes
- Seguramente tienen mucho código duplicado





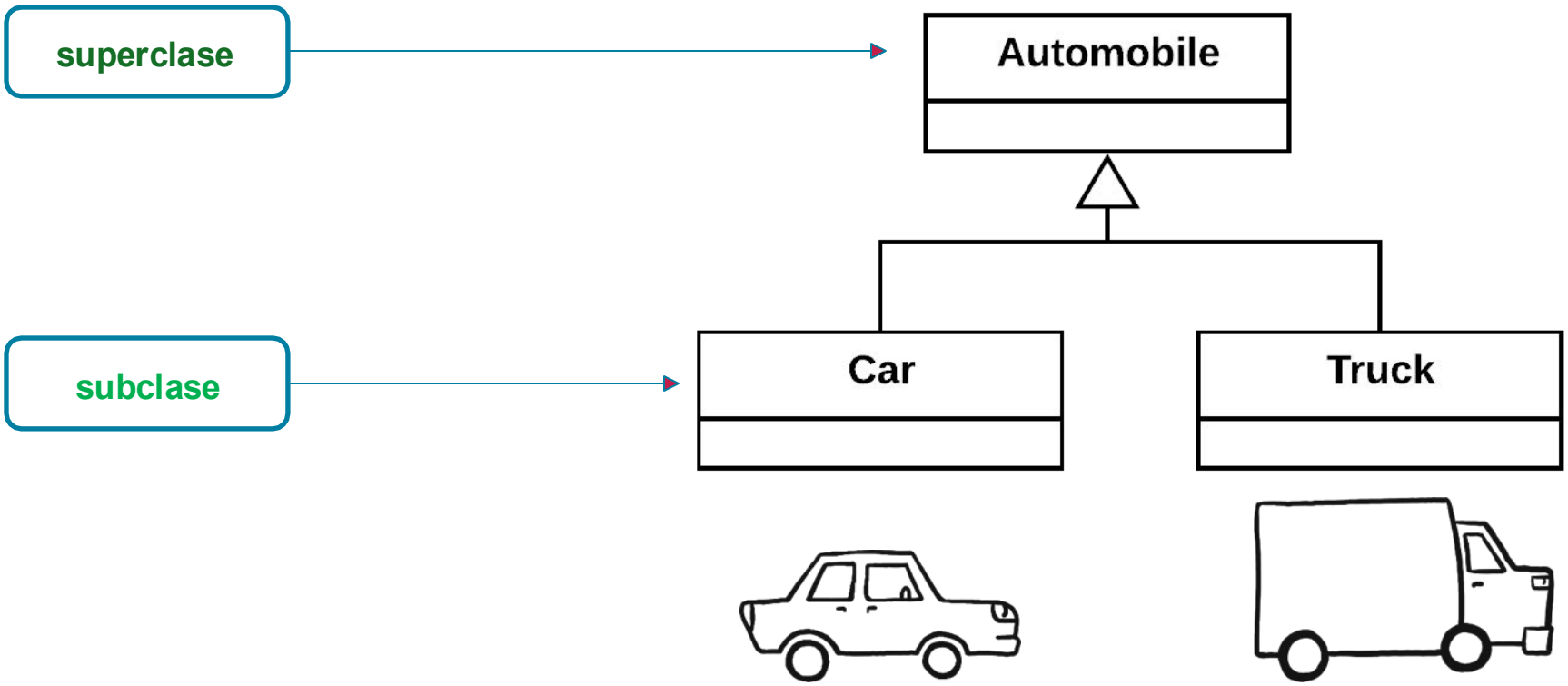
Car
model owner weight size fuel numOfSeats
drive(location) refuel(fuel) transportPeople(People)

Truck
model owner weight size fuel numOfSeats maxLoad
drive(location) refuel(fuel) transportPeople(People)

yright © 2017, 20 Rights Reserved

Herencia

- Un Car es un Automobile
- Un Truck es un Automobile

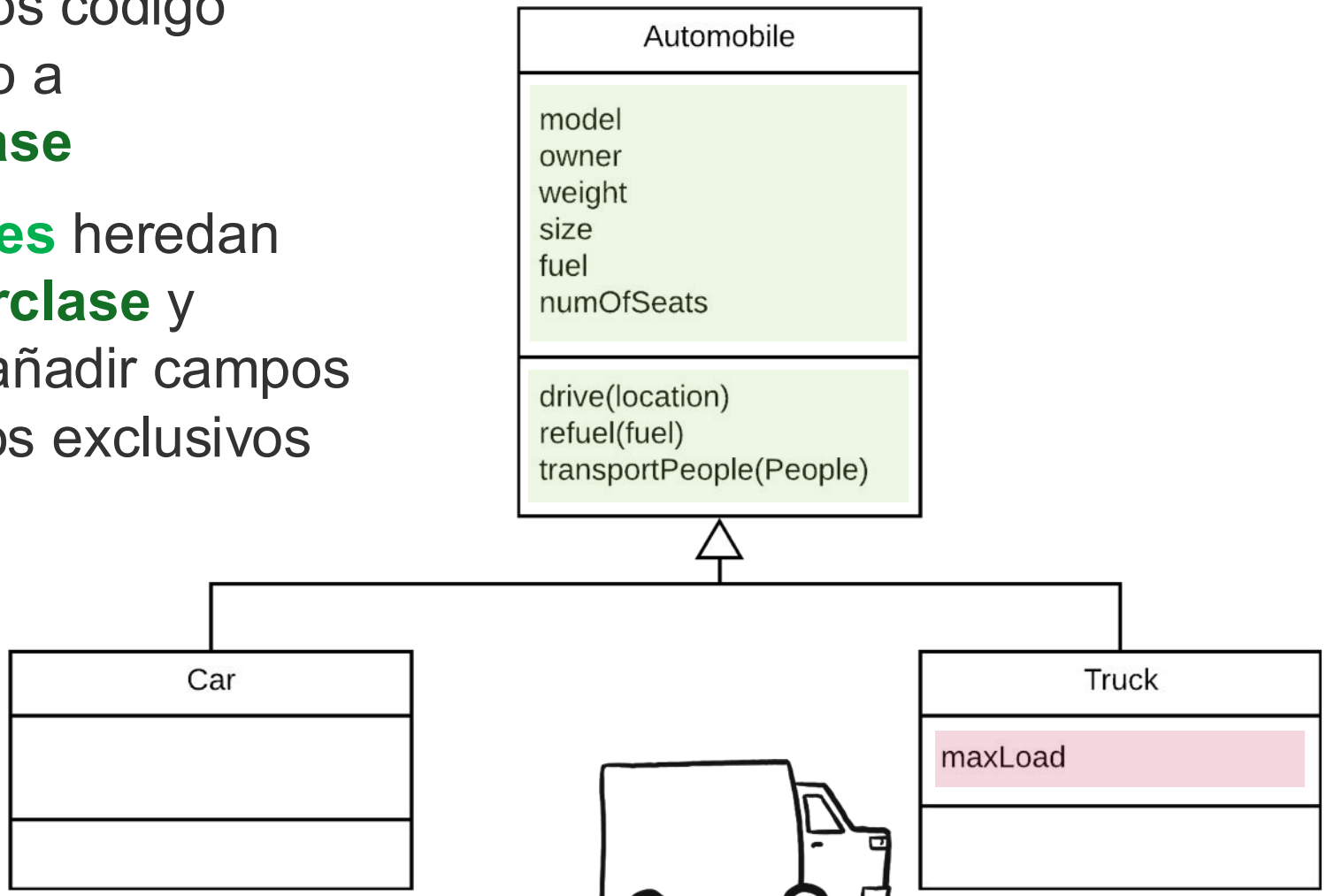


Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Herencia

- Extraemos código duplicado a **superclase**
- **Subclases** heredan de **superclase** y pueden añadir campos y métodos exclusivos



Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Herencia en Java

- Define un protocolo para la jerarquía de clases.
  - Se extrae todo el código común duplicado a una nueva clase denominada **superclase**
  - Todas las **subclases** heredan (**extends**) todos los campos y métodos NO PRIVADOS de la superclase
  - **subclases** mantienen o añaden características exclusivas
- Herencia es una calle de una sola dirección:
  - La **superclase** no sabe nada de sus **subclases** y por tanto tampoco sabe nada de los campos o métodos de sus **subclases**

– Con herencia:
  - Evitamos duplicación de código
  - Facilitamos reutilización de código, mantenimiento y extensibilidad

Herencia en Java

Superclase

```
1  public class Automobile {
2      private String model;
3      public void drive(int x, int y) { [...] }
4      public String getModel() { [...] }
5      public void setModel(String model) { [...] }
6  }
```

Subclase hereda **campos y métodos no privados** de la **superclase**

Subclase extends superclase

```
1  public class Car extends Automobile {
2      [...]
3  }
```



Subclases

```
1  public class Truck extends Automobile {
2      private int maxLoad;
3      [...]
4  }
```



Añade campos y métodos extras

Pearson

Copyright © [illegible] Pearson Education, Inc. All rights Reserved

Herencia en Java

```
1  public class Road {
2     public static void main(String[] args) {
3          Car newCar = new Car();
4          newCar.drive(10, 20);
5     }
6  }
```

Podemos usar el método **drive(...)** con Car porque lo ha heredado de Automobile

```
1  class Automobile {
2      public void drive(int x, int y) { [...] }
3  }
```

```
1  class Car extends Automobile { }
```



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Herencia en Java

```
1  public class Automobile {
2      public void drive(int x, int y) { [...] }
3  }
```

```
1  public class Car extends Automobile {
2     private boolean isConvertible;
3     [...]
4     public void sunshineMode() {
5          if (isConvertible) {
6                openRoof();
7          } else {
8                openWindow();
9          }
10    }
11    [...]
12 }
```

Car puede añadir sus propios métodos exclusivos



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

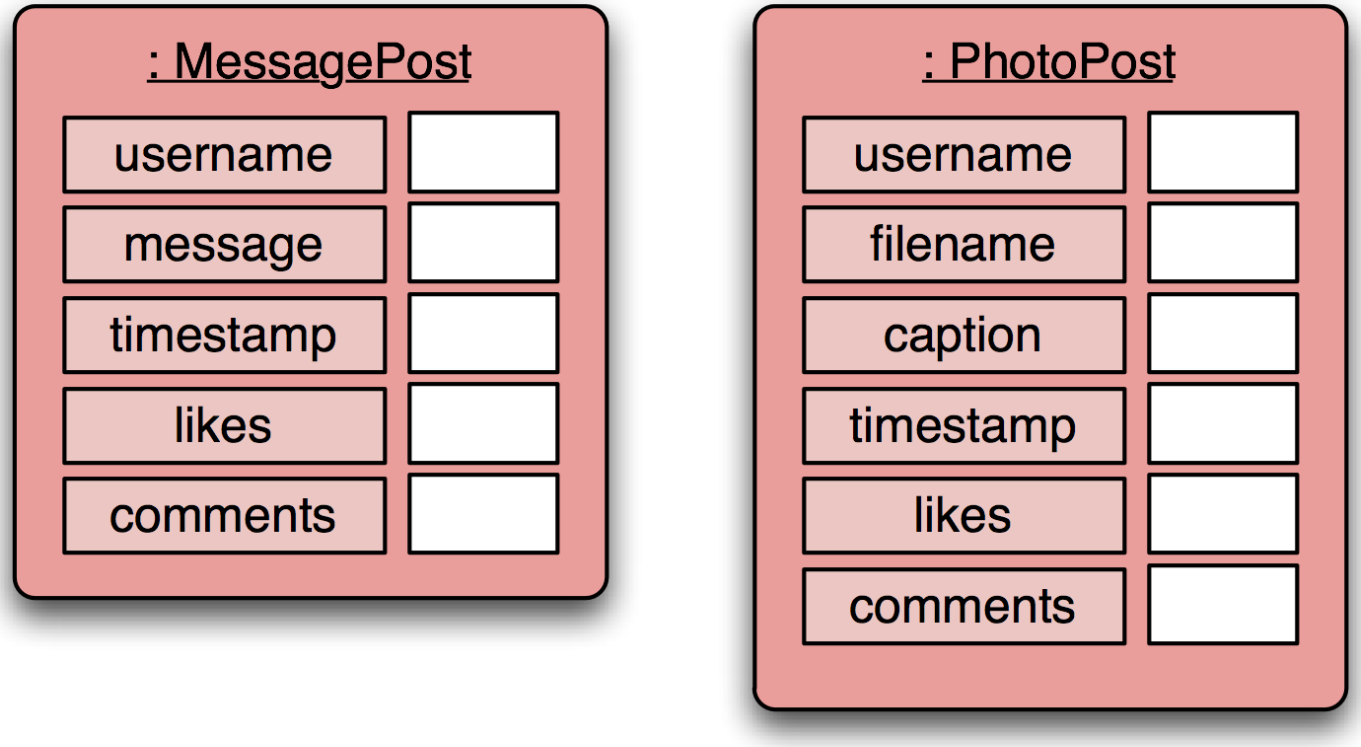
Proyecto “network v0” (Proyectos de ejemplo de Unidad 5)

Un pequeño prototipo de una red social:

- Con un gestor (NewsFeed) de publicaciones de tipo **texto** o **foto**:
  - `MessagePost`: mensaje de texto multilínea
  - `PhotoPost`: foto y pie de foto (caption).
- Permite operaciones como buscar (search), mostrar (display) y eliminar (remove).

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

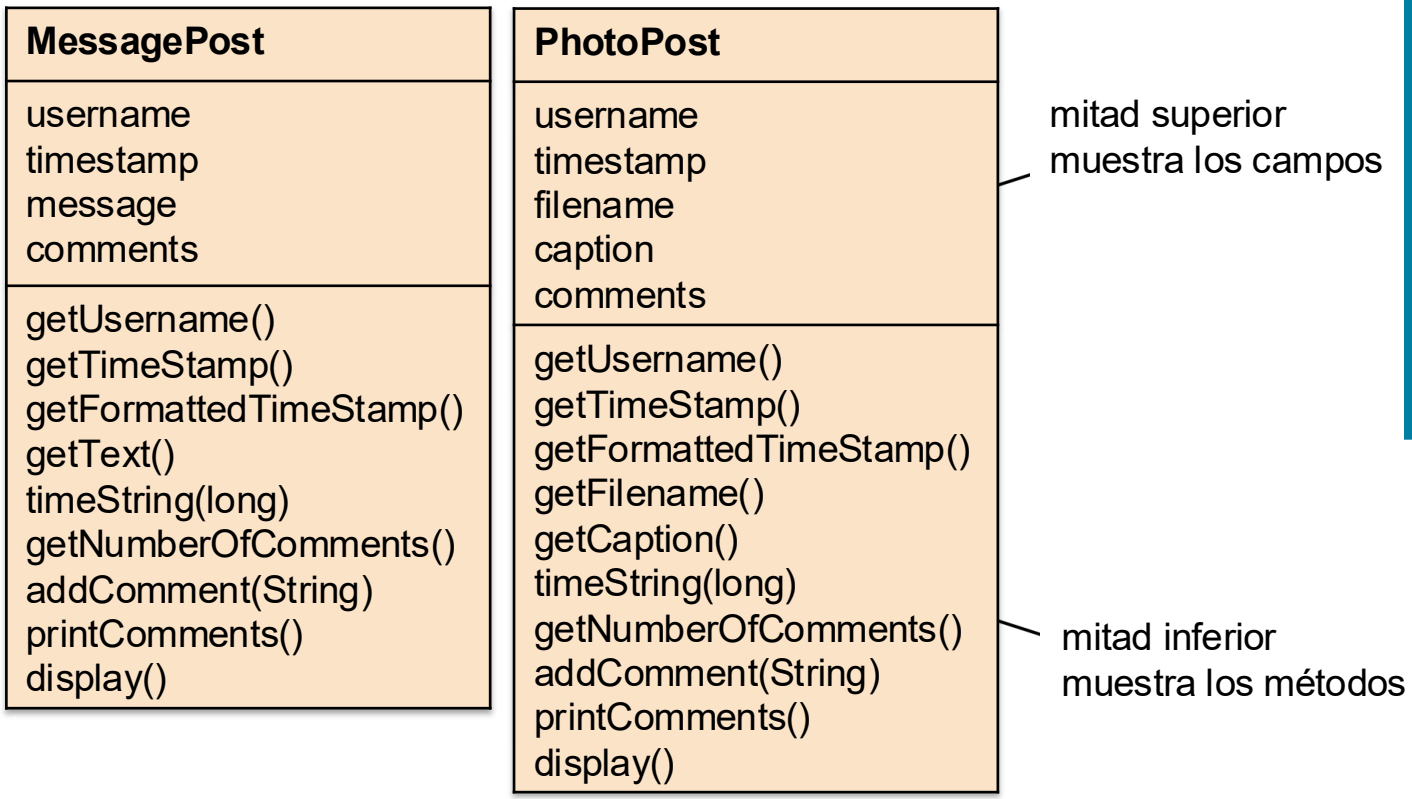
Proyecto “network v0”
Objetos



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

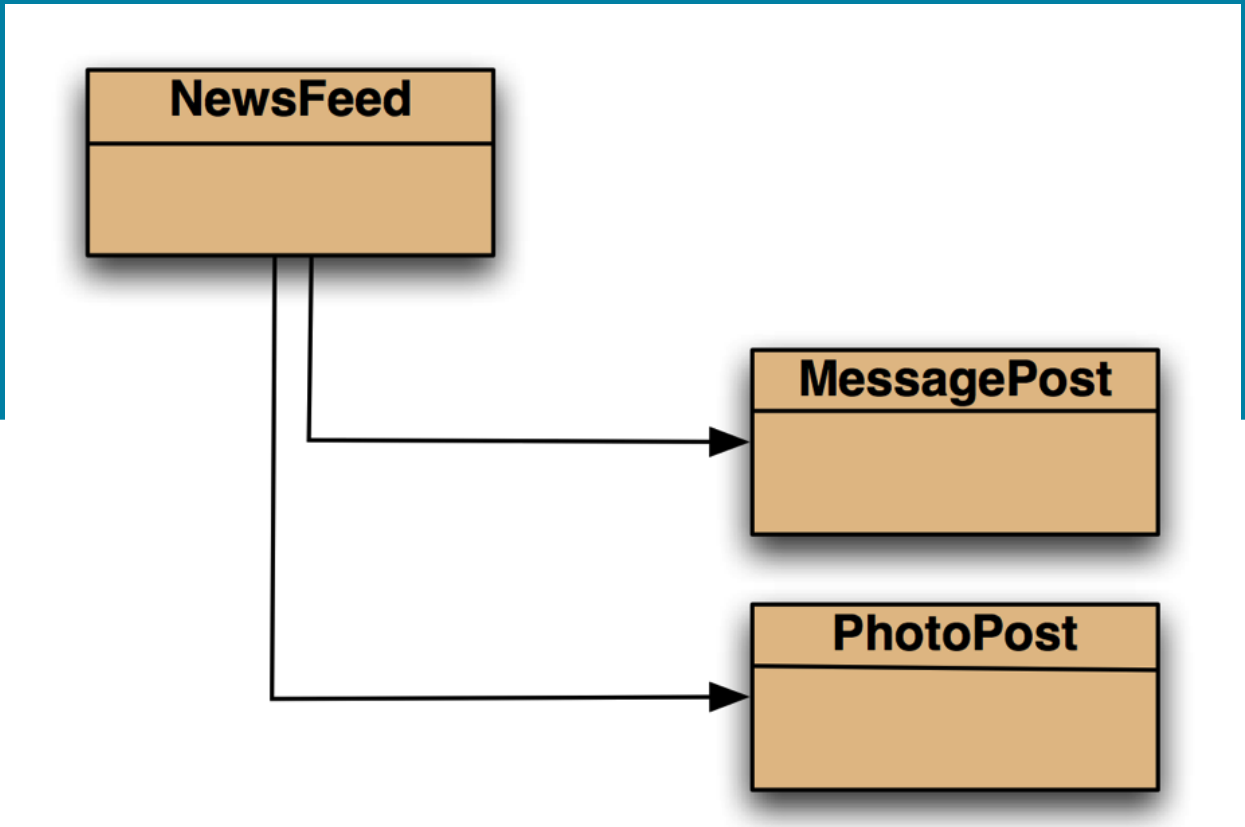
Proyecto “network v0”
Clases



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

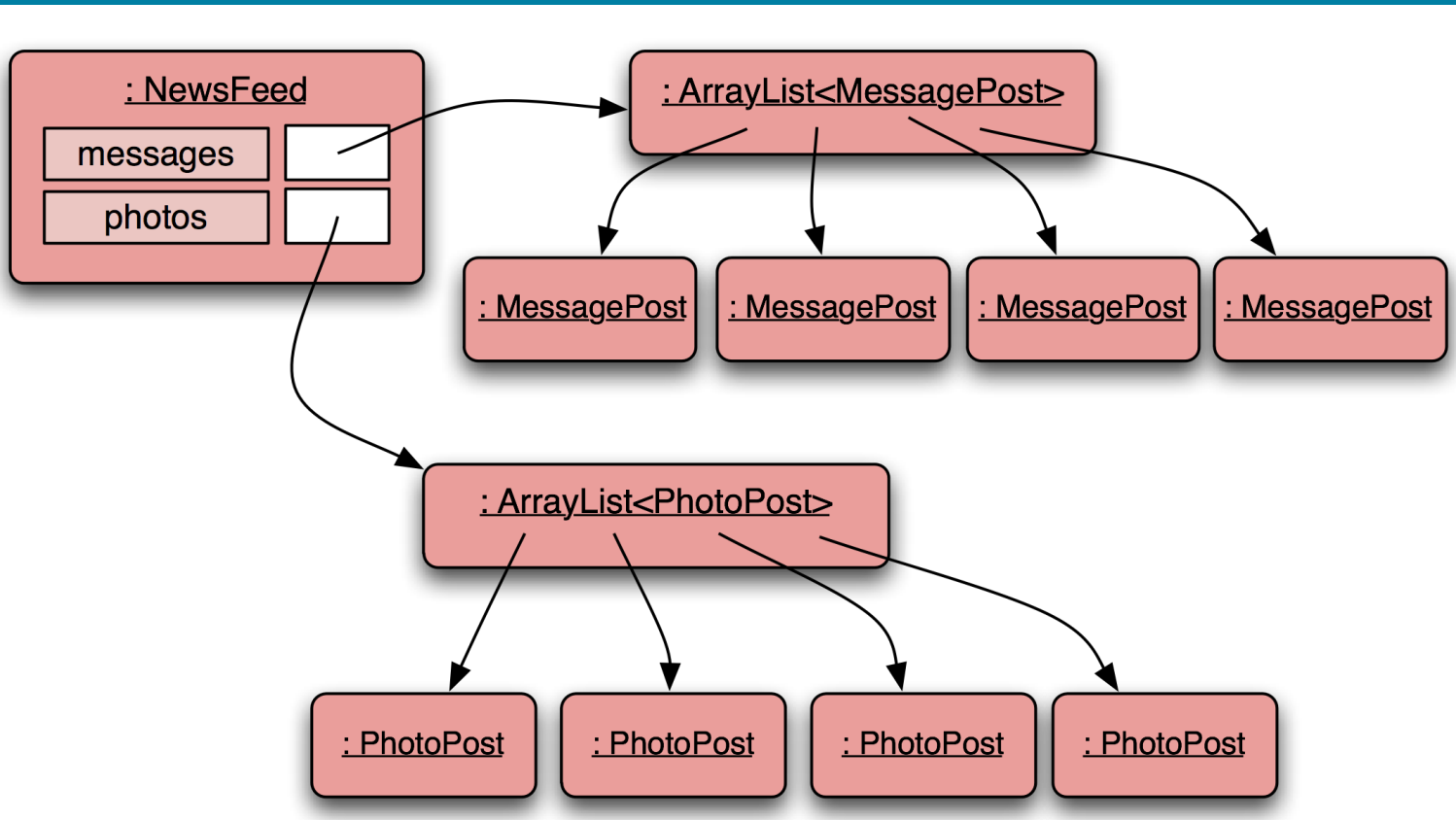
Proyecto “network v0”
Diagrama de clases



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

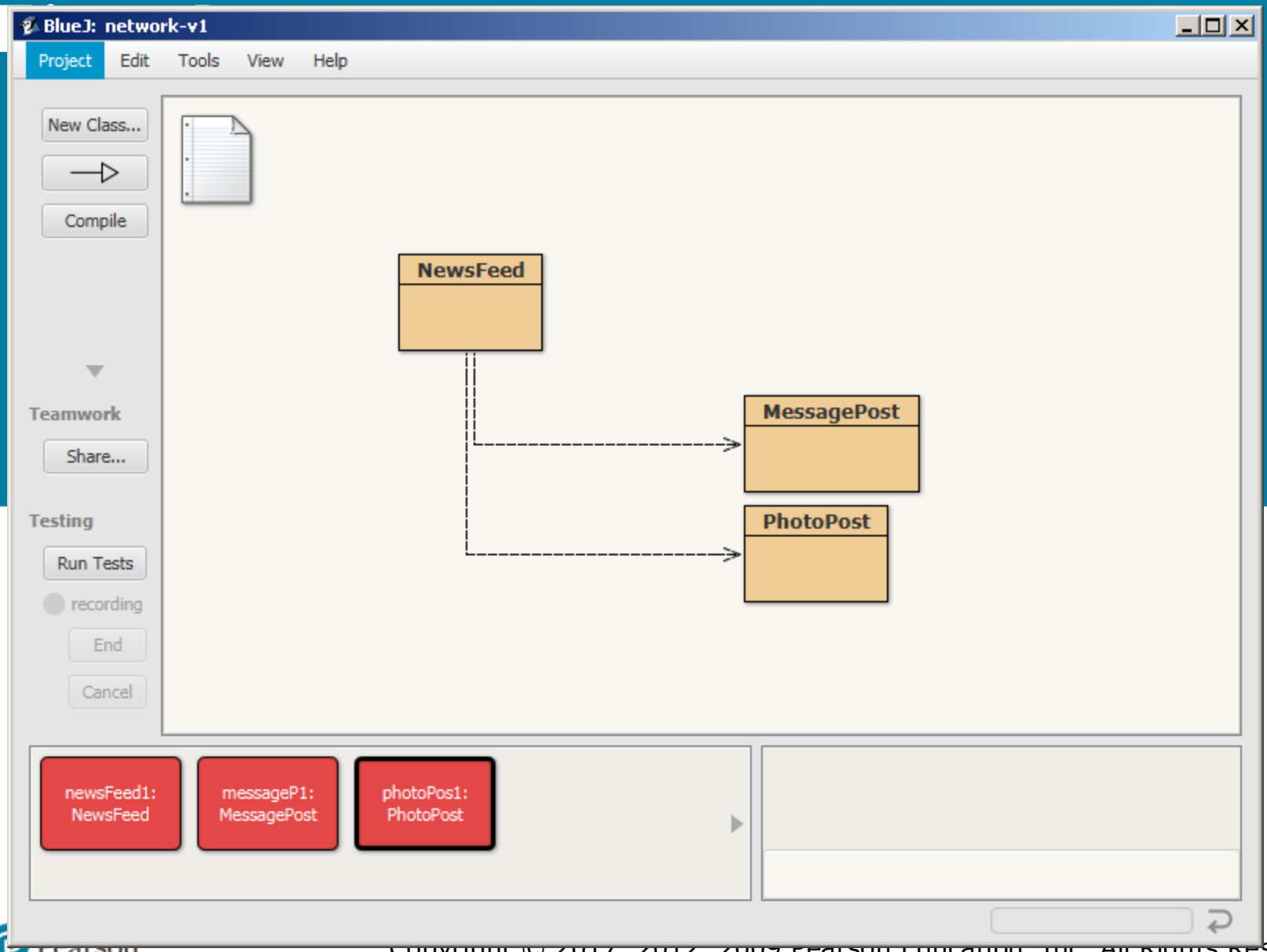
Proyecto “network v0”
Diagrama de objetos



Pearson

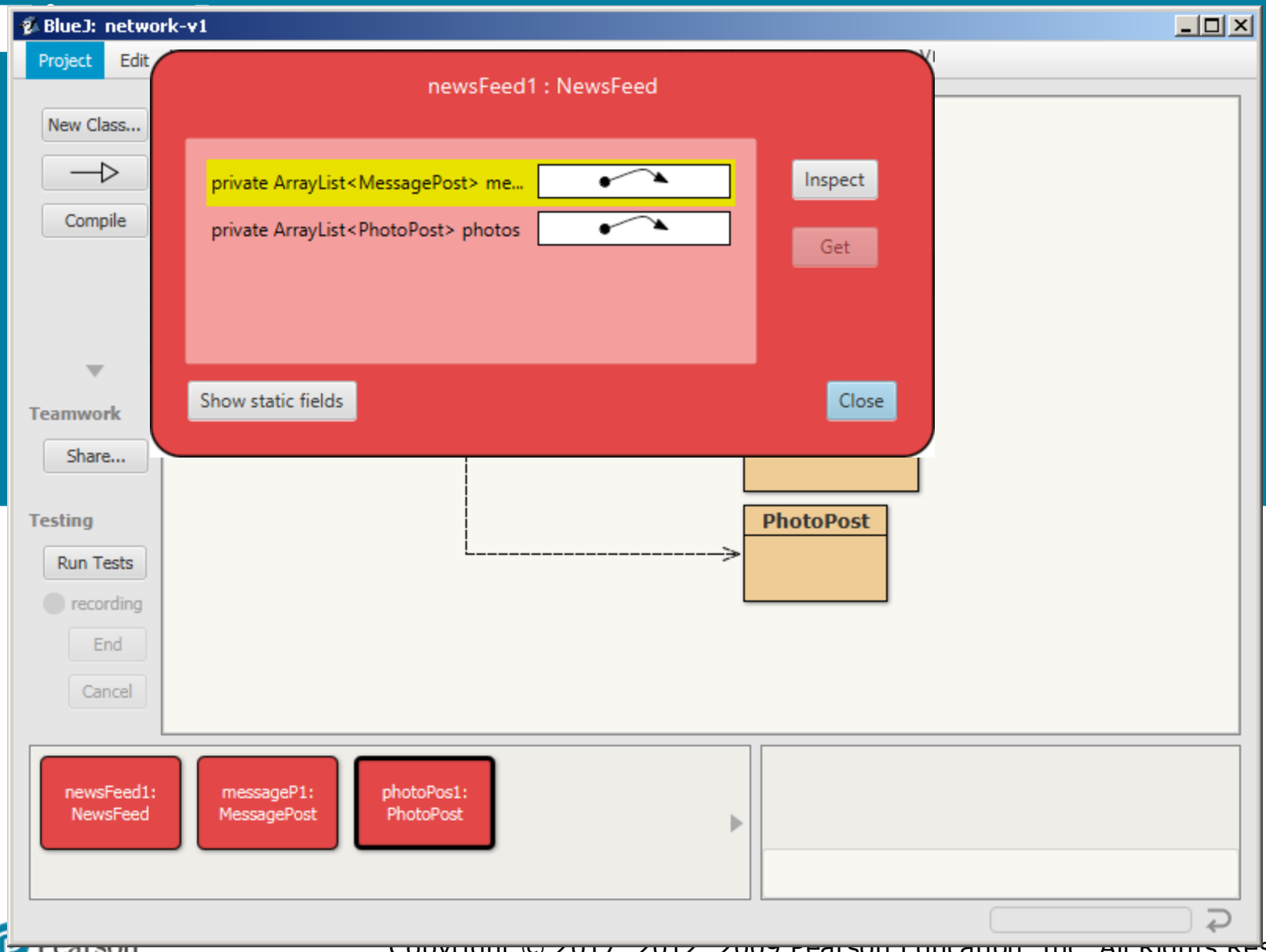
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Proyecto “network v0”



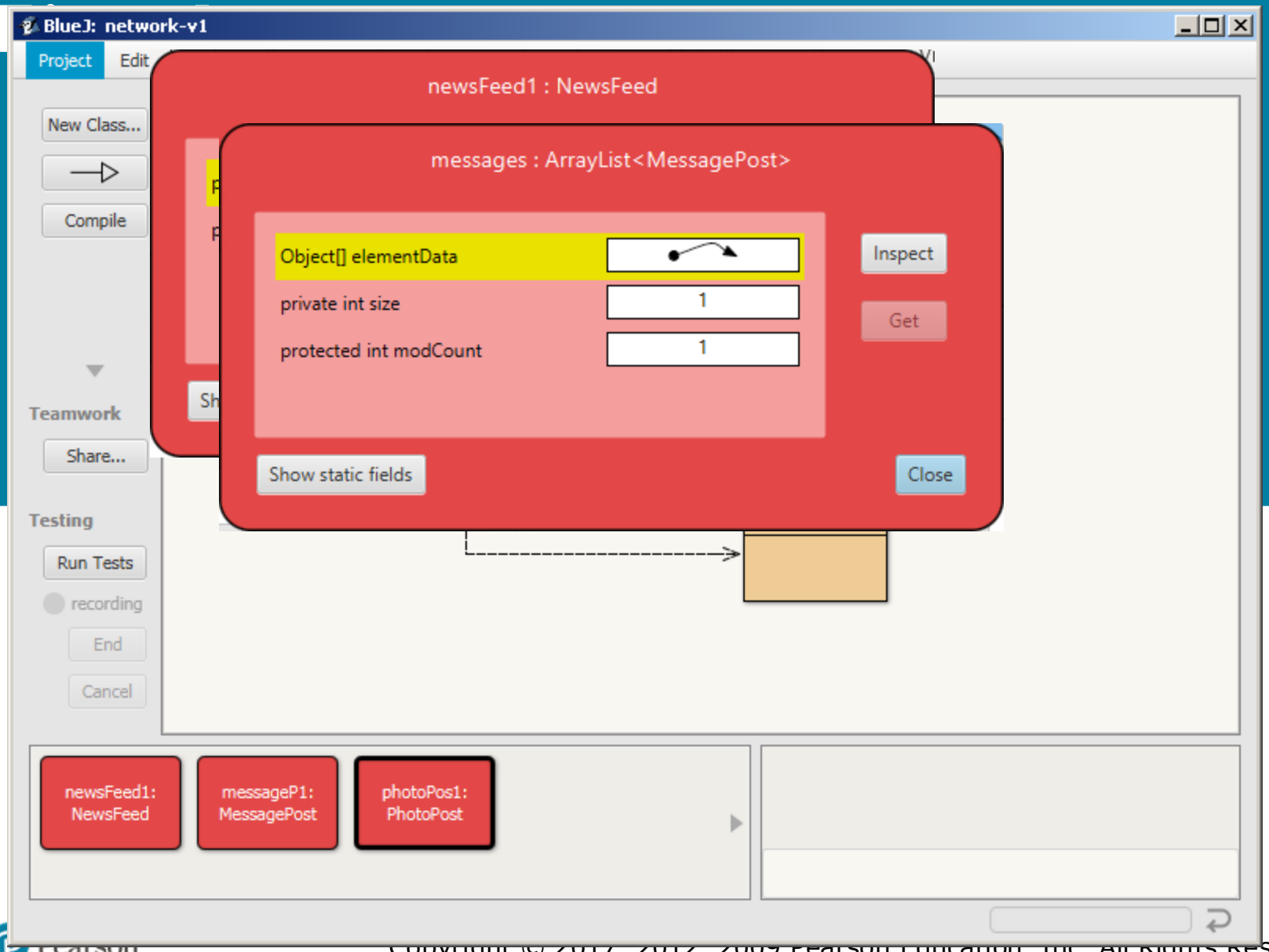
Pearson Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Proyecto “network v0”



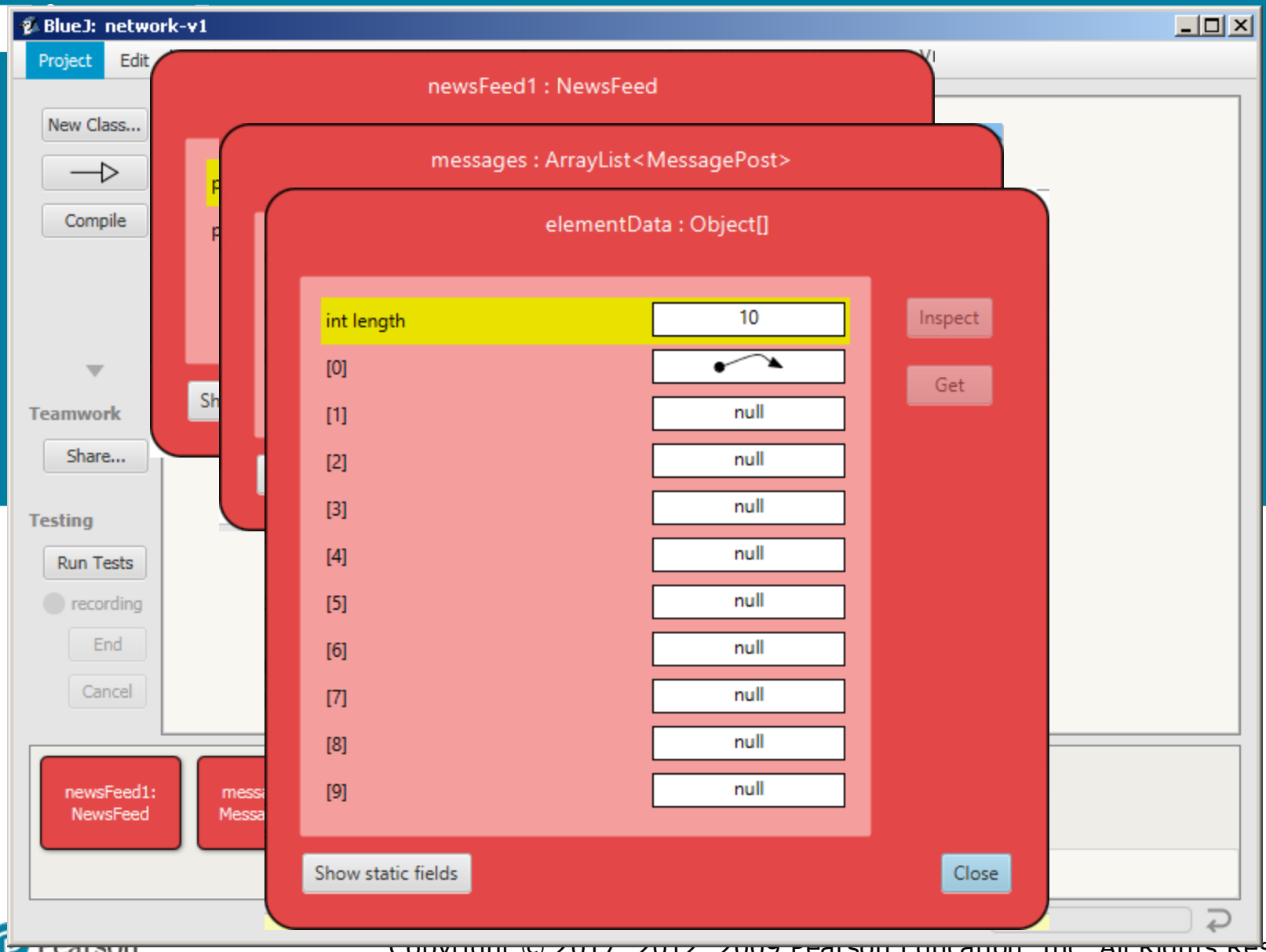
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Proyecto “network v0”



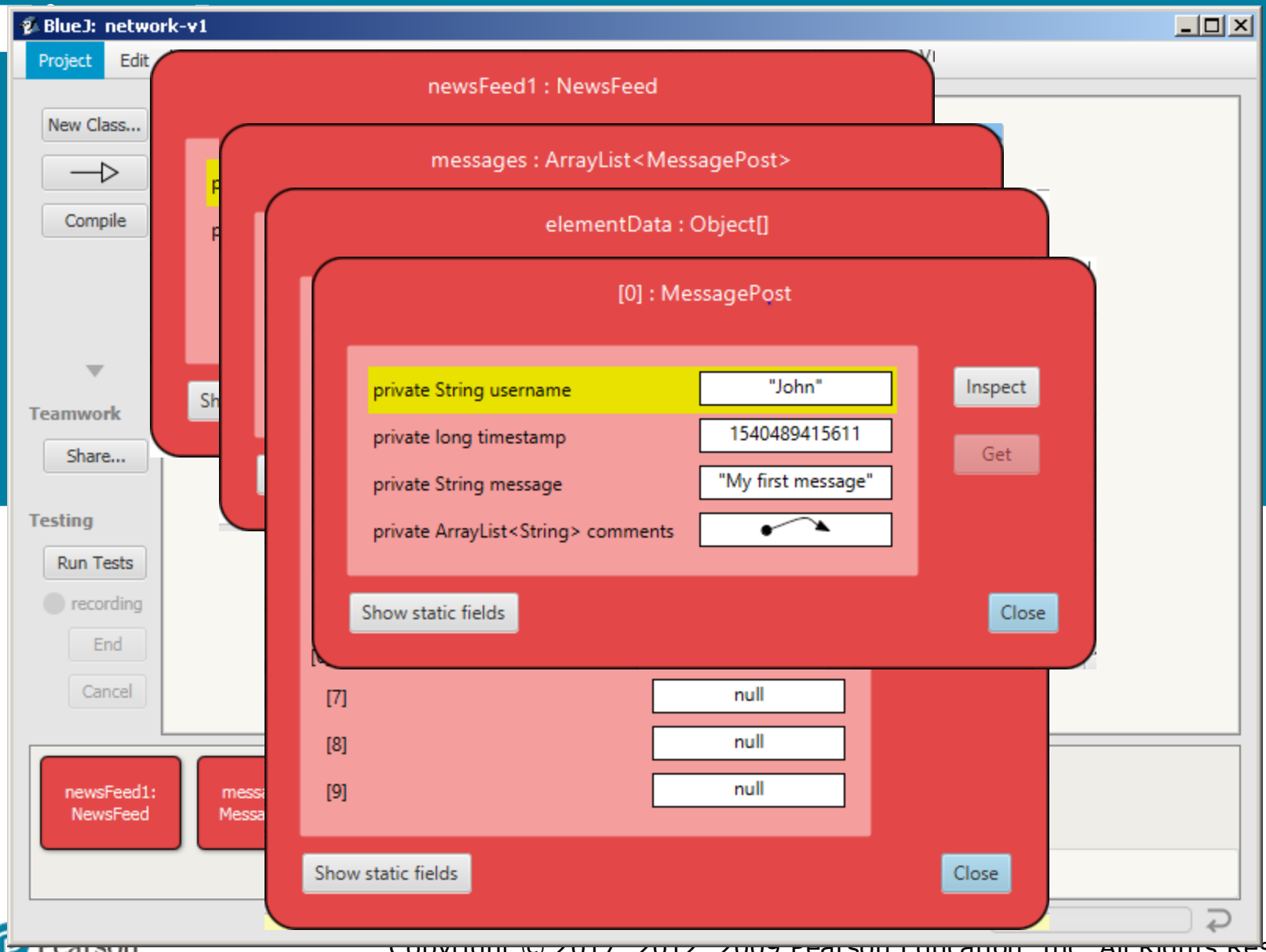
Pearson Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Proyecto “network v0”



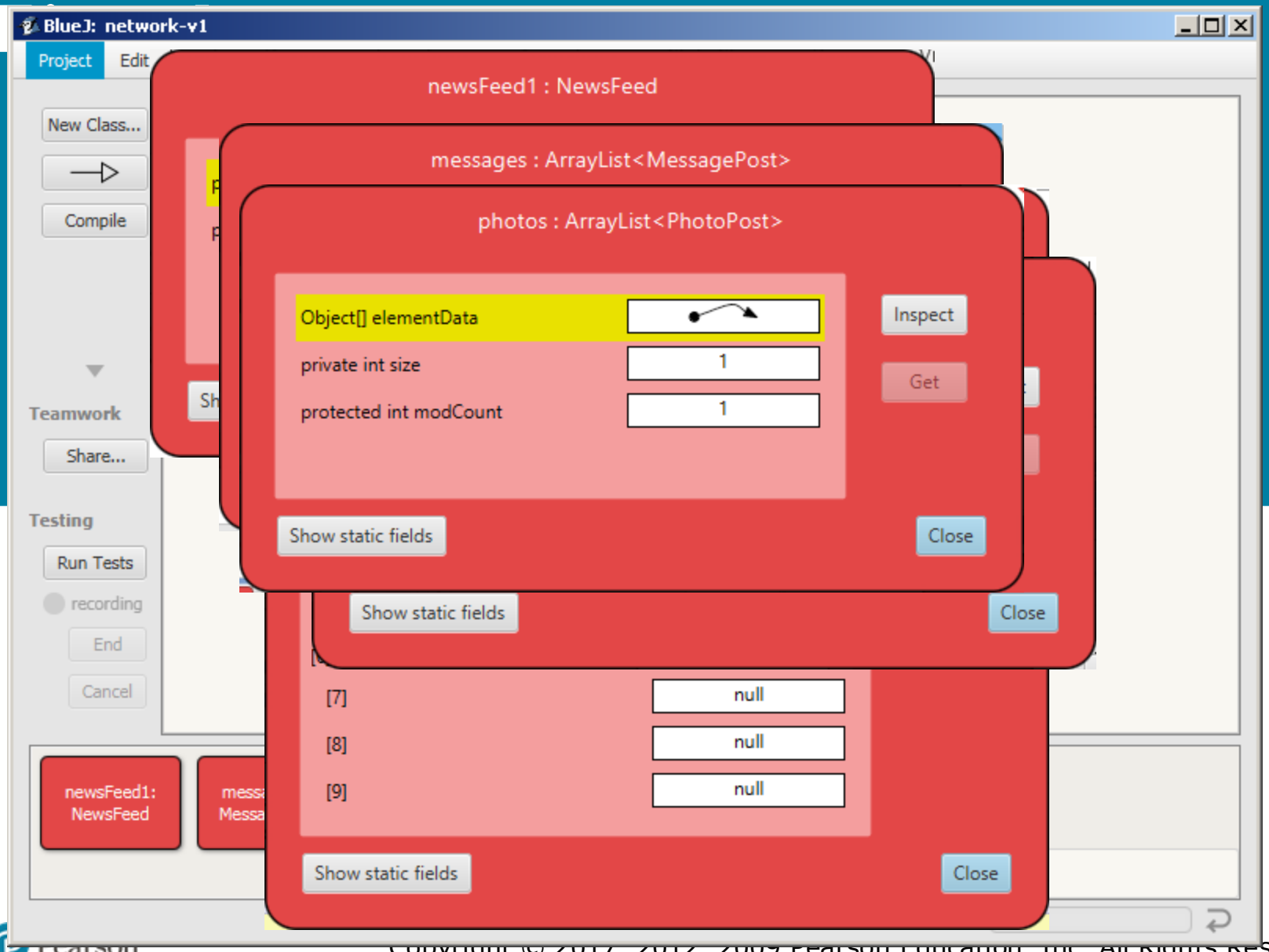
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Proyecto “network v0”



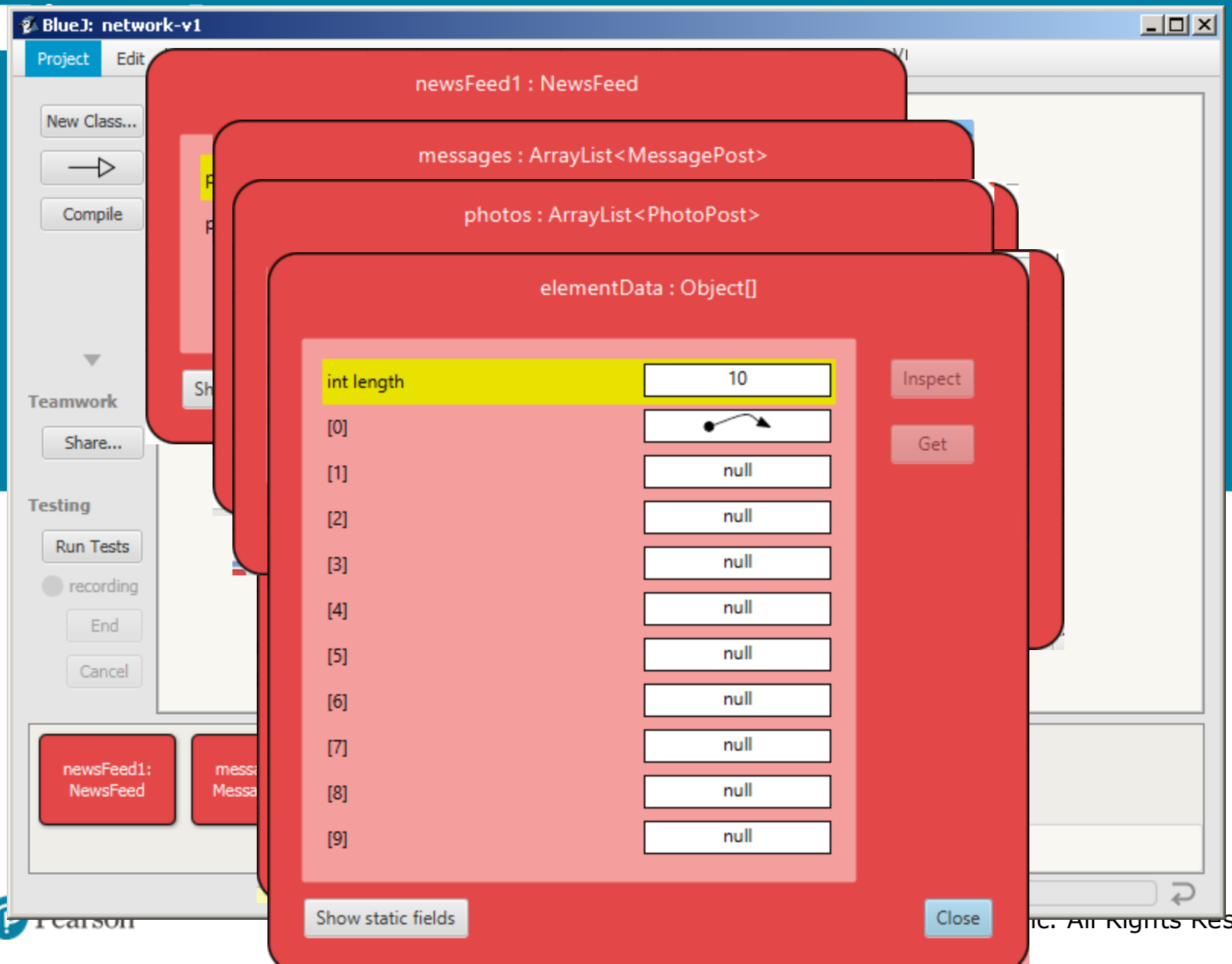
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Proyecto “network v0”



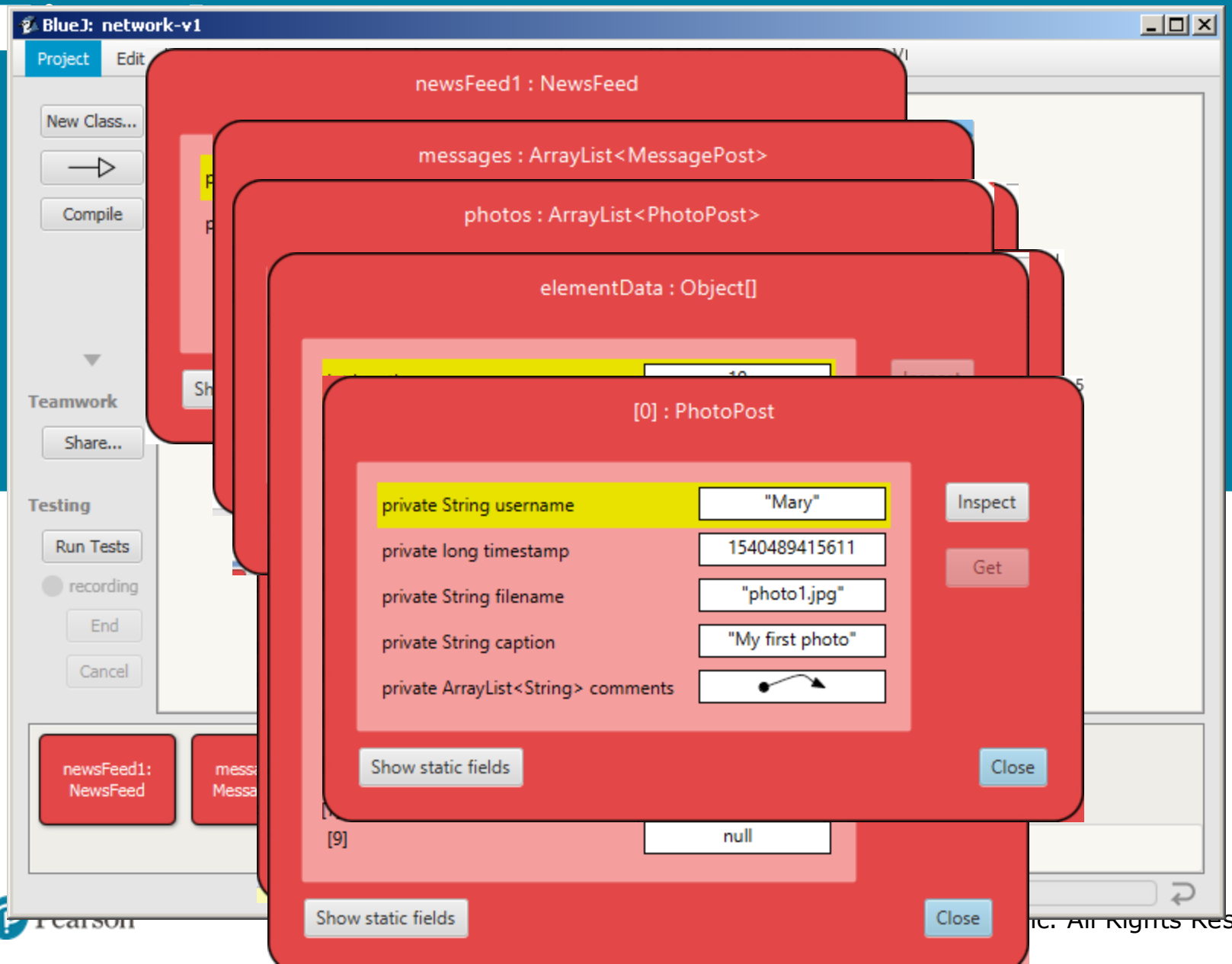
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Proyecto “network v0”



Pearson
ic. All Rights Reserved

Proyecto “network v0”



Pearson
ic. All Rights Reserved

Proyecto “network v0”
Extracto del Código Fuente de Message-Post

```
public class MessagePost
{
    private String username;
    private long timestamp;
    private String message;
    private List<String> comments;

    public MessagePost(String author, String text)
    {
          username = author;
          timestamp = System.currentTimeMillis();
          message = text;
          comments = new ArrayList<>();
    }

    public string getText () ...

    ...

    public void display() ...

    ...
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Proyecto “network v0”
Extracto del Código Fuente de Photo-Post

```
public class PhotoPost
{
   private String username;
   private long timestamp;
   private String filename;
   private String caption;
   private List<String> comments;

   public PhotoPost(String author, String filename,
                    String caption)
   {
         username = author;
         this.filename = filename;
         this.caption = caption;
         timestamp = System.currentTimeMillis();
         comments = new ArrayList<>();
   }

   public void getFilename() ...
   …
   public void display() …
   ...
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Proyecto “network v0”
Extracto del Código Fuente de NewsFeed

```
public class NewsFeed
{
    private List<MessagePost> messages;
    private List<PhotoPost> photos;
    ...
    public void show()
    {
        for(MessagePost message : messages) {
            message.display();
            System.out.println("---------"); }

        for(PhotoPost photo : photos) {
            photo.display();
            System.out.println("---------"); }
    }
}
```

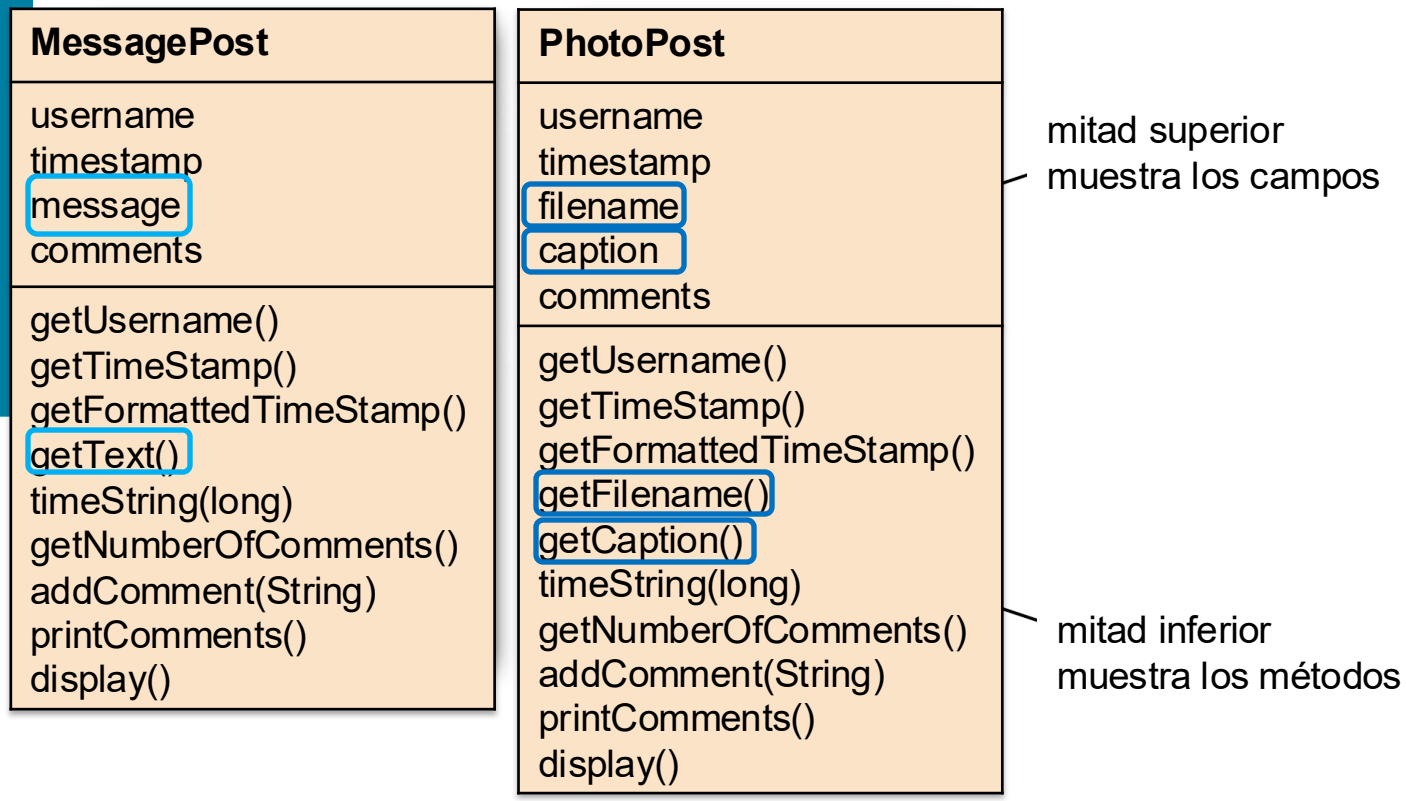
Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Proyecto “network v0”
Críticas a este proyecto

- Código duplicado:
  - `MessagePost` y `PhotoPost` son clases muy similares:
    - con partes completamente idénticas
    - hace más difícil el mantenimiento
    - aumenta la probabilidad de errores con un mantenimiento incorrecto
- Código duplicado también en `NewsFeed`.

Proyecto “network v0”
¡¡Herencia al rescate!!
Examinemos de nuevo las clases



- Tenemos mucho código duplicado en MessagePost y PhotoPost porque tienen muchas similitudes
- En realidad solo se diferencian en los campos y métodos resaltados

Proyecto “network v0”
¡¡Herencia al rescate!!

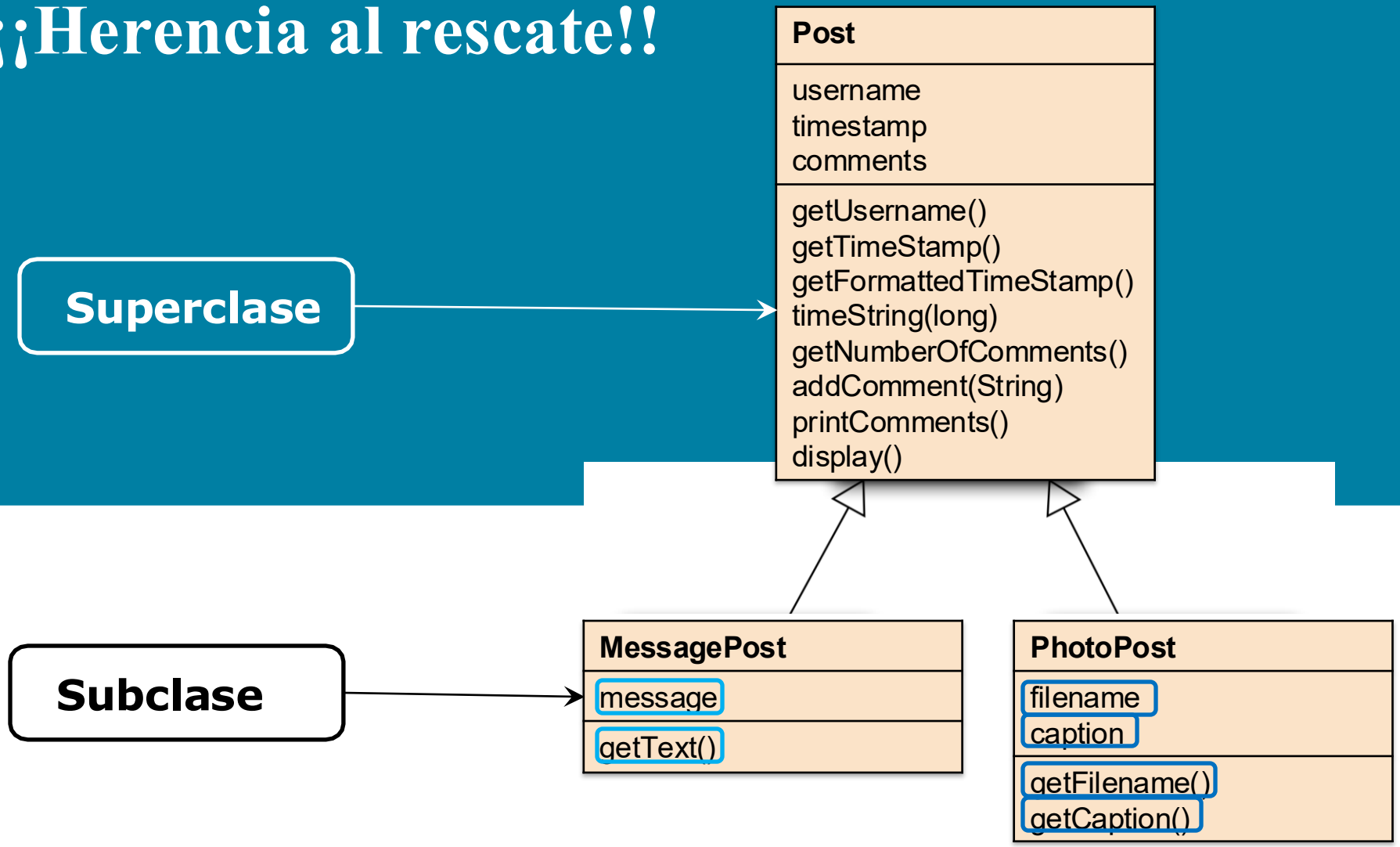
- definimos una **superclase**: `Post`
- definimos **subclases** para `MessagePost` y `PhotoPost`
- **superclase** define campos y métodos comunes
- **subclases** **heredan** todos los campos y métodos NO PRIVADOS de **superclase**
- **subclases** mantienen o añaden otras características

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

“network-v1-with-herencia-without-polymorphism”

¡¡Herencia al rescate!!



- Una publicación de foto (`PhotoPost`) es una publicación (`Post`)
- Una publicación de texto (`MessagePost`) es una publicación (`Post`)

“network-v1-with-herencia-without-polymorphism”

¡¡Herencia al rescate!!

¡¡OJO: superclase nunca sabe si hay clases heredando de ella!!

```
public class Post
{
    ...
}
```

cabecera de la superclase es igual que una clase normal

```
public class PhotoPost extends Post
{
    ...
}
```

```
public class MessagePost extends Post
{
    ...
}
```

En las subclase sí especificamos **extends**

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

“network-v1-with-herencia-without-polymorphism”
Superclase contiene campos y métodos comunes

```
public class Post
{
    private String username;
    private long timestamp;
    private List<String> comments;

    // constructor and methods omitted.
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

“network-v1-with-herencia-without-polymorphism”
Subclases contienen campos exclusivos

```
public class MessagePost extends Post
{
    private String message;

    // constructor and methods omitted.
}
```

```
public class PhotoPost extends Post
{
    private String filename;
    private String caption;

    // constructor and methods omitted.
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

“network-v1-with-herencia-without-polymorphism”
Subclases contienen métodos exclusivos

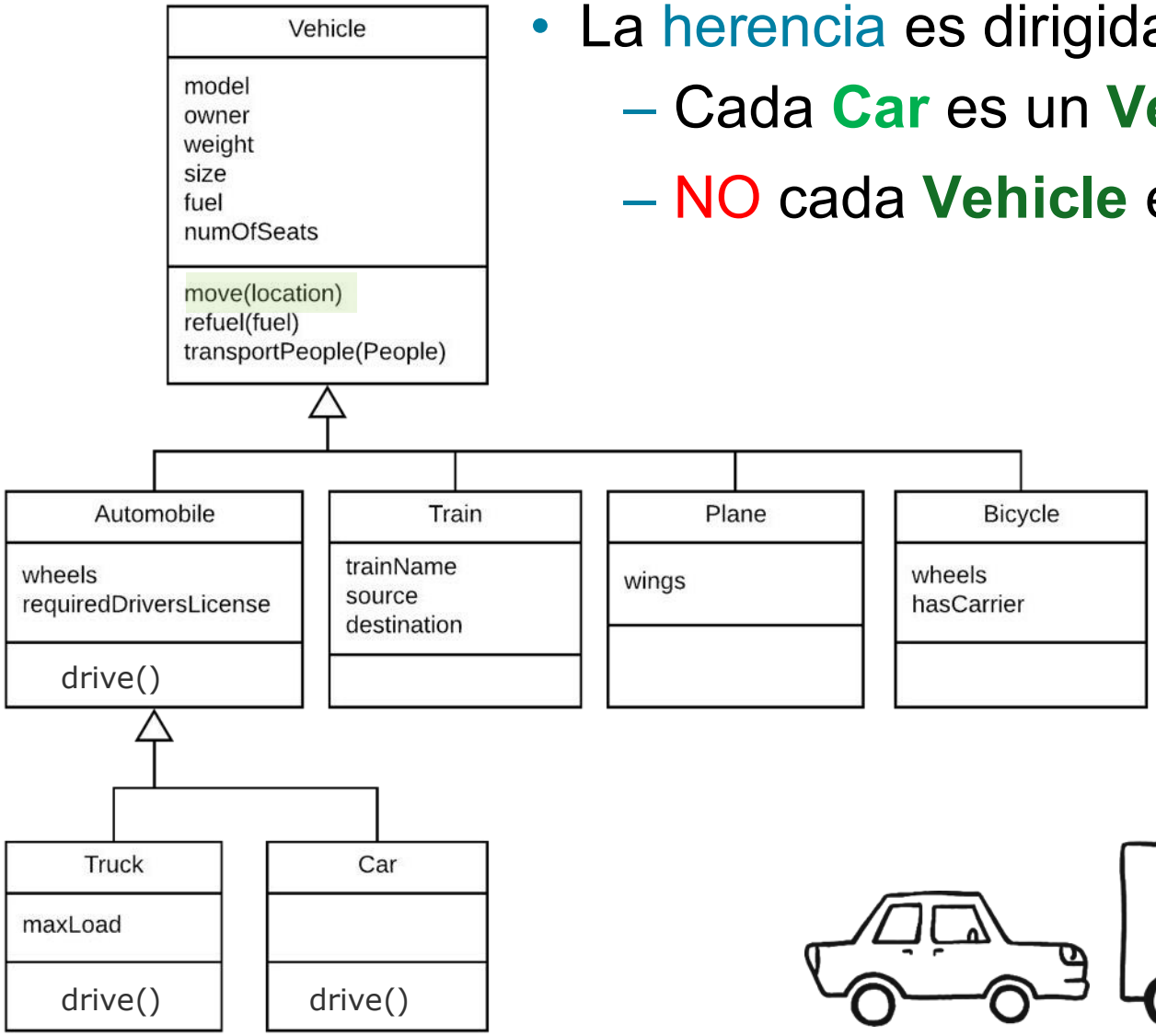
```
public class MessagePost extends Post
{
    …
    public String getText(){
        return message;
    }
}
```

```
public class PhotoPost extends Post
{ …
    public String getFilename(){
        return filename;
    }
    public String getCaption(){
        return caption;
    }
}
```

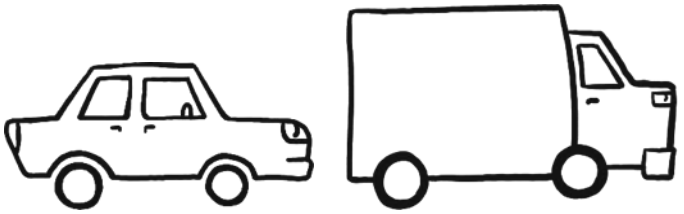
Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Herencia en Java



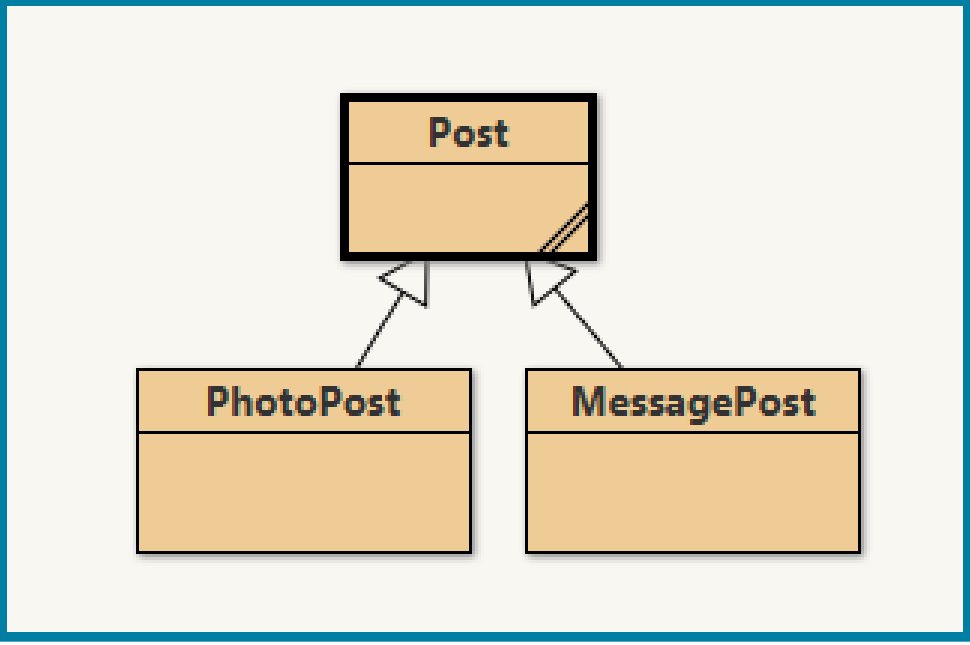
- La herencia es dirigida unidireccional.
  - Cada **Car** es un **Vehicle**
  - NO cada **Vehicle** es un **Car**



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

“network-v1-with-herencia-without-polymorphism”



- ✓ La herencia es dirigida unidireccional:
  - ✓ Cada `MessagePost` y `PhotoPost` es un `Post`
  - ✓ NO cada `Post` es un `MessagePost` o un `PhotoPost`

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales

- Herencia
  - Constructores
- Polimorfismo
  - Subtipos y Sustitución
  - La Clase Object
  - Colecciones Polimórficas
  - Casting
  - Tipo Estático y Dinámico
  - Sobreescritura
  - En Métodos
- StringBuilder
- Clases y métodos abstractos
- Métodos de la clase Object
  - toString
  - equals
  - hashCode
- Extendiendo jerarquías de clases
- Herencia múltiple con Interfaces
- Acceso Protegido (protected)
- La clase “Class”

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Herencia y Constructores

- En el constructor de la superclase:
  - se inicializan los campos comunes
- En el constructor de las subclases:
  - DEBE ser la primera sentencia en el constructor la llamada al constructor de la superclase (`super`)
    - Si no se implementa un constructor en la subclase (no recomendado):
      - El compilador inserta uno sin parámetros
      - Solo compila si la superclase tiene un constructor sin parámetros
  - DEBE tomar valores para todos los campos a inicializar

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

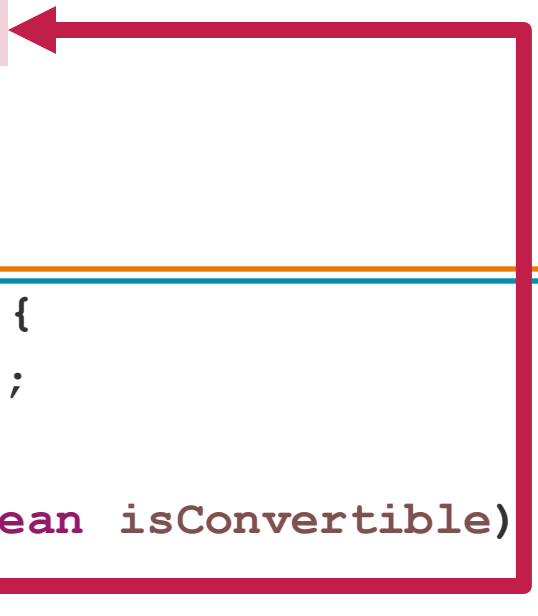
Herencia y Constructores

`super(...)`

Llama al constructor de la superclase

```
1  public class Automobile {
2     private String model;
3     public Automobile(String model) {
4          this.model = model;
5     }
6  }
```

```
1  public class Car extends Automobile {
2          private boolean isConvertible;
3
4          public Car(String model, boolean isConvertible) {
5                  super(model);
6                  this.isConvertible = isConvertible;
7          }
8  }
```



“network-v1-with-herencia-without-polymorphism”
Herencia y Constructores – En la superclase:

```
public class Post
{
    private String username;
    private long timestamp;
    private List<String> comments;

    /**
     * Initialise the fields of the post.
     */
    public Post(String author)
    {
        username = author;
        timestamp = System.currentTimeMillis();
        comments = new ArrayList<String>();
    }

    // methods omitted
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

"network-v1-with-herencia-without-polymorphism"
Herencia y Constructores – En las subclases:

```
public class MessagePost extends Post
{
    private String message;

    /**
     * Constructor for objects of class MessagePost
     */
    public MessagePost(String author, String text)
    {
        super(author);// DEBE ser la primera sentencia
        message = text;
    }

    // methods omitted
}
```

- la subclase DEBE llamar al constructor de la superclase
- DEBE tomar valores para todos los campos a inicializar

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales

- Herencia
  - Constructores
- Polimorfismo
  - Subtipos y Sustitución
  - La Clase Object
  - Colecciones Polimórficas
  - Casting
  - Tipo Estático y Dinámico
  - Sobreescritura
  - En Métodos
- StringBuilder
- Clases y métodos abstractos
- Métodos de la clase Object
  - toString
  - equals
  - hashCode
- Extendiendo jerarquías de clases
- Herencia múltiple con Interfaces
- Acceso Protegido (protected)
- La clase “Class”

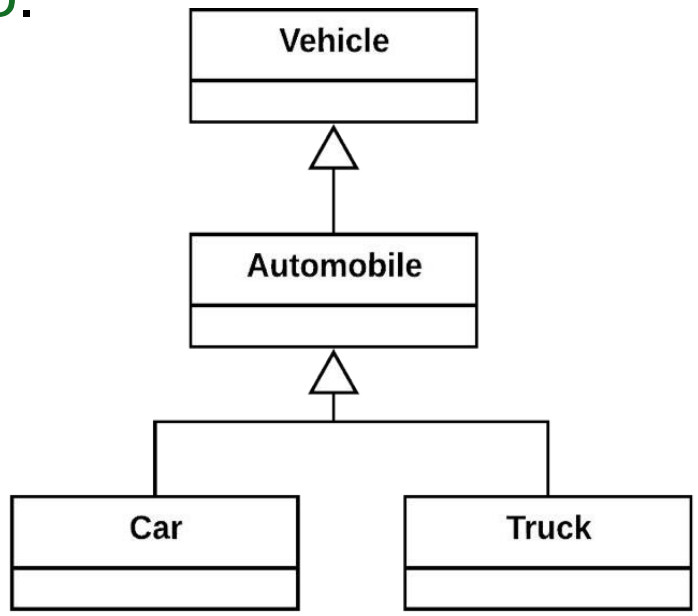
Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Polimorfismo



- Clases definen tipos.
- Subclases definen **subtipos**.
- Instancias de los **subtipos** se pueden usar cuando se requieren instancias del supertipo.
  - Se denomina **sustitución**

```
Vehicle v1 = new Vehicle();
Vehicle v2 = new Car();
Vehicle v3 = new Truck ();
```

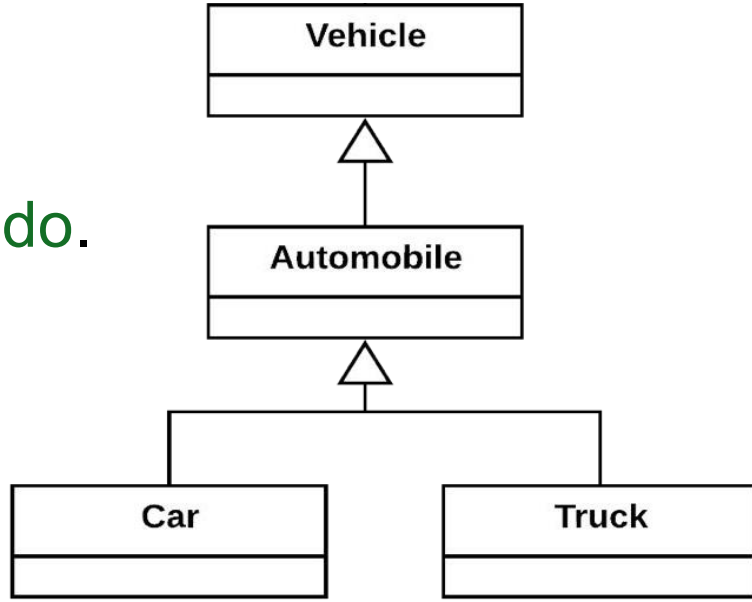


Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Polimorfismo

- Variables de objetos en Java son **polimórficas**. (Pueden mantener objetos de más de un tipo)
- Pueden mantener objetos:
  - del tipo declarado
  - de los subtipos del tipo declarado.

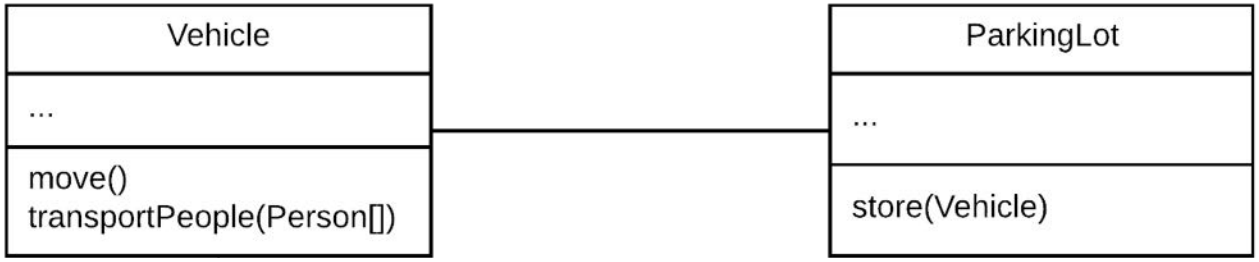



```
Vehicle v1 = new Vehicle();
Vehicle v2 = new Car();
Vehicle v3 = new Truck ();
```

Variables del supertipo Vehicle pueden mantener objetos de subtipos Car/Truck (y Automobile)

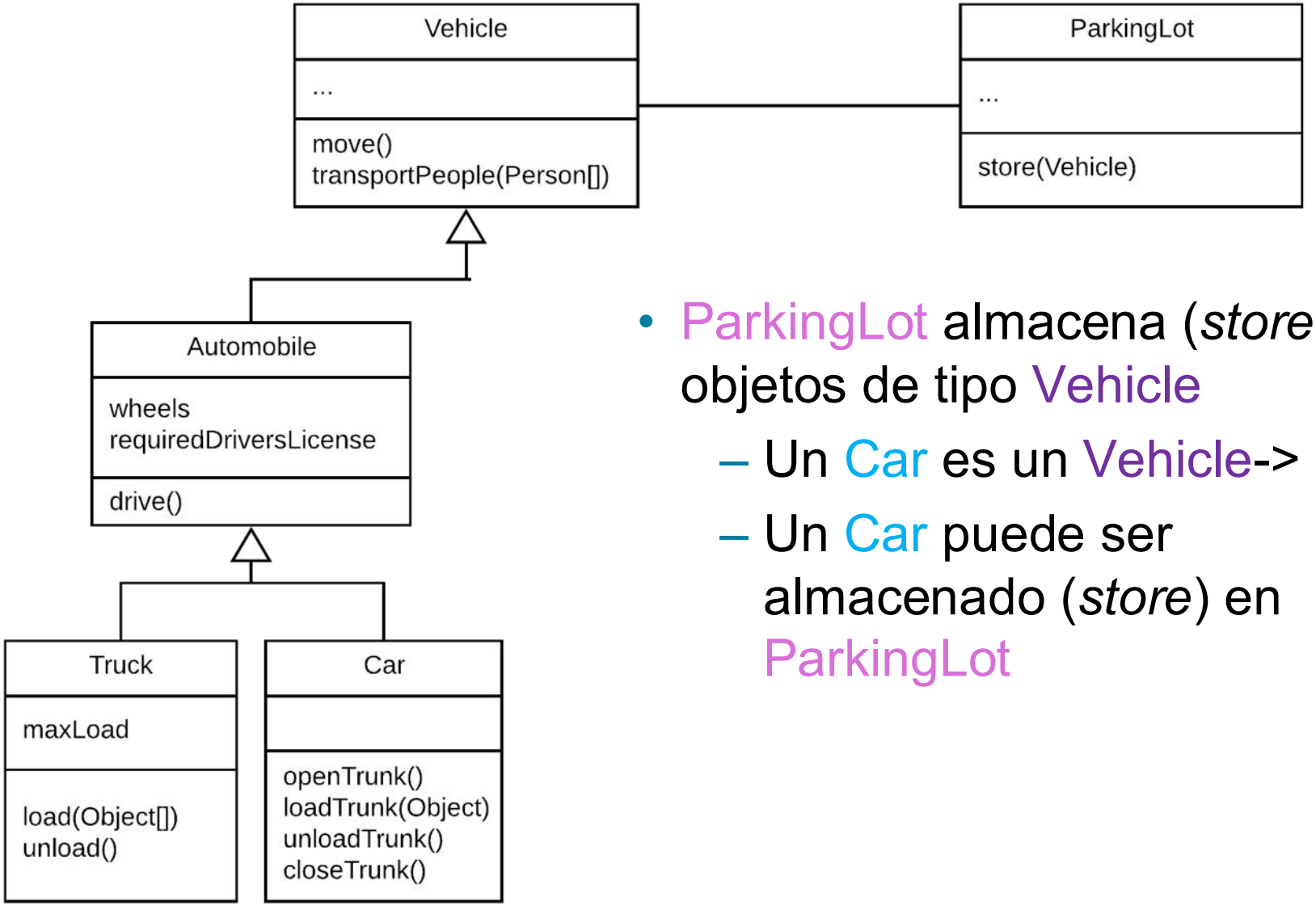
Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Polimorfismo



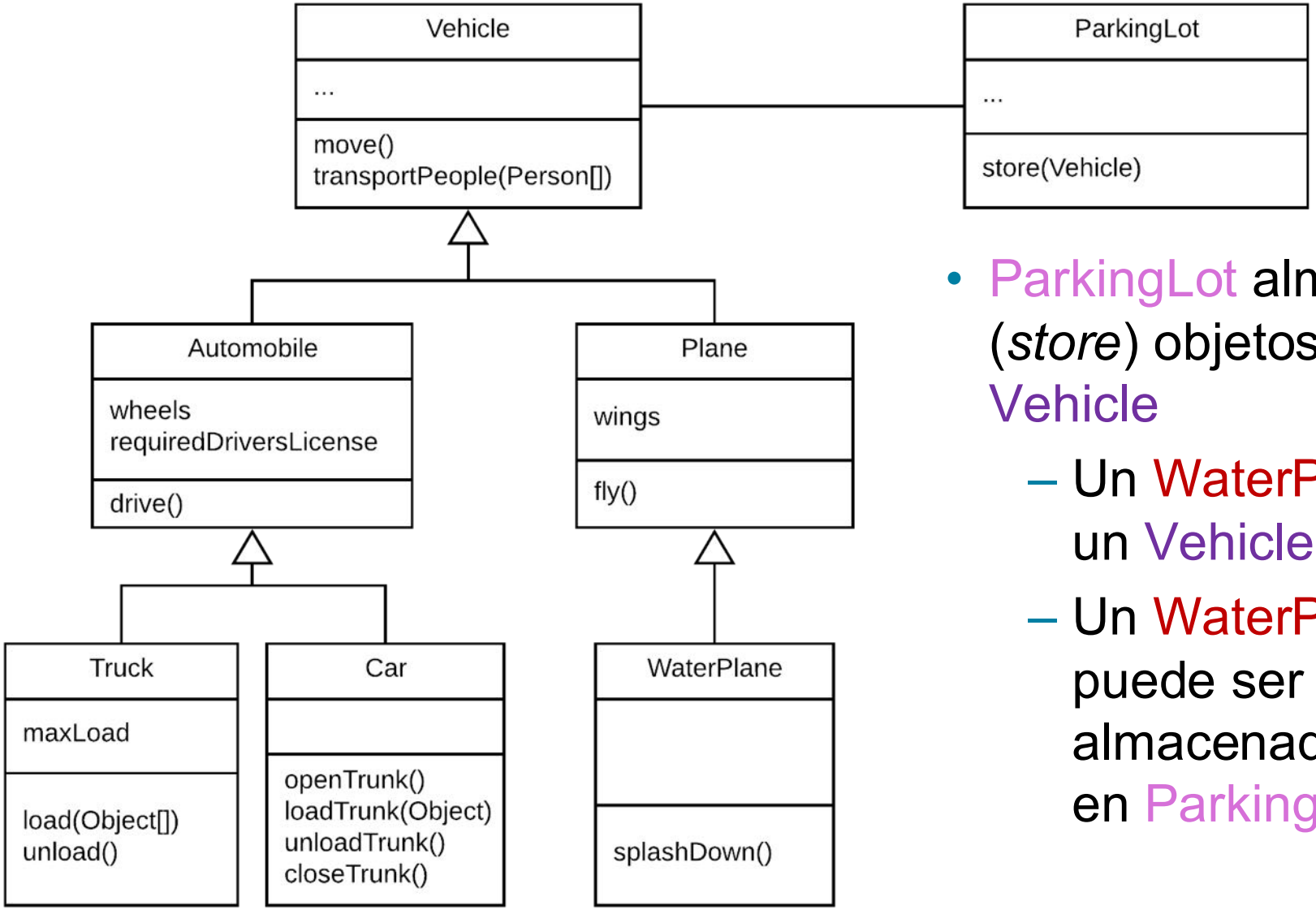
- ParkingLot ofrece plazas de aparcamiento para objetos de tipo Vehicle
- ParkingLot almacena (*store*) objetos de tipo Vehicle

Polimorfismo



- ParkingLot almacena (*store*) objetos de tipo Vehicle
  - Un Car es un Vehicle->
  - Un Car puede ser almacenado (*store*) en ParkingLot

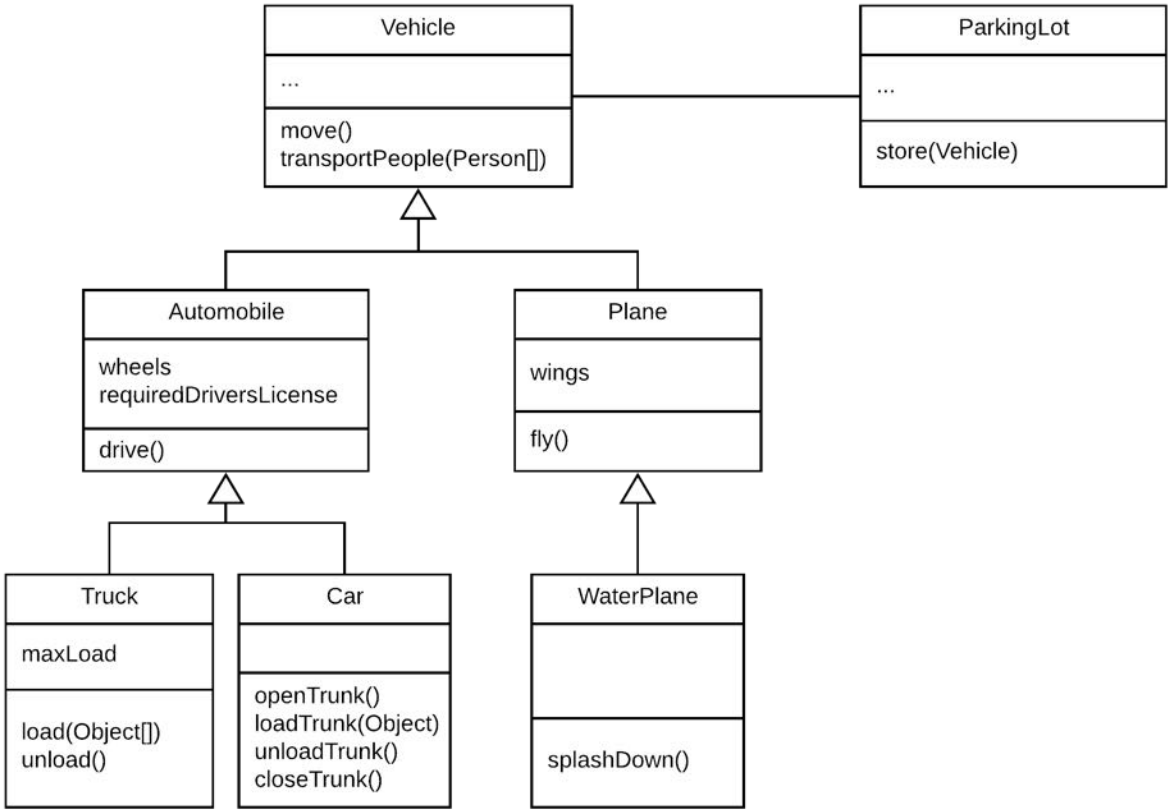
Polimorfismo



- ParkingLot almacena (*store*) objetos de tipo Vehicle
  - Un WaterPlane es un Vehicle->
  - Un WaterPlane puede ser almacenado (*store*) en ParkingLot

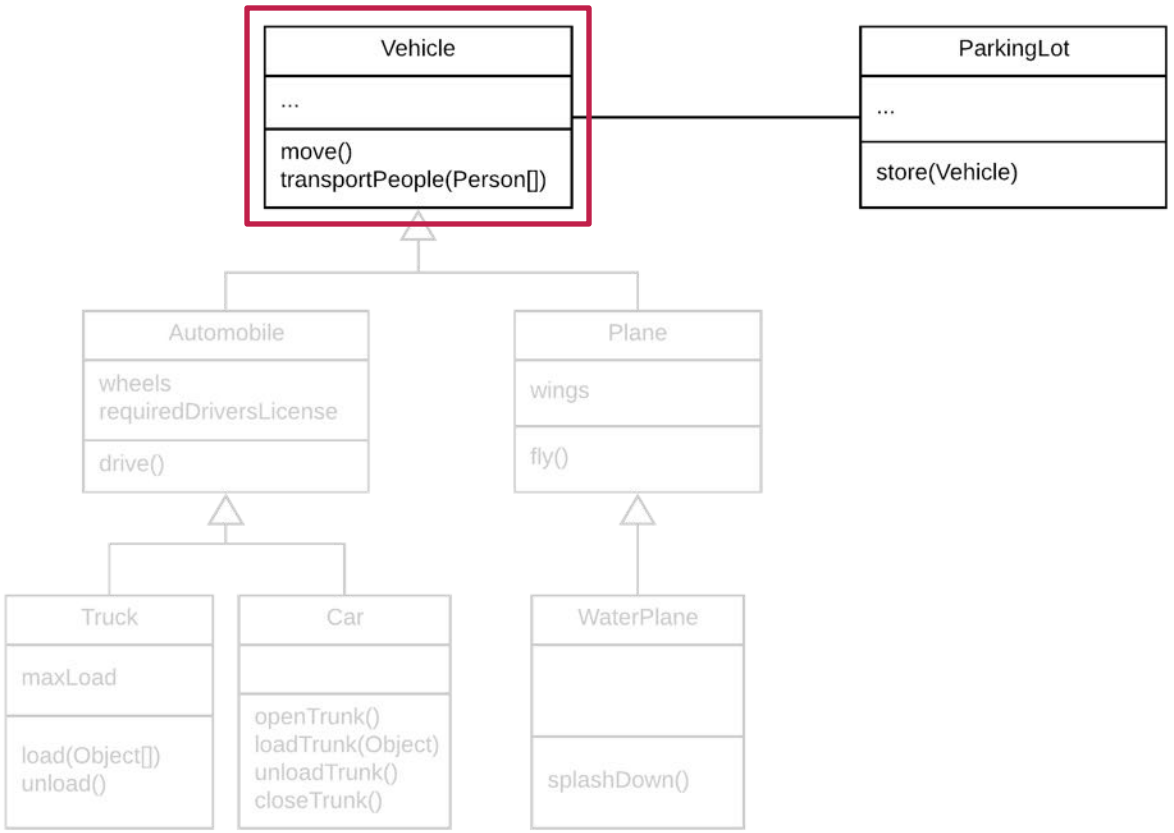
Polimorfismo

```
1   [...]
2      ParkingLot lot = new ParkingLot();
3      Vehicle v = new Vehicle();
4   [...]
```



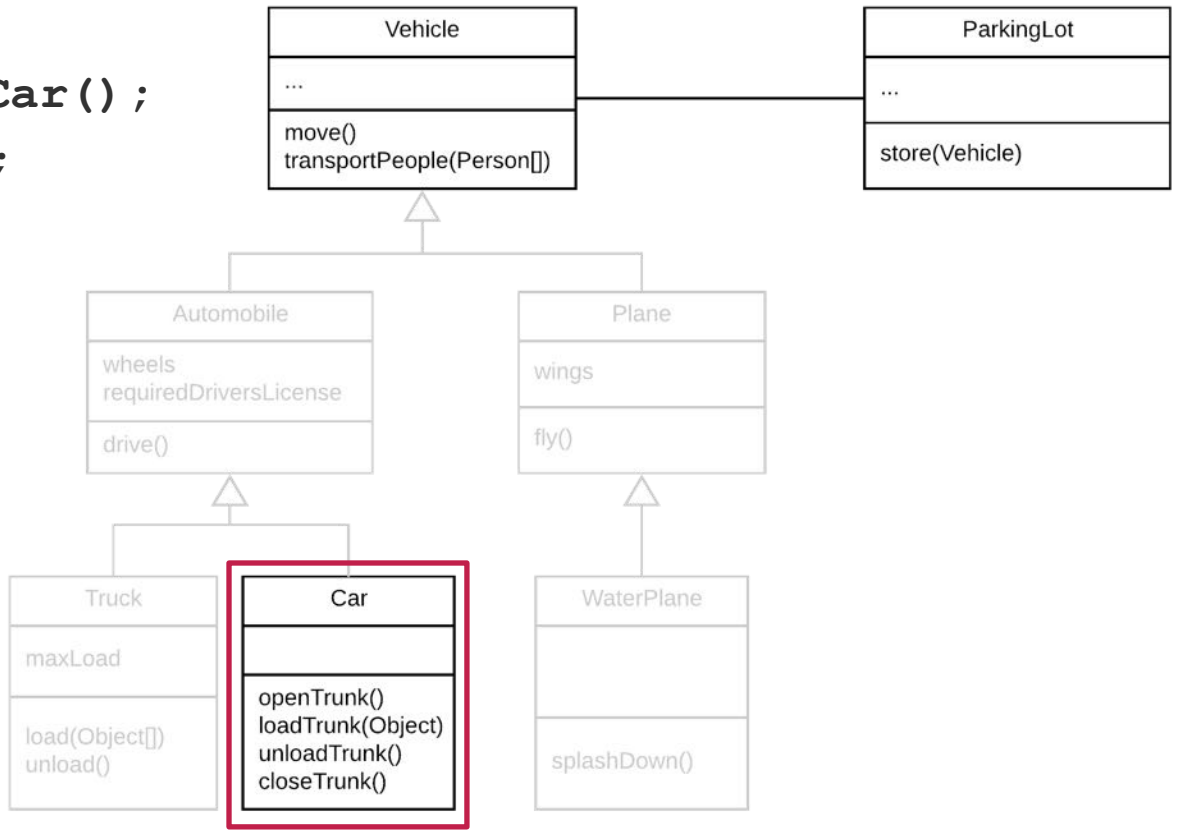
Polimorfismo

```
1  [...]
2     ParkingLot lot = new ParkingLot();
3     Vehicle v = new Vehicle();
4     lot.store(v);
5  [...]
```



Polimorfismo

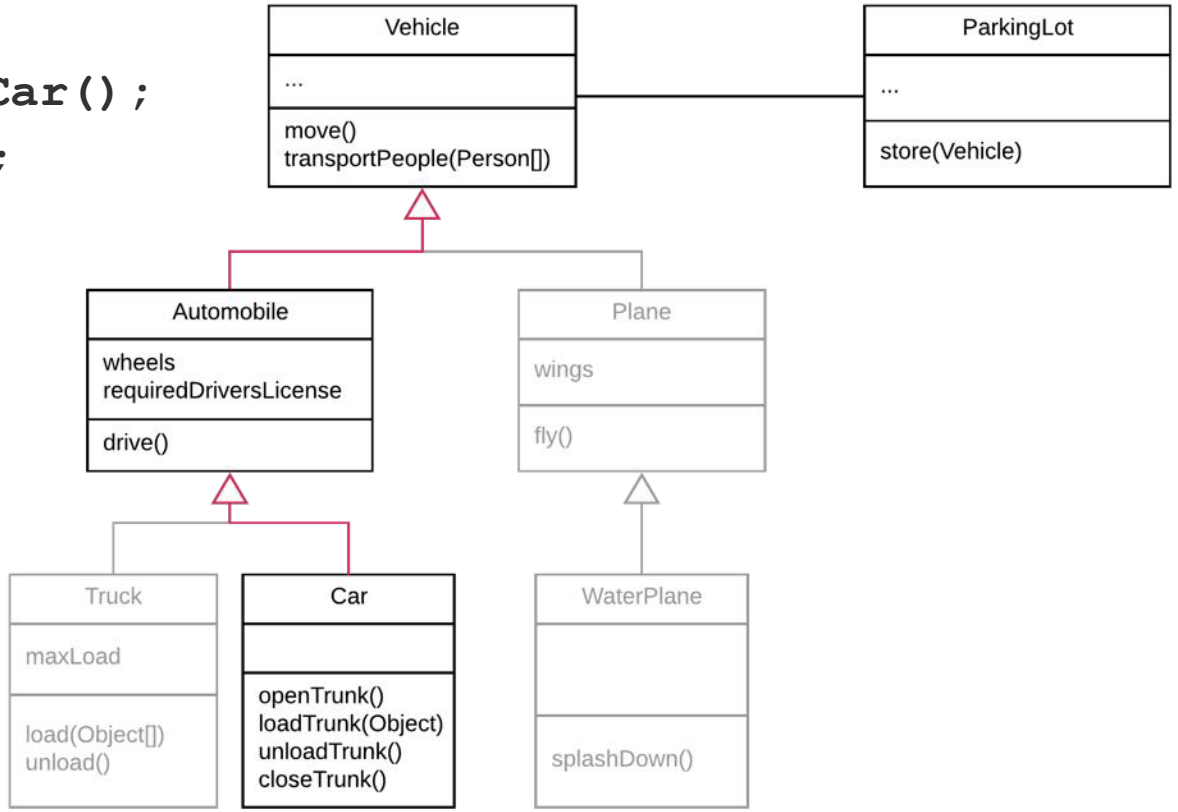
```
1  [...]
2     ParkingLot lot = new ParkingLot();
3     Vehicle v = new Vehicle();
4     lot.store(v);
5     Car c = new Car();
6     lot.store(c);
7  [...]
```



- Podemos **sustituir** un objeto de la superclase por uno de la **subclase**.

Polimorfismo

```
1  [...]
2     ParkingLot lot = new ParkingLot();
3     Vehicle v = new Vehicle();
4     lot.store(v);
5     Car c = new Car();
6     lot.store(c);
7  [...]
```



- Podemos sustituir un objeto de la superclase por uno de la subclase.

"network-v1-with-herencia-without-polymorphism"
Revisando el Código de `Newsfeed`:

```
public class NewsFeed
{
   private List<MessagePost> messages;
   private List<PhotoPost> photos;
   ...
   public void show()
   {
       for(MessagePost message : messages) {
          message.display();
          System.out.println("---------"); }

       for(PhotoPost photo : photos) {
          photo.display();
          System.out.println("---------"); }
   }
}
```

¡¡Deberíamos evitar también código duplicado en la clase cliente!!

No deberíamos tener ArrayList's *messages* Y *photos*

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

“network-v2-with-herencia-with-polymorphism”
`Newsfeed` aplicando Polimorfismo

```
public class NewsFeed
{
    private List<Post> posts;

    /**
     * Construct an empty news feed.
     */
    public NewsFeed(){
        posts = new ArrayList<Post>();
    }

    /**
     * Add a post to the news feed.
     */
    public void addPost(Post post){
        posts.add(post);
    }
    ...
}
```

Podemos aplicar **polimorfismo** en la clase cliente



cation, Inc. All Rights Reserved

"network-v2-with-herencia-with-polymorphism"
`Newsfeed` aplicando Polimorfismo

Primero teníamos:

```
public void addMessagePost(
                        MessagePost message)
public void addPhotoPost(
                        PhotoPost photo)
```

Ahora tenemos:

```
public void addPost(Post post)
```

Ejemplo de llamada a este nuevo método:

```
Post myPhoto = new PhotoPost(...);
feed.addPost(myPhoto);
```

Podemos hacerlo porque

PhotoPost **es un subtipo de** ***Post***

"network-v2-with-herencia-with-polymorphism"
`Newsfeed` aplicando Polimorfismo

```
public class NewsFeed
{

    public void addPost(Post post)
    {
        ...
    }
}
```

```
Post photo = new PhotoPost(...);
Post message = new MessagePost(...);

feed.addPost(photo);
feed.addPost(message);
```

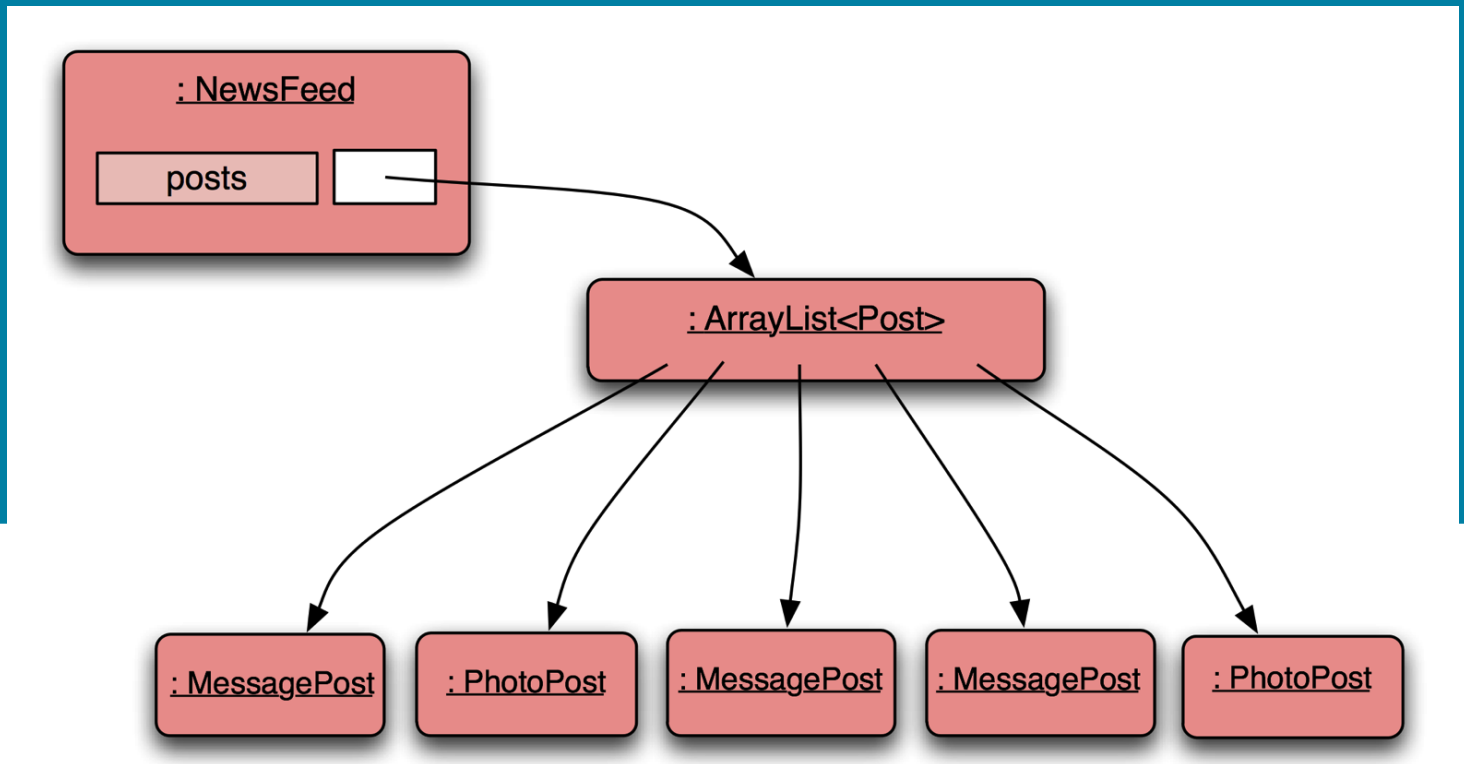
objetos del tipo de las subclases pueden ser usados como parámetros reales sustituyendo a la superclase

PhotoPost & *MessagePost* son ambos subtipos de *Post*

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

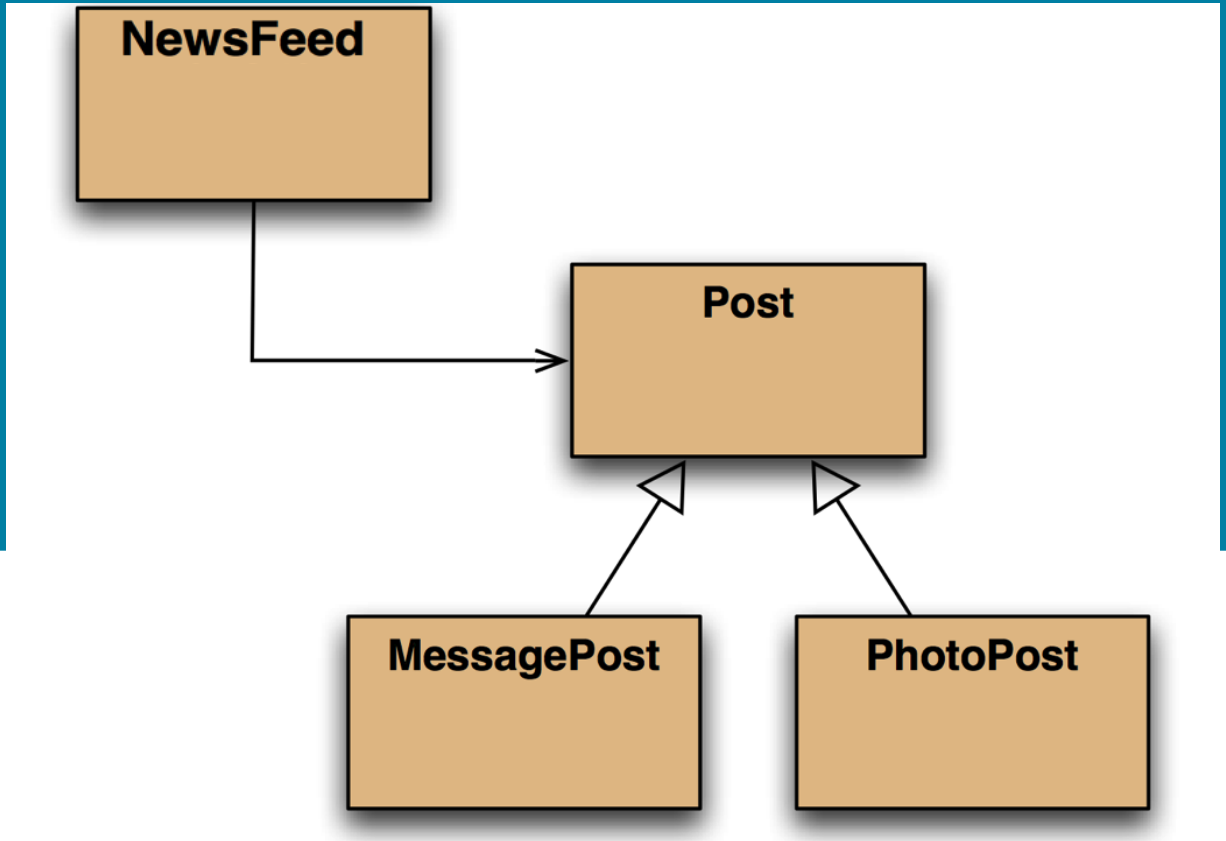
“network-v2-with-herencia-with-polymorphism”
Diagrama de Objetos



objeto *NewsFeed* puede mantener una colección de objetos mezclados del supertipo *Post* y de los subtipos PhotoPost/MessagePost

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

“network-v2-with-herencia-with-polymorphism”
Diagrama de Clases



NewsFeed ahora solo conoce al supertipo *Post*
y NO a sus subclasses MessagePost/PhotoPost

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

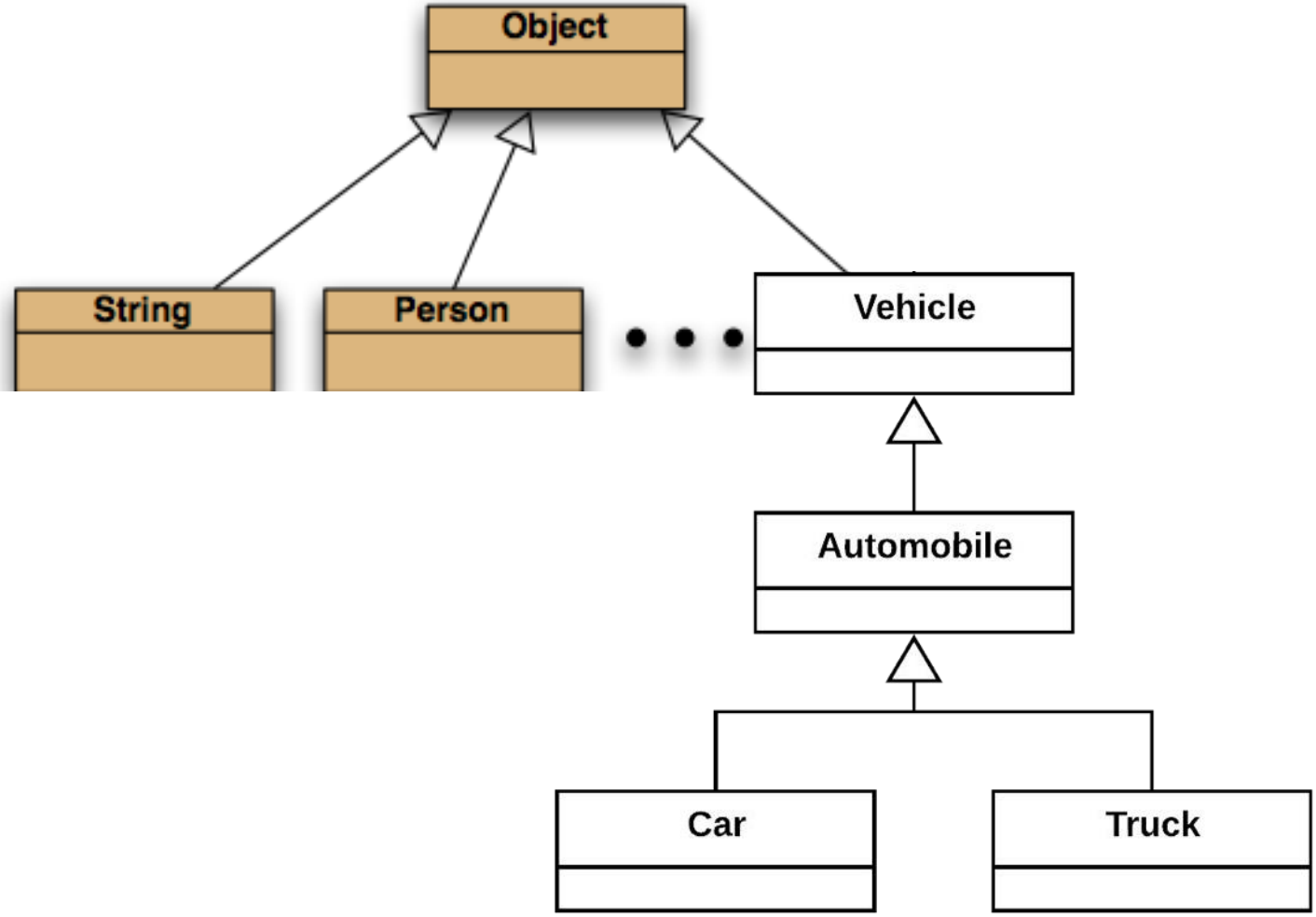
Conceptos fundamentales

- Herencia
  - Constructores
- Polimorfismo
  - Subtipos y Sustitución
  - La Clase Object
  - Colecciones Polimórficas
  - Casting
  - Tipo Estático y Dinámico
  - Sobreescritura
  - En Métodos
- StringBuilder
- Clases y métodos abstractos
- Métodos de la clase Object
  - toString
  - equals
  - hashCode
- Extendiendo jerarquías de clases
- Herencia multiple con Interfaces
- Acceso Protegido (protected)
- La clase “Class”

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

La Clase Object

Todas las clases en Java heredan de la clase **Object.**



Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales

- Herencia
  - Constructores
- Polimorfismo
  - Subtipos y Sustitución
  - La Clase Object
  - Colecciones Polimórficas
  - Casting
  - Tipo Estático y Dinámico
  - Sobreescritura
  - En Métodos
- StringBuilder
- Clases y métodos abstractos
- Métodos de la clase Object
  - toString
  - equals
  - hashCode
- Extendiendo jerarquías de clases
- Herencia múltiple con Interfaces
- Acceso Protegido (protected)
- La clase “Class”

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Colecciones Polimórficas

- Todas las colecciones son **polimórficas**.
- Los elementos podrían ser simplemente de tipo `Object`.

```
public void add(Object element)
public Object get(int index)
```

- Generalmente se evita mediante el uso de un parámetro de tipo con la colección.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Colecciones Polimórficas

- Un parámetro de tipo limita el grado de polimorfismo: `List<Post>`
- Los métodos de la colección pasan a tener tipo.
- Sin un parámetro de tipo de forma implícita :
  - `List<Object>` :
- Probable recibir un warning:
  - **"unchecked or unsafe operations"**
- Muy probable tener que usar **casting**.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales

- Herencia
  - Constructores
- Polimorfismo
  - Subtipos y Sustitución
  - La Clase Object
  - Colecciones Polimórficas
  - Casting
  - Tipo Estático y Dinámico
  - Sobreescritura
  - En Métodos
- StringBuilder
- Clases y métodos abstractos
- Métodos de la clase Object
  - toString
  - equals
  - hashCode
- Extendiendo jerarquías de clases
- Herencia múltiple con Interfaces
- Acceso Protegido (protected)
- La clase “Class”

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Casting

- Instancias de los **subtipos** se pueden usar cuando se requieren instancias del supertipo
- Pero las instancias del supertipo **NO** se pueden usar cuando se requieren instancias del **subtipo**

```
Vehicle v;
  Car c = new Car();
  v = c; // correct
  c = v; // no compilará
```

- El **Casting** arregla el problema:

```
c = (Car) v;
```

OJO: ¡¡run-time error si `v` no es un `Car`!!

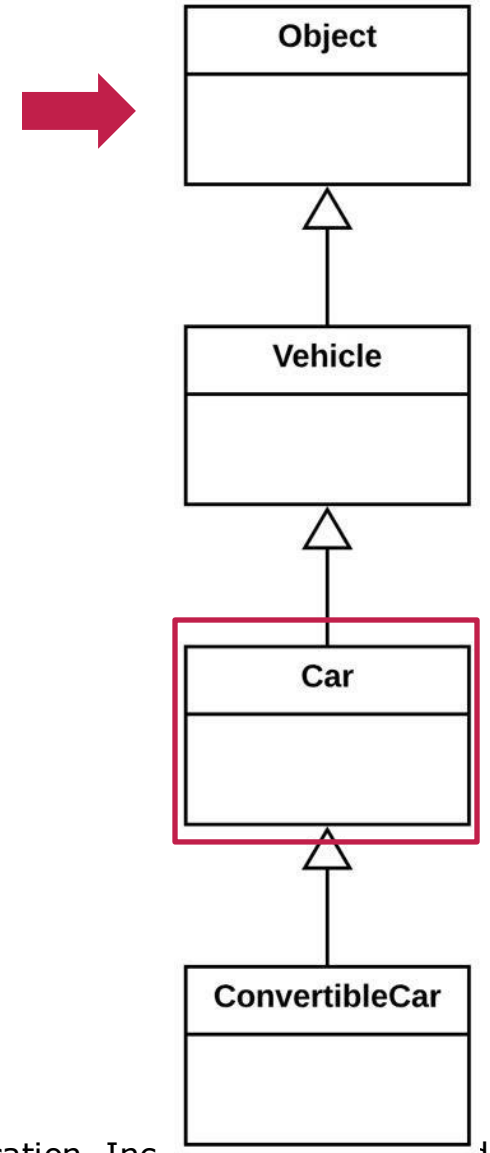
Casting

- Se indica el tipo de objeto entre paréntesis.
- Usado para superar la “Pérdida de tipo”.
- El objeto no se modifica de ninguna manera.
- Se realiza una comprobación en tiempo de ejecución para asegurar que el objeto es realmente de ese tipo:
  - ¡`ClassCastException` si no lo es!

OJO: ¡¡el casting solo debería usarse en un número muy reducido de casos!!

Casting

```
1   Object car = new Car();
```



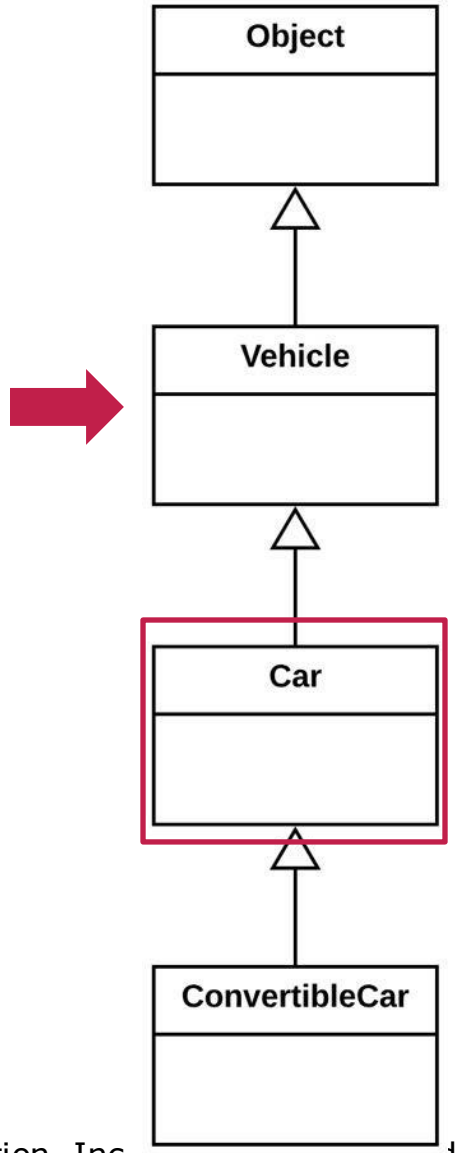
Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Casting

```
1   Object car = new Car();
2
3   Vehicle v = (Vehicle) car;
```

Un Car es un Vehicle



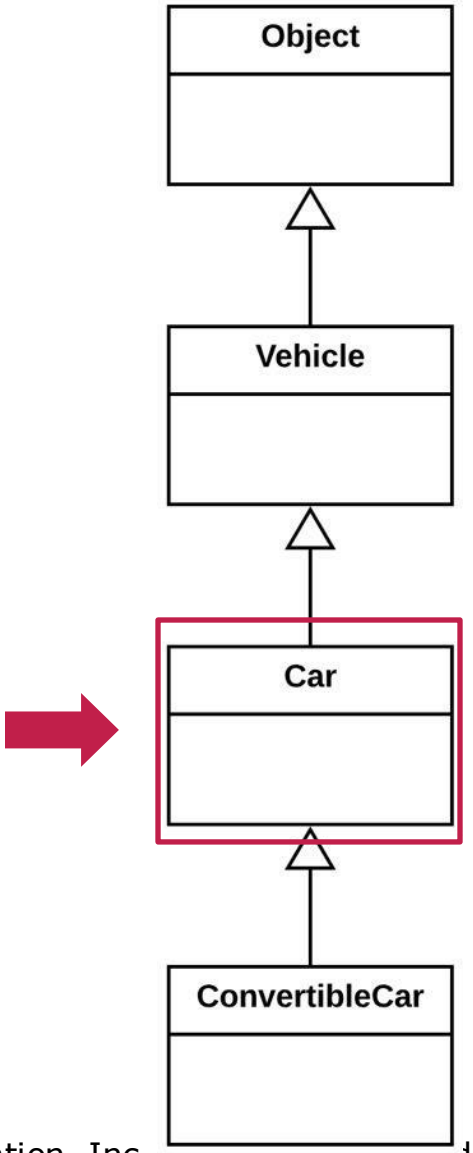
Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Casting

```
1   Object car = new Car();
2
3   Vehicle v = (Vehicle) car;
4
5   Car sameCar = (Car) v;
```

Por supuesto un Car es un Car



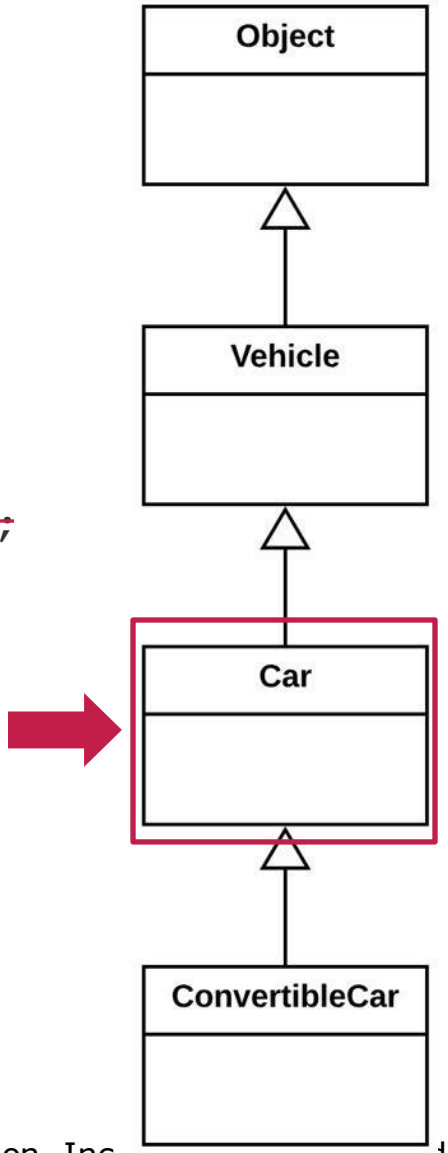
Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Casting

```
1   Object car = new Car();
2
3   Vehicle v = (Vehicle) car;
4
5   Car sameCar = (Car) v;
6
7   ConvertibleCar cc = (ConvertibleCar) car;
```

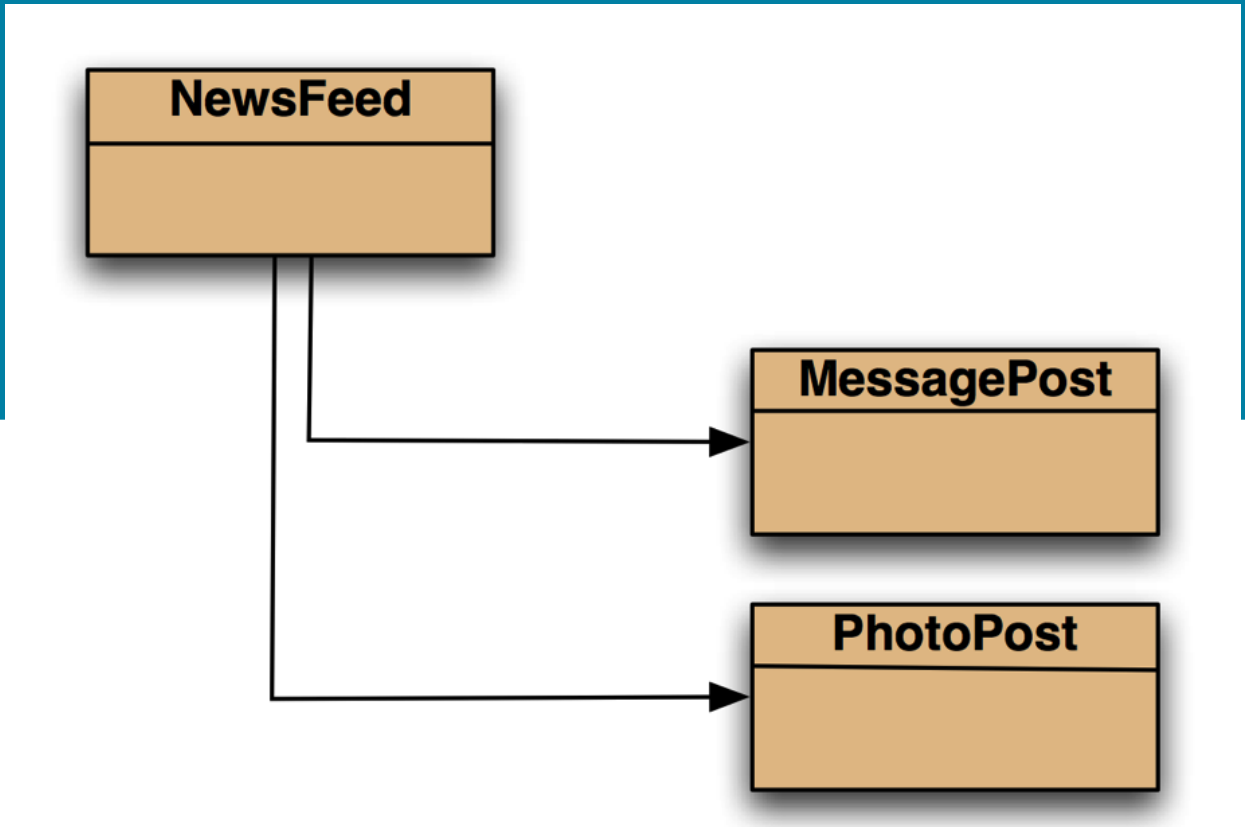
No todos los Car
son un ConvertibleCar



Se puede hacer casting de un **objeto** a cada tipo de dato **linealmente por encima** en su jerarquía de herencia

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

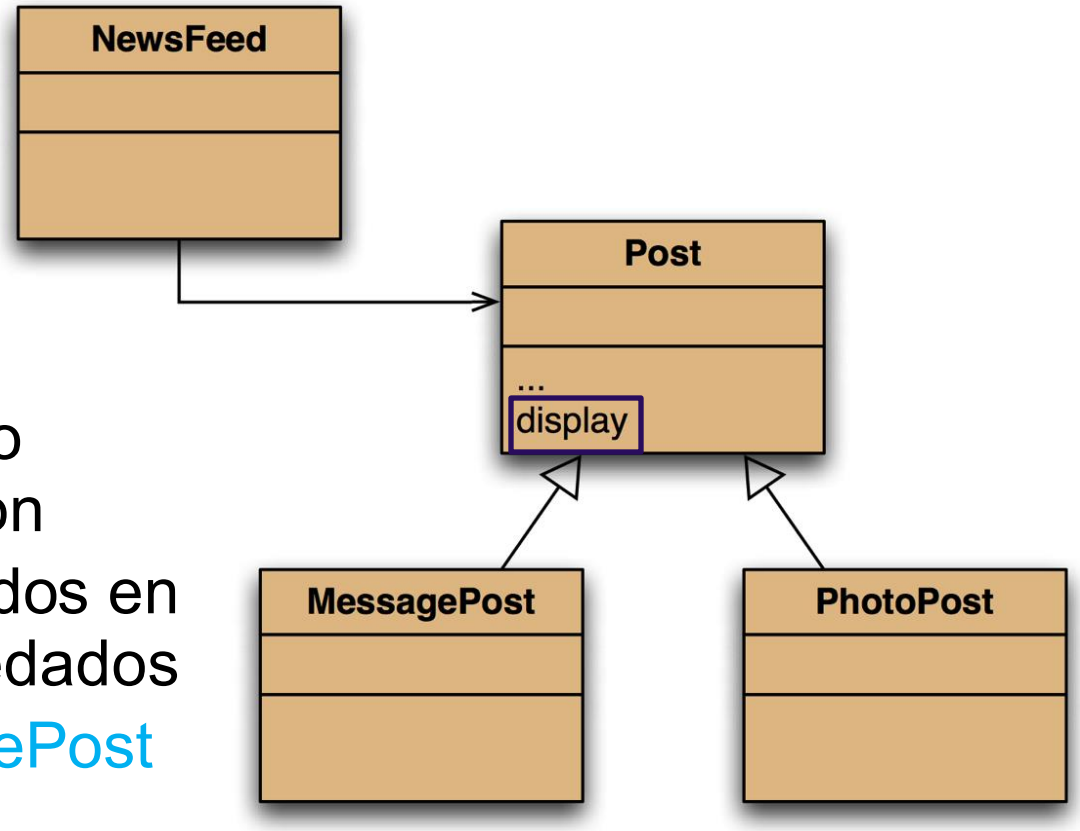
“network-v2-with-herencia-with-polymorphism”
Antes de aplicar herencia



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

“network-v2-with-herencia-with-polymorphism”
Después de aplicar herencia



Métodos como `display` son implementados en `Post` y heredados por MessagePost y PhotoPost

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

"network-v2-with-herencia-with-polymorphism"
`Newsfeed` aplicando Polimorfismo

```
/**
 * Show the news feed. Currently: print the
 * news feed details to the terminal.
 * (Later: display in a web browser.)
 */
public void show()
{
    for(Post post : posts) {
        post.display();
        System.out.println(---------); }
}
```

Gracias a la herencia ahora solo tenemos un bucle en el método show que invoca al método *display* de *Post* -> Podemos hacer esto porque implementación de display es heredada por MessagePost y PhotoPost

Todo parece ir bien, ¿o no?

"network-v2-with-herencia-with-polymorphism"

Gran Problema después de aplicar herencia

Salida del método show() que invoca a display() de post

```
Leonardo da Vinci //Salida de MessagePost
40 seconds ago
Had a great idea this morning.
But now I forgot what it was. Something to do with flying ...
No comments.
```

```
Alexander Graham Bell  //Salida de PhotoPost
12 minutes ago.
Photo Path : experiment.jpg
I think I might call this thing 'telephone'.
No comments.
```

Lo que esperábamos

```
Leonardo da Vinci //Salida de MessagePost
40 seconds ago.
  No comments.
```

```
Alexander Graham Bell  //Salida de PhotoPost
12 minutes ago.
  No comments.
```

Lo que obtenemos

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

"network-v2-with-herencia-with-polymorphism"

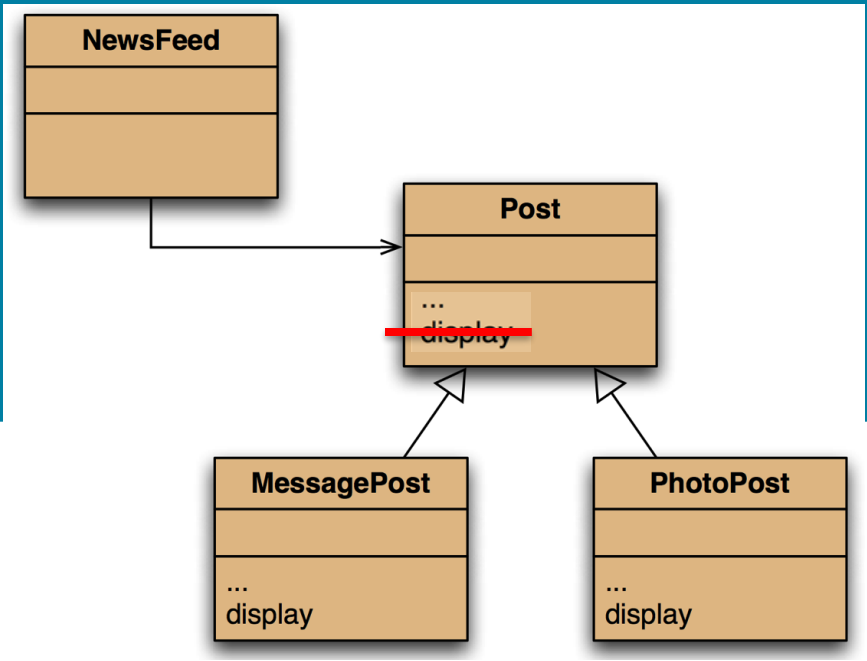
Gran Problema después de aplicar herencia

```
public class Post{
     …
   public void display()   {
    System.out.println(getUsername());
    System.out.println(getFormattedTimeStamp());
    printComments();
   }
}
```

Problema:

- El método **display** implementado en **Post** tras aplicar herencia solo contenía las llamadas a los métodos que muestran los campos comunes.
- Herencia es una calle de una sola dirección:
  - Una **subclase** hereda los campos de la **superclase**.
  - La **superclase** no sabe nada de sus **subclases** y por tanto tampoco sabe nada de los campos de sus **subclases**

“network-v2-with-herencia-with-polymorphism”
Intentando resolver el Problema

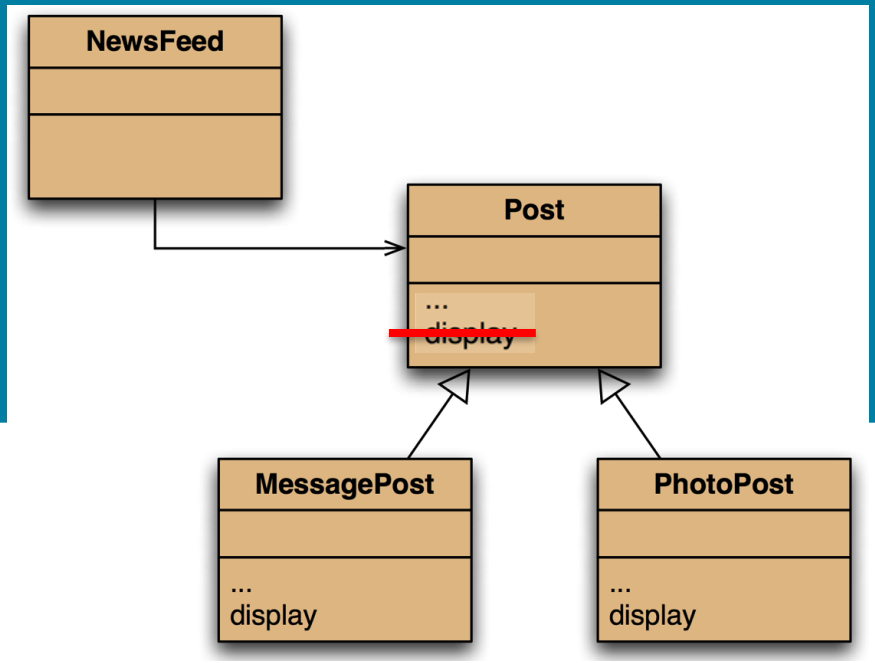


- Colocar `display` donde tenga acceso a la información que necesita y quitarlo de `Post`.
- Cada subclase tendría su propia version completa de `display`
  - incluyendo la duplicación de los métodos que muestran los campos comunes a ambas.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

“network-v2-with-herencia-with-polymorphism”
Intentando resolver el Problema

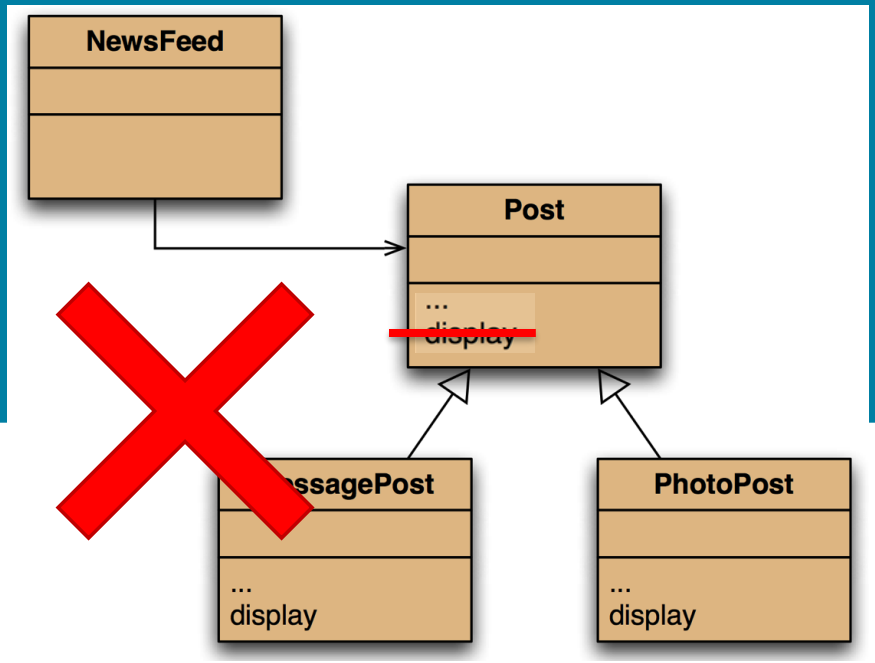


- Colocar `display` donde tenga acceso a la información que necesita y quitarlo de `Post`.
- Cada subclase tendría su propia version completa de `display`
  - incluyendo la duplicación de los métodos que muestran los campos comunes a ambos.



Todo solucionado, ¿o no?

"network-v2-with-herencia-with-polymorphism"
Intentando resolver el Problema



- Ya la duplicación de los métodos que muestran los campos comunes a ambas no era una solución aceptable en absoluto
- Pero es que tenemos aún un problema más grave: **`NewsFeed`** no puede encontrar un método **`display`** en **`Post`**.

```
for(Post post : posts) {
        post.display(); // Error en tiempo de
                        // compilación porque
                        // display no está ahora
                        // definido en Post
}
```

Conceptos fundamentales

- Herencia
  - Constructores
- Polimorfismo
  - Subtipos y Sustitución
  - La Clase Object
  - Colecciones Polimórficas
  - Casting
  - Tipo Estático y Dinámico
  - Sobreescritura
  - En Métodos
- StringBuilder
- Clases y métodos abstractos
- Métodos de la clase Object
  - toString
  - equals
  - hashCode
- Extendiendo jerarquías de clases
- Herencia múltiple con Interfaces
- Acceso Protegido (protected)
- La clase “Class”

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Tipo Estático y Tipo Dinámico

¿Cuál es el tipo de c1?

```
Car c1 = new Car();
```

¿Cuál es el tipo de **v1**?

```
Vehicle v1 = new Car();
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

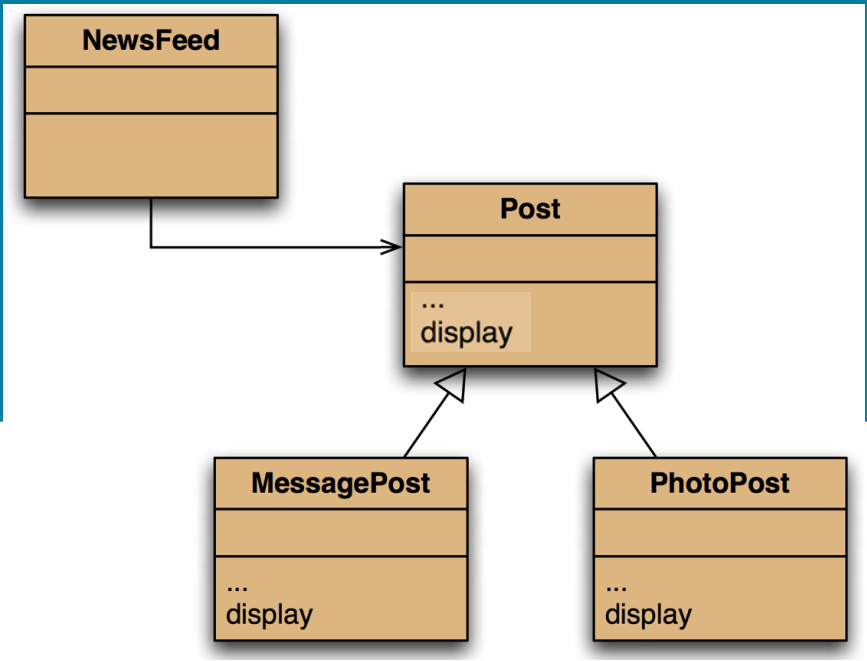
Tipo Estático y Tipo Dinámico

- El tipo declarado de una variable es su **Tipo Estático**.
- El tipo del objeto al que realmente referencia la variable es su **Tipo Dinámico**.
- Es tarea del compilador comprobar posibles violaciones del **Tipo Estático**.

```
for(Post post : posts) {
        post.display(); // Error en tiempo de
                        // compilación porque
                        // display no está ahora
                        // definido en Post
}
```

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

“network-v2-with-herencia-with-polymorphism”
Intentando resolver el Problema



- Necesitamos que el comportamiento común de display siga en la superclase y el comportamiento específico de cada subclase se implemente en cada subclase

- Y necesitamos alguna forma de invocar a ambos comportamientos a la vez

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales

- Herencia
  - Constructores
- Polimorfismo
  - Subtipos y Sustitución
  - La Clase Object
  - Colecciones Polimórficas
  - Casting
  - Tipo Estático y Dinámico
  - Sobreescritura
  - En Métodos
- StringBuilder
- Clases y métodos abstractos
- Métodos de la clase Object
  - toString
  - equals
  - hashCode
- Extendiendo jerarquías de clases
- Herencia múltiple con Interfaces
- Acceso Protegido (protected)
- La clase “Class”

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Sobreescritura

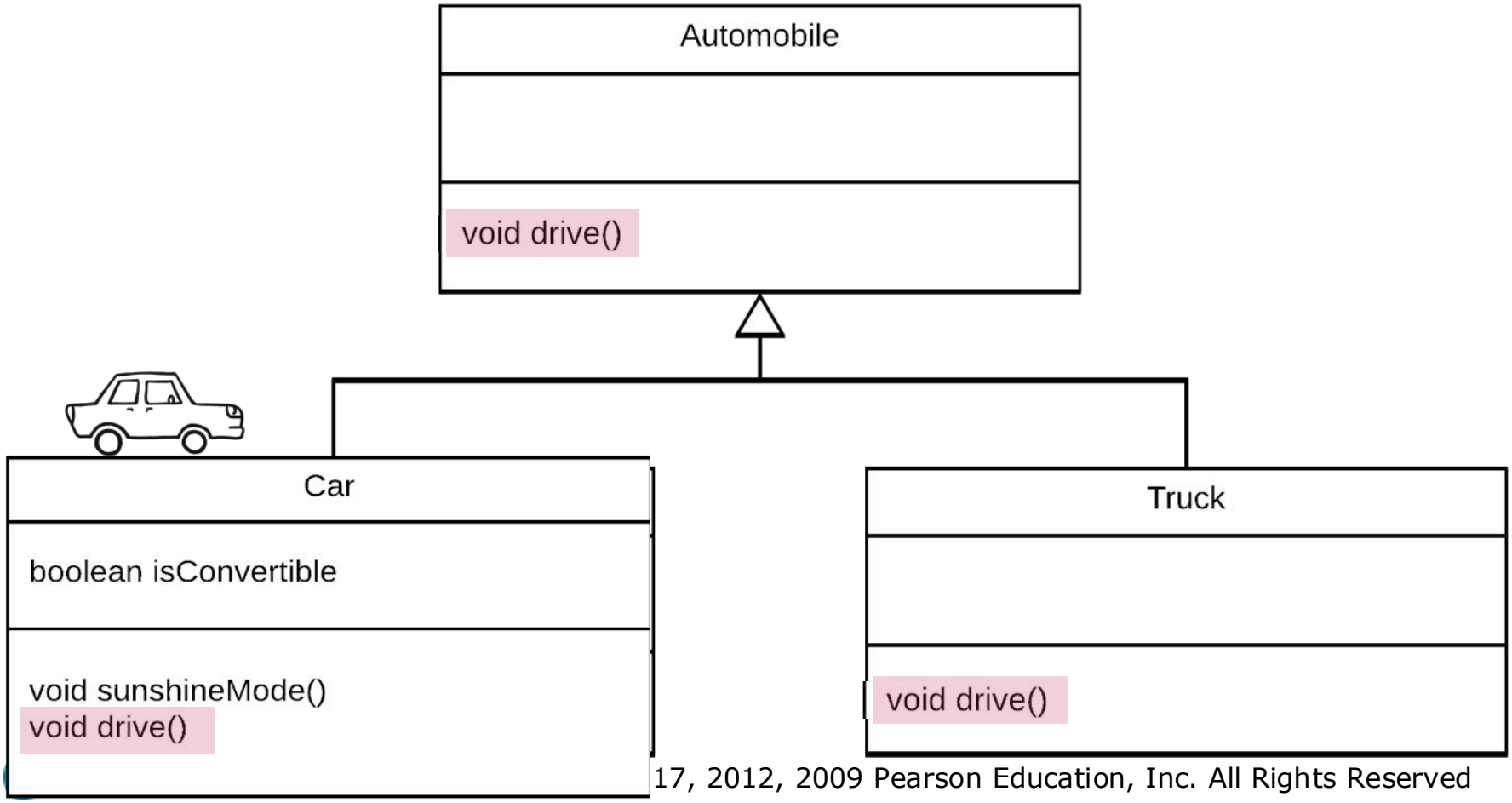
- Superclase y subclases definen métodos con la misma signatura.
- Cada uno tiene acceso a los campos de sus clases.
- Superclase satisface el chequeo de Tipo Estático.
- Método de la Subclase es invocado en tiempo de ejecución – **sobreescribe** la version de la superclase.
  - La anotación @Override informa al compilador que el elementos está destinado a sobreescribir (anular) un elemento declarado en una superclase

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Sobreescritura

- Permite implementar un comportamiento alternativo para cada Subclase



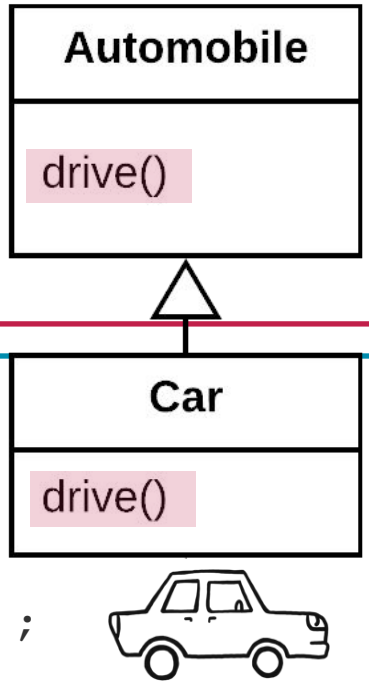
17, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Sobreescritura

- Permite implementar un comportamiento alternativo para cada Subclase

```
1  public class Automobile {
2      public void drive(int x) {
3          System.out.println("Speed: " + x);
4      }
5  }
```

```
1  public class Car extends Automobile {
2      @Override
3      public void drive(int x) {
4          System.out.println("Speed: " + (2*x));
5      }
6  }
```



Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Sobreescritura

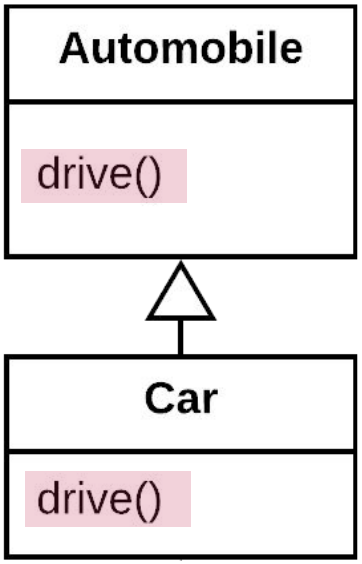
Anotación `@Override` para informar al compilador

Superclass:

```
public void drive() {
    //...
}
```

Subclass:

```
@Override
public void drive() {
    //...
}
```



Parámetros deben ser **exactamente** los mismos

El tipo de retorno debe ser compatible

La sobreescritura de métodos no pueden tener un modificador de acceso más restrictivo que el método que sobreescriben

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Sobreescritura

- ¿Qué ocurre entonces con la version de la superclase?
- Métodos sobreescritos se ocultan ...
- ... pero a menudo querremos ser capaces de poder invocarlos.
- Un método sobreescrito **puede** ser llamado desde el método que lo sobreescribe.
  - `super.method(...)`
  - Es similar al uso de `super` en los constructores.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

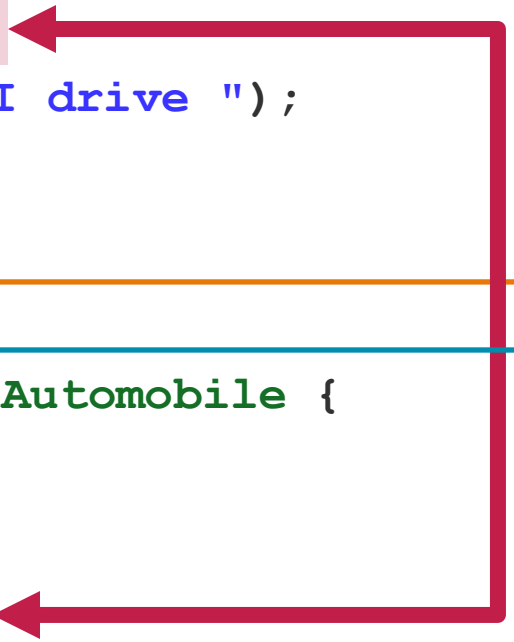
Sobreescritura

`super.method(...)`

Accede a métodos de la superclase

```
1  public class Automobile {
2     public void drive() {
3          System.out.print("I drive ");
4     }
5  }
```

```
1  public class Car extends Automobile {
2     @Override
3     public void drive() {
4          super.drive();
5          this.driveFast();
6     }
7     private void driveFast() {
8          System.out.println("pretty fast!");
9     }
10
```



Super vs This*

```
super.<<identificadorMétodo>>;
```

- ✓ Acceder a métodos de **la superclase**
- ✓ Distingue métodos sobreescritos **en la subclase** de métodos originales de **la superclase**

*** this y this() se trataron en la Unidad 2**

Super vs This*

```
super.<<identificadorMétodo>>;
```

- ✓ Acceder a métodos de **la superclase**
- ✓ Distingue métodos sobreescritos **en la subclase** de métodos originales de **la superclase**

```
super();
```

- ✓ Llama al constructor de **la superclase**
- ✓ Solo puede ser llamado dentro del constructor
- ✓ Tiene que ser la primera sentencia dentro del constructor **de la subclase**

*** this y this() se trataron en la Unidad 2**

Super vs This*

`super.<<identificadorMétodo>>;`

- ✓ Acceder a métodos de **la superclase**
- ✓ Distingue métodos sobreescritos **en la subclase** de métodos originales de **la superclase**

`super();`

- ✓ Llama al constructor de **la superclase**
- ✓ Solo puede ser llamado dentro del constructor
- ✓ Tiene que ser la primera sentencia dentro del constructor **de la subclase**

`this.<<identificador>>;`

- ✓ Accede a campos o métodos de **la misma clase**
- ✓ Distingue variables locales de campos

*** this y this() se trataron en la Unidad 2**

Super vs This*

`super.<<identificadorMétodo>>;`

- ✓ Acceder a métodos de **la superclase**
- ✓ Distingue métodos sobreescritos **en la subclase** de métodos originales de **la superclase**

`super();`

- ✓ Llama al constructor de **la superclase**
- ✓ Solo puede ser llamado dentro del constructor
- ✓ Tiene que ser la primera sentencia dentro del constructor **de la subclase**

`this.<<identificador>>;`

- ✓ Accede a campos o métodos de **la misma clase**
- ✓ Distingue variables locales de campos

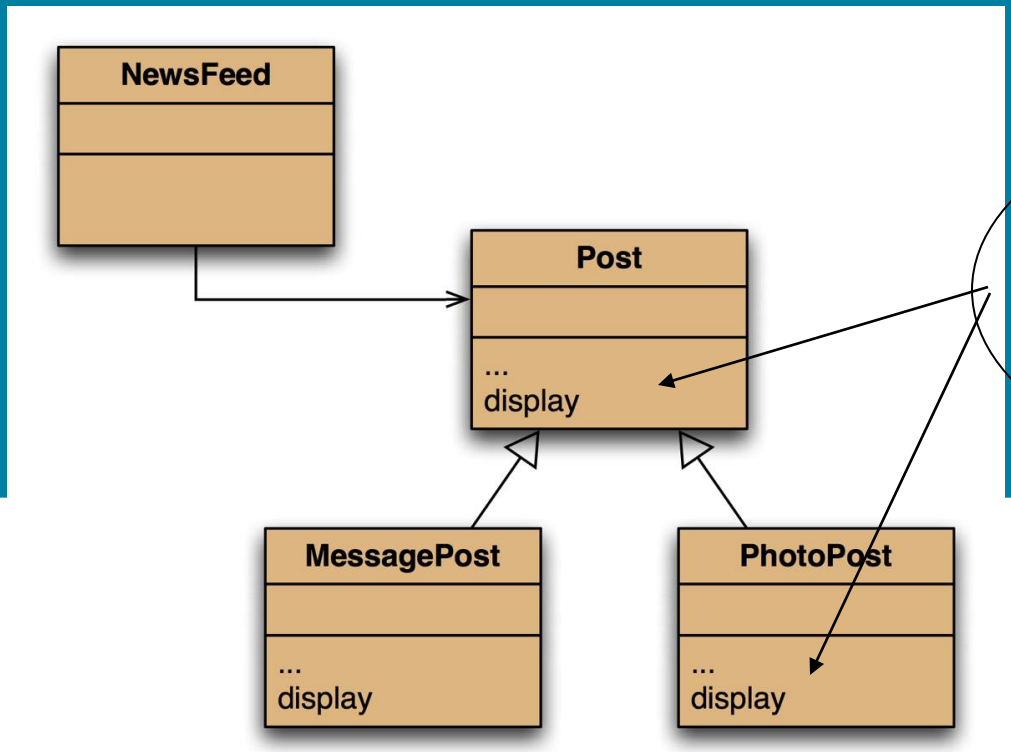
`this();`

- ✓ Llama a constructor sobrecargado de **la misma clase**
- ✓ Solo puede ser llamado dentro del constructor
- ✓ Si se incluye, tiene que ser la primera sentencia dentro del constructor
- ✓ Es incompatible con el uso de `super()`

*** this y this() se trataron en la Unidad 2**

"network-v3-with-herencia-with-polymorphism-plus-display-overriding"

Solución: Sobreescritura (Overriding)



método display tanto en superclase como en subclase

Satisface tanto chequeo estático como dinámico

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

"network-v3-with-herencia-with-polymorphism-plus-display-overriding"

Solución: Sobreescritura (Overriding)

```
public class Post{
     …
   public void display()   {
    System.out.println(getUsername());
    System.out.println(getFormattedTimeStamp());
    printComments();
   }
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

"network-v3-with-herencia-with-polymorphism-plus-display-overriding"

Solución: Sobreescritura (Overriding)

```
public class Post{
     …
   public void display()   {
    System.out.println(getUsername());
    System.out.println(getFormattedTimeStamp());
    printComments();
   }
}
```

```
public class MessagePost extends Post{
     …
   @Override
   public void display(){
    super.display();
    System.out.println(getText());
   }
}
```

"network-v3-with-herencia-with-polymorphism-plus-display-overriding"

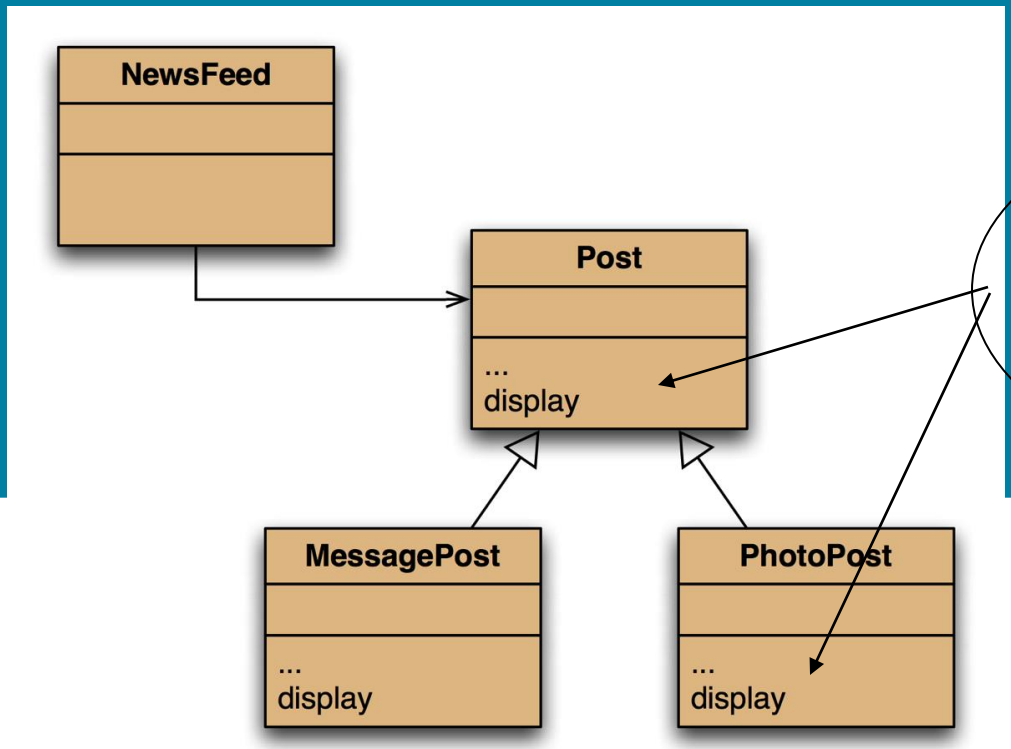
Solución: Sobreescritura (Overriding)

```
public class Post{
     …
   public void display()   {
    System.out.println(getUsername());
    System.out.println(getFormattedTimeStamp());
    printComments();
   }
}
```

```
public class PhotoPost extends Post{
     …
   @Override
   public void display()   {
    super.display();
    System.out.println("Photo Path : " + getFilename());
    System.out.println(" " + getCaption());
   }
}
```

"network-v3-with-herencia-with-polymorphism-plus-display-overriding"

Solución: Sobreescritura (Overriding)



método display tanto en superclase como en subclase

Pero el código del método show invoca al método *display* de *Post*

```
for(Post post : posts) {
        post.display();}
```

Entonces, ¿No seguirían mostrándose solo los valores de los campos comunes?

Conceptos fundamentales

- Herencia
  - Constructores
- Polimorfismo
  - Subtipos y Sustitución
  - La Clase Object
  - Colecciones Polimórficas
  - Casting
  - Tipo Estático y Dinámico
  - Sobreescritura
  - En Métodos
- StringBuilder
- Clases y métodos abstractos
- Métodos de la clase Object
  - toString
  - equals
  - hashCode
- Extendiendo jerarquías de clases
- Herencia múltiple con Interfaces
- Acceso Protegido (protected)
- La clase “Class”

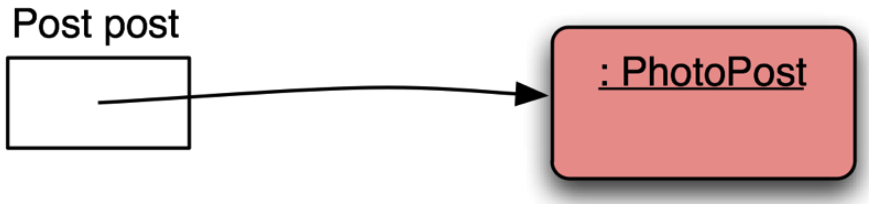
Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Polimorfismo de métodos

- Una **variable polimórfica** puede almacenar objetos de varios tipos.
- Las invocaciones a métodos también son **polimórficas**.

¿Qué implica el polimorfismo en la invocación de métodos?

Es decir, ¿qué método se ejecuta cuando el tipo Estático y Dinámico no coinciden?



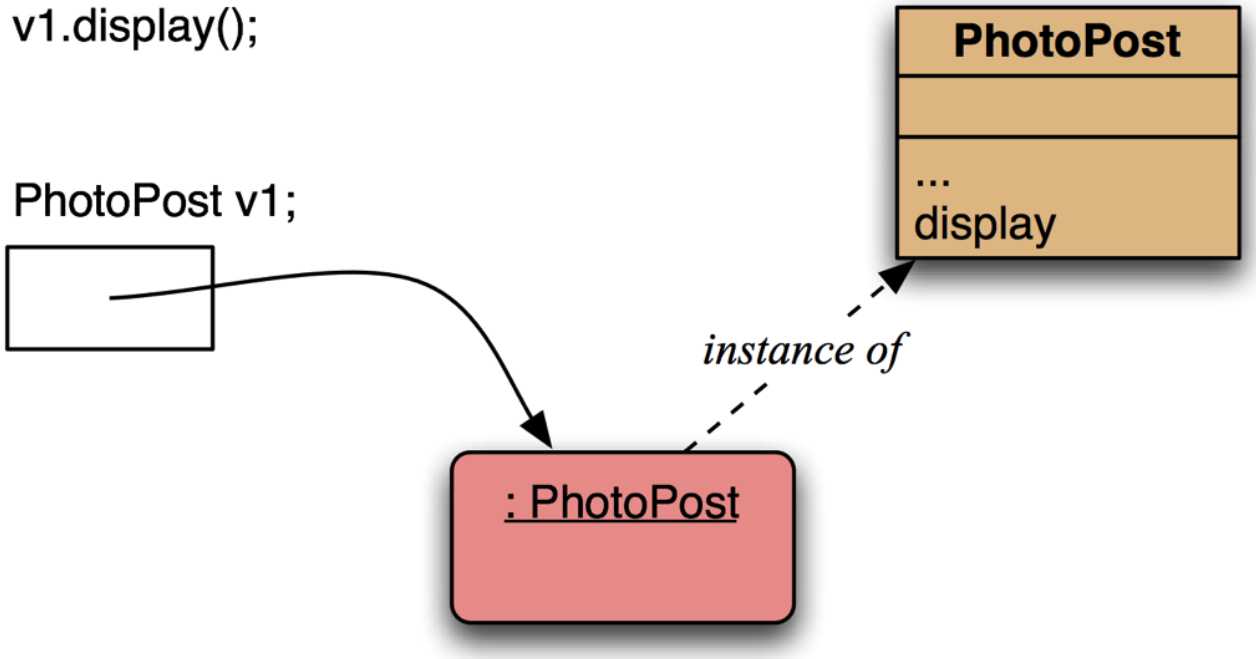
El método realmente invocado depende del Tipo Dinámico del objeto.

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Polimorfismo de métodos

- Búsqueda dinámica de método a ejecutar (Method Lookup):

  Si no hay herencia o polimorfismo:
  Se selecciona y ejecuta el método obvio.

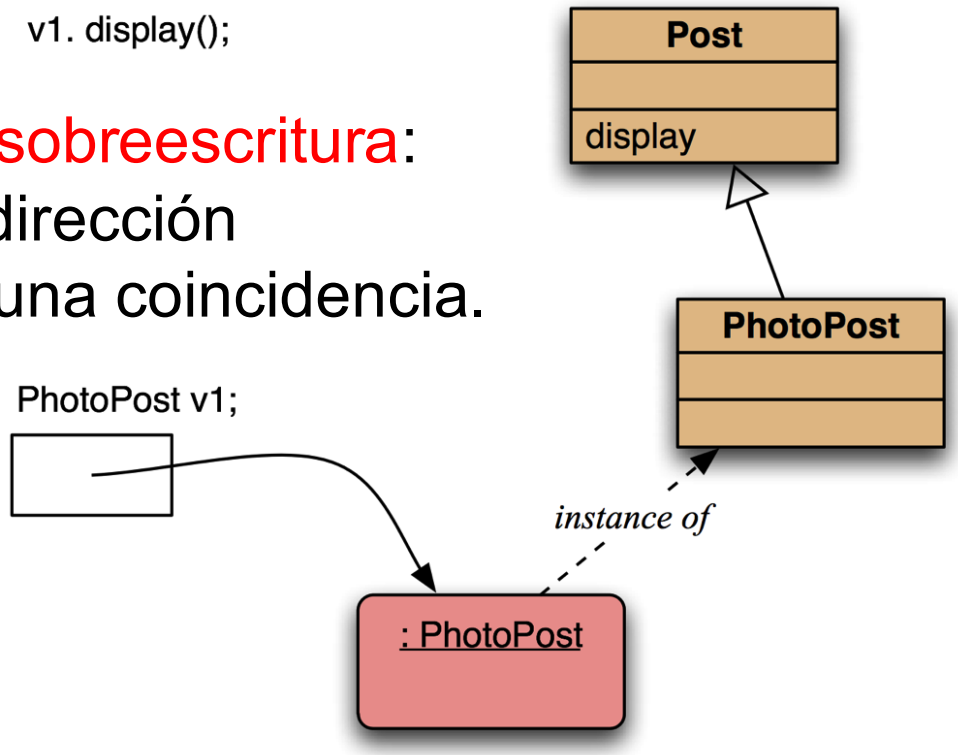


Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Polimorfismo de métodos

- Búsqueda dinámica de método a ejecutar (Method Lookup):

Si hay herencia pero no hay sobreescritura:
Se busca en la jerarquía en dirección ascendente hasta encontrar una coincidencia.



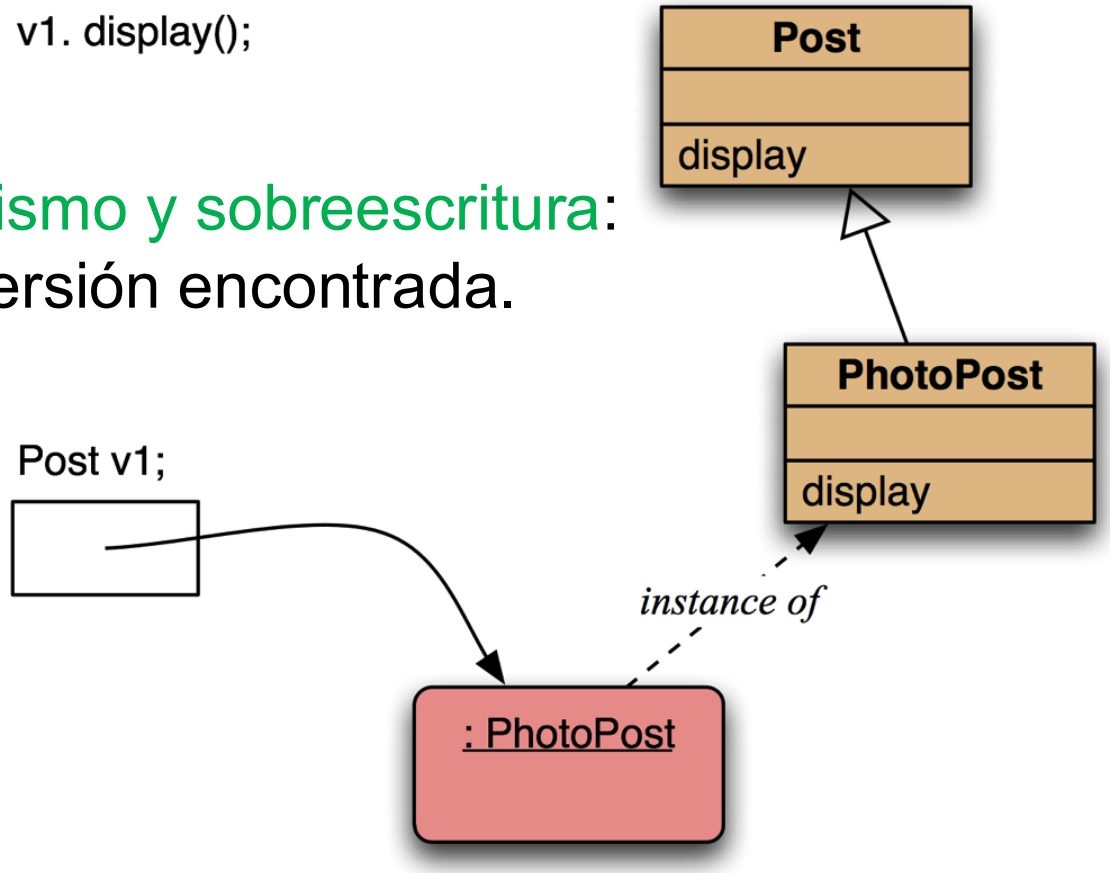
Pearson
Copyright © 20

Polimorfismo de métodos

- Búsqueda dinámica de método a ejecutar (Method Lookup):

v1. display();

Si hay herencia, polimorfismo y sobreescritura:
Se ejecuta la "primera" versión encontrada.



Copyric

Polimorfismo de métodos

- Pasos en la búsqueda dinámica de método a ejecutar:
  - La variable es accedida.
  - El objeto almacenado en la variable es encontrado.
  - La clase del objeto es encontrada.
  - Se busca el método en la clase.
  - Si no se encuentra, se busca en la superclase.
  - Se repite hasta encontrar el método en la jerarquía, o hasta que se ha recorrido entera.
  - Métodos sobreescritos tienen precedencia – anulan a las copias heredadas.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Polimorfismo de métodos

- Podríamos tener muchos tipos de **Vehículo** para diferentes propósitos:



- Una **Bicicleta** para dar una Vuelta



- Un **Coche** para ir más lejos y más rápido



- Un **Camión** para cargar más cosas



- Una **Barco** para navegar



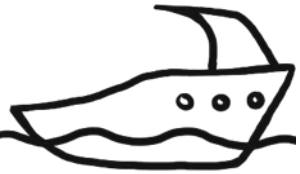
- **Un Avión** para volar

Polimorfismo de métodos

- Cada **Vehículo** se mueve de una forma diferente



  - Con un **Coche** se conduce por la carretera



  - Con un **Barco** se navega por el mar

- Diferencias para moverse:
  - El **Coche** necesita ruedas y consume combustible
  - El **Barco** necesita velas y que haya viento

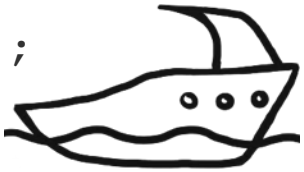
Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Polimorfismo de métodos

```
1   [...]
2      Car car    = new Car();
3      Ship ship  = new Ship();
4
5      car.drive();
6      ship.sail();
7   [...]
```





Pearson

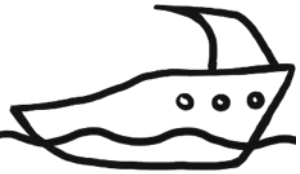
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Polimorfismo de métodos

- Cada **Vehículo** se mueve de una forma diferente



  - Con un **Coche** se conduce por la carretera



  - Con un **Barco** se navega por el mar

- Pero **todos** se **mueven** de alguna manera….

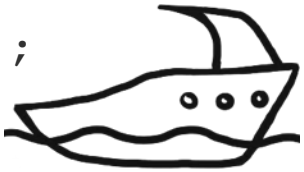
Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Polimorfismo de métodos

```
1   [...]
2      Car car    = new Car();
3      Ship ship  = new Ship();
4
5      car.move();
6      ship.move();
7   [...]
```





Cada subclase de **Vehículo** puede implementar un su version del método move

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Polimorfismo de métodos

```
1    [...]
2       Vehicle car   = new Car();
3       Vehicle ship  = new Ship();
4    [...]
```





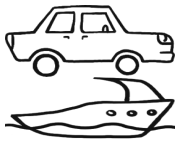
Una variable de tipo Vehicle puede contener objetos de todas sus subclases

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Polimorfismo de métodos

```
1   [...]
2      Vehicle car = new Car();
3      Vehicle ship = new Ship();
4
5      Vehicle[] vehicles =  new  Vehicle[2];
6      vehicles[0] = car;
7      vehicles[1] = ship;
8
9      for(inti = 0; i< vehicles.length; i++){
10         vehicles[i].move();
11     }
12  [...]
```



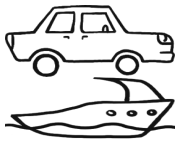
Podemos llamar a un método de la clase Vehicle y obtener el comportamiento más especializado de cada subclase

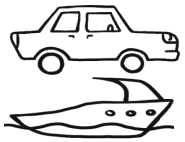
Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

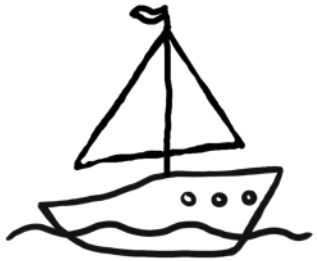
Polimorfismo de métodos

```
1   [...]
2      Vehicle car = new Car();
3      Vehicle ship = new Ship();
4
5      Vehicle[] vehicles =  new  Vehicle[2];
6      vehicles[0] = car;
7      vehicles[1] = ship;
8
9      for(inti = 0; i< vehicles.length; i++){
10         vehicles[i].move();
11     }
12  [...]
```











Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Polimorfismo de métodos

```
1   [...]
2       public class Car{
3         [...]
4         public void move(){
5           System.out.println("Wrumm wrumm honk honk…");
6         }
7       }
8   [...]
9
```





Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Polimorfismo de métodos

```
1   [...]
2       public class Ship{
3         [...]
4         public void move(){
5           System.out.println("Shhh  shhh…");
6         }
7       }
8   [...]
9
```

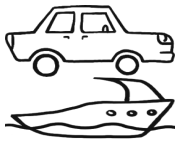


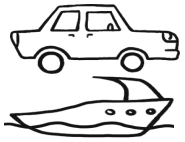


Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Polimorfismo de métodos

```
1   [...]
2      Vehicle car = new Car();
3      Vehicle ship = new Ship();
4
5      Vehicle[] vehicles =  new  Vehicle[2];
6      vehicles[0] = car;
7      vehicles[1] = ship;
8
9      for(int i = 0; i< vehicles.length; i++){
10         vehicles[i].move();
11     }
12  [...]
```

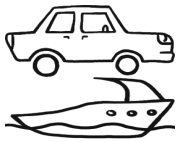




```
> Wrumm wrumm honk honk...
  Shh shh...
```

Polimorfismo de métodos

```
1   [...]
2      Vehicle car = new Car();
3      Vehicle ship = new Ship();
4
5      Vehicle[] vehicles =  new  Vehicle[2];
6      vehicles[0] = car;
7      vehicles[1] = ship;
8
9      for(inti = 0; i< vehicles.length; i++){
10         vehicles[i].move();
11         vehicles[i].sail();
12     }
13  [...]
```



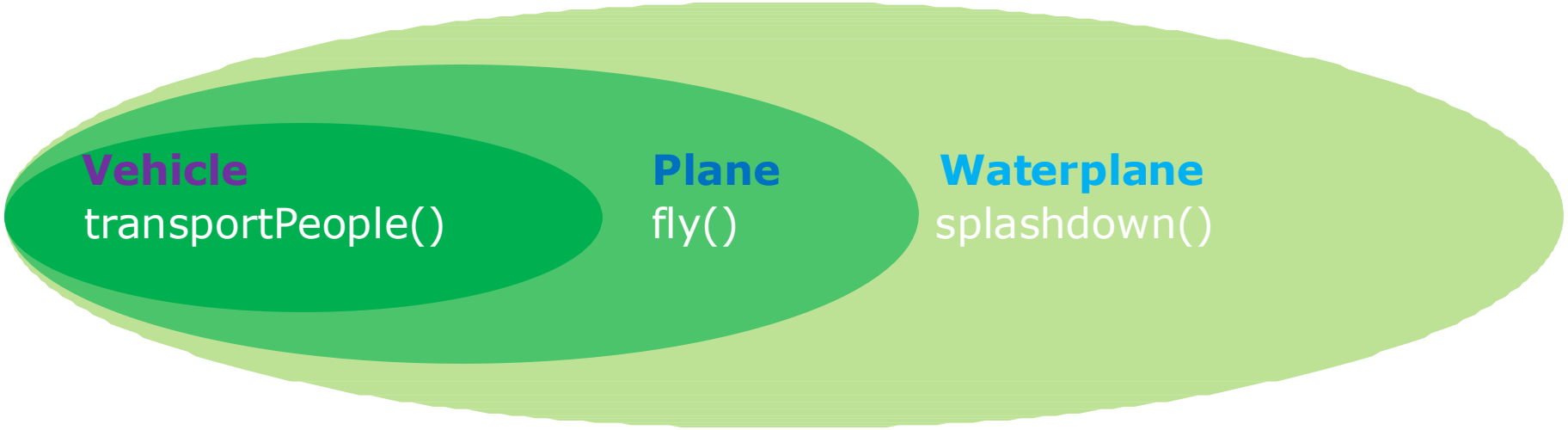
Solo `Ship` implementa `sail`

Pero recuerda que esto funciona porque move está declarado en la clase Vehicle

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Polimorfismo de métodos




```
Vehicle    p = new Waterplane(); p.transportPeople(); p.fly(); p.splashdown();
Plane      p = new Waterplane(); p.transportPeople(); p.fly(); p.splashdown();
Waterplane p = new Waterplane(); p.transportPeople(); p.fly(); p.splashdown();
```



Conceptos fundamentales

- Herencia
  - Constructores
- Polimorfismo
  - Subtipos y Sustitución
  - La Clase Object
  - Colecciones Polimórficas
  - Casting
  - Tipo Estático y Dinámico
  - Sobreescritura
  - En Métodos
- StringBuilder
- Clases y métodos abstractos
- Métodos de la clase Object
  - toString
  - equals
  - hashCode
- Extendiendo jerarquías de clases
- Herencia múltiple con Interfaces
- Acceso Protegido (protected)
- La clase “Class”

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

StringBuilder

- Preferibles a los String si el contenido puede cambiar gran número de veces al no ser inmutables como los Strings.
  - Algunos métodos importantes con StringBuilder:
    - append:
      - permite concatenar al final del StringBuilder existente un String o la representación en forma de String de un dato asociado a una variable primitiva
    - toString:
      - devuelve el StringBuilder como String

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

“network-v4-using-StringBuilder”

StringBuilder

```
public String getFormattedComments()
{
  int numberOfComments = comments.size();
  if (numberOfComments==0) return "\nNo comments.";
  else{
    StringBuilder builder = new StringBuilder();
    for(int commentNumber = 0;
        commentNumber<numberOfComments;commentNumber++) {
          builder.append("\nComment ");
          builder.append(commentNumber+1);
          builder.append(" of ");
          builder.append(numberOfComments);
          builder.append(":");
          builder.append('\n');
          builder.append(comments.get(commentNumber));
        }
     return builder.toString();
     }
  }
}
```

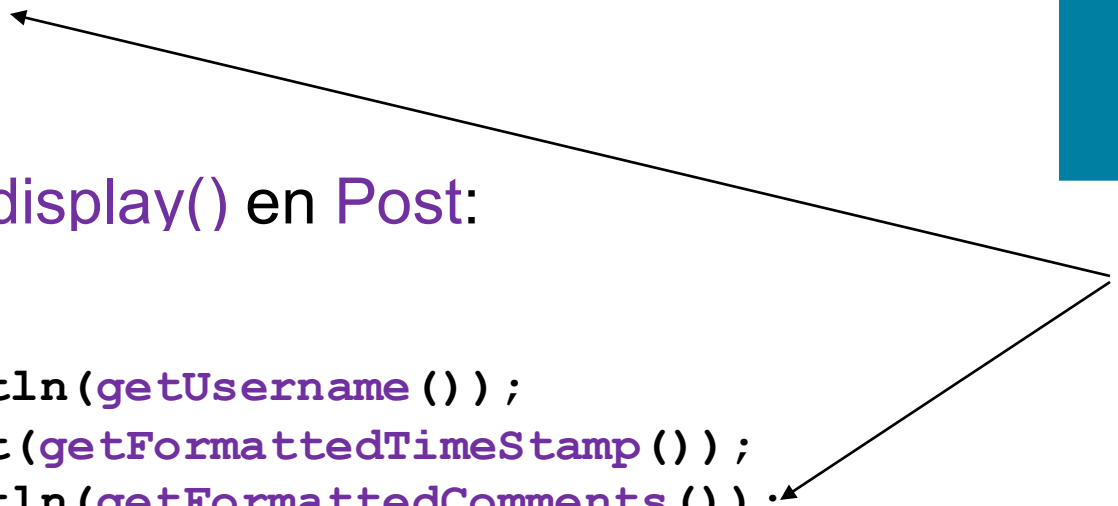
“network-v4-using-StringBuilder”
StringBuilder

- Teníamos este display() en Post:

```
public void display()
    {
        System.out.println(getUsername());
        System.out.println(getFormattedTimeStamp());
        printComments();
    }
```

- Ahora, tenemos este display() en Post:

```
public void display()
    {
        System.out.println(getUsername());
        System.out.print(getFormattedTimeStamp());
        System.out.println(getFormattedComments());
    }
```



Conceptos fundamentales

- Herencia
  - Constructores
- Polimorfismo
  - Subtipos y Sustitución
  - La Clase Object
  - Colecciones Polimórficas
  - Casting
  - Tipo Estático y Dinámico
  - Sobreescritura
  - En Métodos
- StringBuilder
- Clases y métodos abstractos
- Métodos de la clase Object
  - toString
  - equals
  - hashCode
- Extendiendo jerarquías de clases
- Herencia múltiple con Interfaces
- Acceso Protegido (protected)
- La clase “Class”

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Clases Abstractas

- Motivación para las **Clases Abstractas**

✓ `Car coche = new Car();`



✓ `Plane avion = new Plane();`



✓ `Bicycle bici = new Bicycle();`



❖ `Vehicle vehiculo = new Vehicle()`

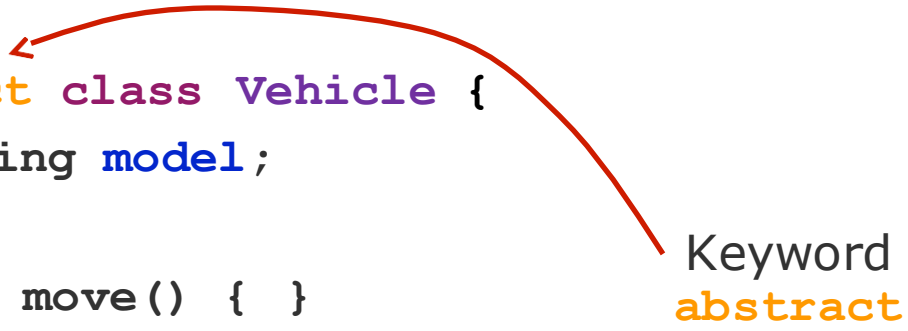


¿¿Qué sentido tiene una instancia de tipo Vehicle??

Clases Abstractas

- Las **Clases Abstractas no** pueden ser **instanciadas**
- Tienen que tener **subclases**
- **Solo** las **subclases** pueden ser **instanciadas**

```
1  public abstract class Vehicle {
2     private String model;
3
4     public void move() { }
5      [...]
6  }
```



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Clases Abstractas

- **Subclases** de **Clases** **Abstractas** también pueden ser **Abstractas**

```
1  public abstract class Vehicle {
2     [...]
3  }
```

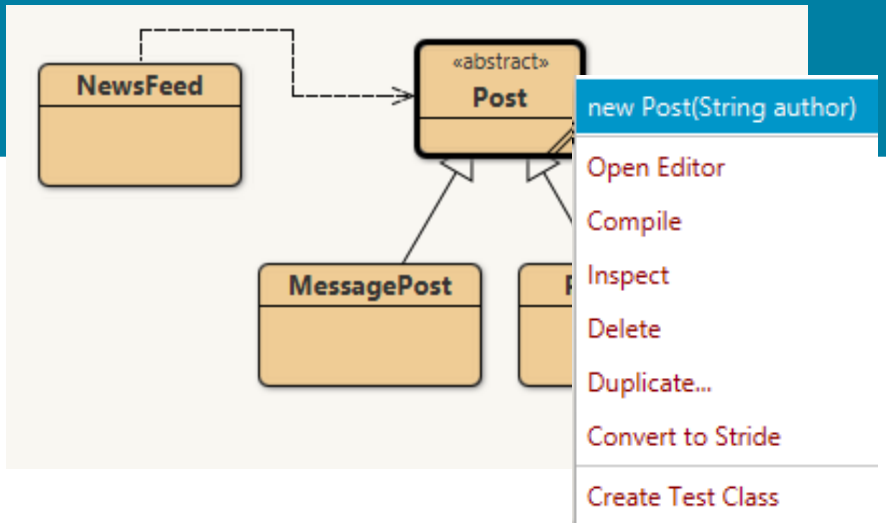
```
1  public abstract class Automobile extends Vehicle {
2     [...]
3  }
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

“network-v4-using-StringBuilder”

- Ahora mismo, podríamos crear instancias de tipo Post
  - ¿¿Qué sentido tiene una instancia de este tipo??

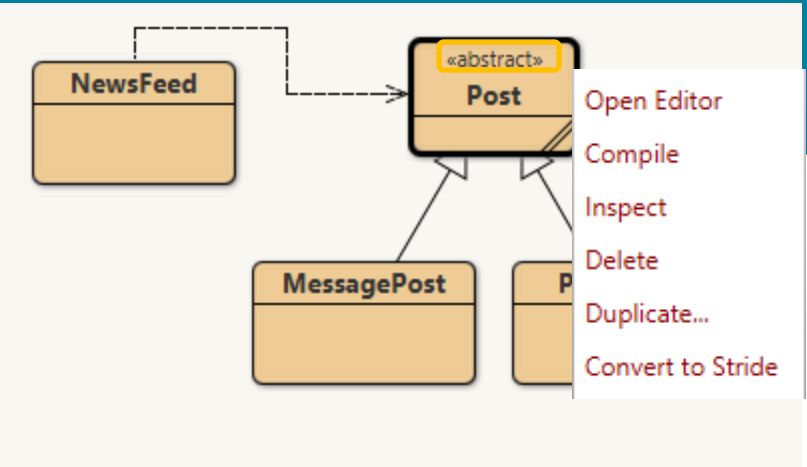


Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

“network-v5-post-abstract-class” Post como Clases Abstracta

```
public abstract class Post
{
…
}
```



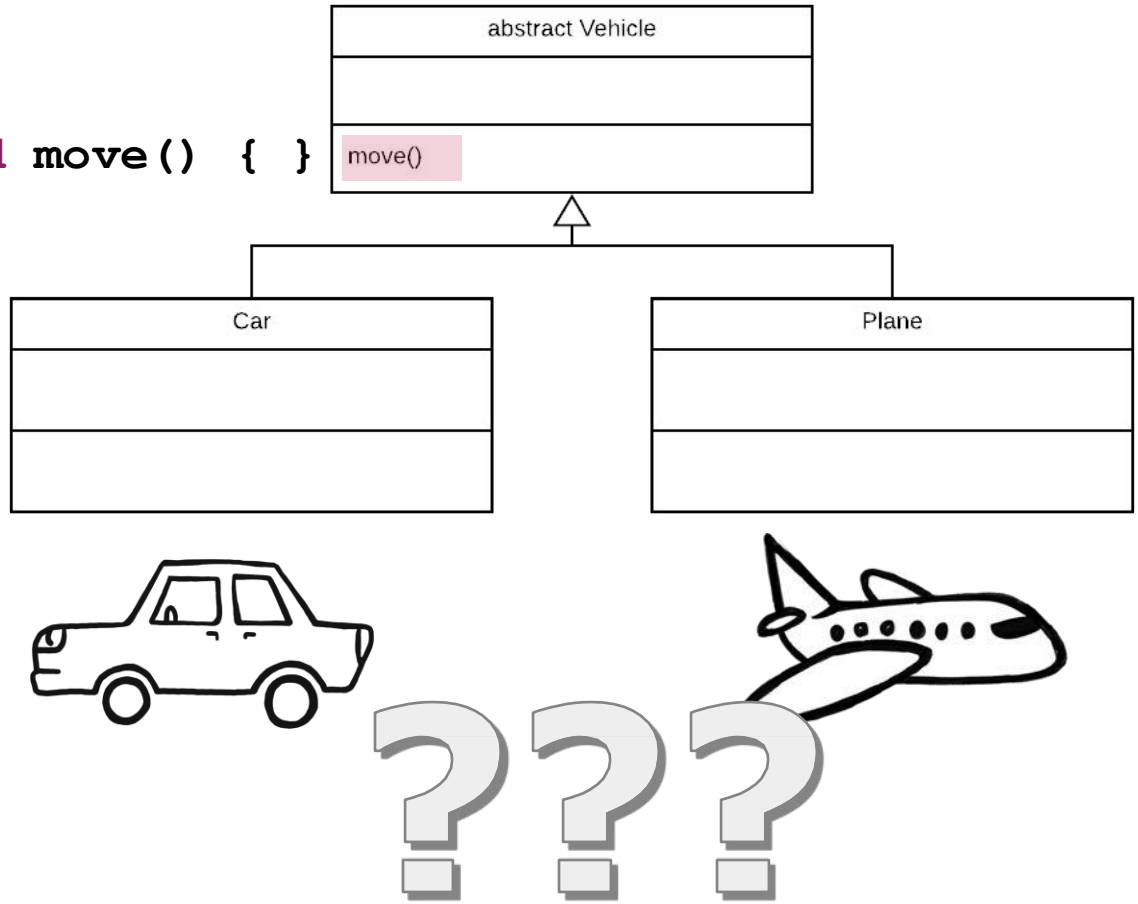
Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Métodos Abstractos

- Motivación para los **Métodos Abstractos**

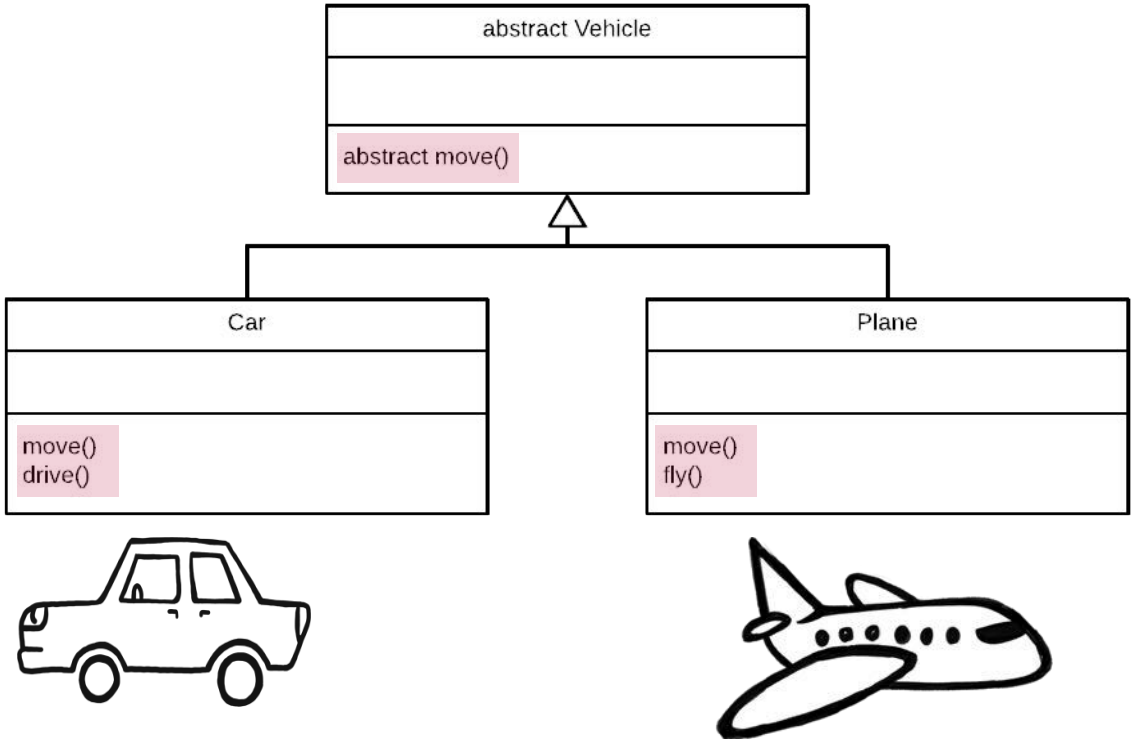
```
public void move() { }
```



¿¿Algo en común en movimiento de un coche y un avión??

Métodos Abstractos

- Motivación para los **Métodos Abstractos**



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Métodos Abstractos

- Métodos **Abstractos**
  - tienen `abstract` en la signatura.
  - no tienen cuerpo. -> la cabecera acaba con ;
    - no hay una implementación obvia compartida entre sus subclases
  - obliga a hacer a la clase **abstracta**.
- Cada subclase concreta realiza su implementación.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Métodos Abstractos

```
1   public abstract class Vehicle {
      public abstract void move();
3   }
```

Cabecera sin cuerpo

Keyword `abstract` en el método

```
1   public class Car
    extends Vehicle {
2      @Override
3
4
5
6
7       public void drive() {
8           [...]
9      }
10  }
```

```
1   public class Plane
    extends Vehicle {
2      @Override
3
4
5
6
7       public void fly() {
8           [...]
9      }
10  }
```

Cada subclase concreta realiza su implementación.

Métodos Abstractos

```
1  public abstract class Vehicle {
2      public abstract void move();
3  }
```

```
1  public class Car
   extends Vehicle {
2      @Override
3  Dado que esta clase tiene ya
4  implementado el método propio
5  drive() podemos reutilizar su
   funcionalidad para move()
6
7      public void drive() {
8          [...]
9      }
10 }
```

```
1  public class Plane
   extends Vehicle {
2      @Override
3  Dado que esta clase tiene ya
4  implementado el método propio
5  fly() podemos reutilizar su
   funcionalidad para move()
6
7      public void fly() {
8          [...]
9      }
10 }
```

Cada subclase concreta realiza su implementación.

Métodos Abstractos

```
1  public abstract class Vehicle {
2      public abstract void move();
3  }
```

```
1  public class Car
   extends Vehicle  {
2      @Override
3      public void move() {
4          drive();
5      }
6
7      private void drive() {
8          [...]
9      }
```

```
1  public class
   Plane      extends
   Vehicle {
2      @Override
3      public void move() {
4          fly();
5      }
6
7      private void fly() {
8          [...]
9      }
```

Cada subclase concreta realiza su implementación.

“network-v6-post-abstract-class-plus-abstract-method”
Método Abstracto en Post

```
/**
* A method that allow showing the details of a post.
* It is abstract because only
* non-abstract subclasses implement it
* (This class has`t got the specific details)
*/
public abstract String getFormattedContent();
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

“network-v6-post-abstract-class-plus-abstract-method”
Implementando Método Abstracto en MessagePost

```
/**
* A method that allow changing the details of a post.
* In this case it will return the text of a messagePost.
*/
@Override
public String getFormattedContent(){
                return getText();
        }
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

“network-v6-post-abstract-class-plus-abstract-method”
Implementando Método Abstracto en PhotoPost

```
/**
* A method that allow changing the details of a post.
* In this case it will return the text of a photoPost.
*/
@Override
public String getFormattedContent(){
            StringBuilder builder = new StringBuilder();
            builder.append("Photo path : ");
            builder.append(getFilename());
            builder.append("\n");
            builder.append(getCaption());
            return builder.toString();
      }
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

"network-v6-post-abstract-class-plus-abstract-method"
Cambiando Método *display* en Post

- Teníamos este display() en Post:

```
public void display()
    {
       System.out.println(getUsername());
       System.out.print(getFormattedTimeStamp());
       System.out.println(getFormattedComments());
    }
```

- Ahora, tenemos este display() en Post:

```
public void display()
    {
       System.out.println(getUsername());
       System.out.println(getFormattedContent());
       System.out.print(getFormattedTimeStamp());
       System.out.println(getFormattedComments());
    }
```

Además podemos eliminar los métodos *display* que habíamos sobreescrito en MessagePost y PhotoPost porque ya no son necesarios

Conceptos fundamentales

- Herencia
  - Constructores
- Polimorfismo
  - Subtipos y Sustitución
  - La Clase Object
  - Colecciones Polimórficas
  - Casting
  - Tipo Estático y Dinámico
  - Sobreescritura
  - En Métodos
- StringBuilder
- Clases y métodos abstractos
- Métodos de la clase Object
  - toString
  - equals
  - hashCode
- Extendiendo jerarquías de clases
- Herencia múltiple con Interfaces
- Acceso Protegido (protected)
- La clase “Class”

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Métodos de la clase `Object`

- Los métodos en `Object` son heredados por todas las clases.
- Cualquiera de ellos puede ser sobreescrito.
  - Recomendable sobreescribir:
    - `public String toString()`
    - `public boolean equals(Object obj)`
    - `public int hashCode()`

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales

- Herencia
  - Constructores
- Polimorfismo
  - Subtipos y Sustitución
  - La Clase Object
  - Colecciones Polimórficas
  - Casting
  - Tipo Estático y Dinámico
  - Sobreescritura
  - En Métodos
- StringBuilder
- Clases y métodos abstractos
- Métodos de la clase Object
  - toString
  - equals
  - hashCode
- Extendiendo jerarquías de clases
- Herencia múltiple con Interfaces
- Acceso Protegido (protected)
- La clase “Class”

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Método toString

- Método de la clase **`Object`**

```
public String toString()
```

– Devuelve representación del objeto en forma de **`String`**.

– Por defecto: *ClassName@HashCode*
  – Ej: `Car@70dea4e`

– El método **`toString`** es recomendable sobreescribirlo en las clases para que muestre los valores significativos del objeto.
  – Es llamado de forma implícita con invocaciones:
    - **`System.out.print(variableObjeto)`**
    - **`System.out.println(variableObjeto)`**

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

“network-v7-toString-deleting-display” Sobreescribiendo *toString* en Post

```
@Override
public String toString(){
StringBuilder builder = new StringBuilder();
builder.append(getUsername());
builder.append('\n');
builder.append(getFormattedContent());
builder.append('\n');
builder.append(getFormattedTimeStamp());
builder.append(getFormattedComments());
builder.append('\n');
return builder.toString();
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

"network-v7-toString-deleting-display" Eliminando *display* en Post

- No necesitamos el método **display** en **Post**
  - Métodos de visualización explícita como **display** pueden ser a menudo omitidos de una clase usando el método **toString** sobreescrito en su lugar:

```
System.out.println(post.toString());
```

  - Es más, dado que las invocaciones a `println` con un objeto automáticamente invocan al método **toString** del objeto, podemos dejar la sentencia así:

```
System.out.println(post);
```

"network-v7-toString-deleting-display" Cambiando *show* en NewsFeed

- Antes en NewsFeed:

```
public void show()
{
    for(Post post : posts) {
        post.display();
        System.out.println(---------); }
}
```

- Ahora en NewsFeed:

```
public void show()
{
    for(Post post : posts) {
        System.out.println(post);
        System.out.println("---------"); }
}
```

Copyright © 2021, 2022, 2023 Pearson Education, Ltd. All rights reserved

Conceptos fundamentales

- Herencia
  - Constructores
- Polimorfismo
  - Subtipos y Sustitución
  - La Clase Object
  - Colecciones Polimórficas
  - Casting
  - Tipo Estático y Dinámico
  - Sobreescritura
  - En Métodos
- StringBuilder
- Clases y métodos abstractos
- Métodos de la clase Object
  - toString
  - equals
  - hashCode
- Extendiendo jerarquías de clases
- Herencia multiple con Interfaces
- Acceso Protegido (protected)
- La clase “Class”

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Método equals

- Ya hemos visto en unidades anteriores qué sucede si comparamos variables de tipo String con **==** o con **equals**:
  - **==** : Igualdad de Referencia.
  - **equals**: Igualdad de Contenido.
- La forma de probar de **equals** la Igualdad de Contenido entre dos objetos es:
  - probar si los valores de sus campos son iguales.
- El método **equals** es necesario sobreescribirlo en nuestras **clases**
  - Si no lo hacemos, por defecto usarán el heredado de **Object** que compara las direcciones de memoria (**==**)

Método equals

- Plantilla de sobreescritura de **equals**:

Operador que **solo debe usarse** para **determinar** el Tipo Dinámico en método equals

```
@Override
public boolean equals(Object obj){
    if(this == obj) {
        return true; //Ambos referencian al mismo objeto
    }
    if(!(obj instanceof ThisType)) {
        return false; //Tienen diferentes tipos
    }
    ThisType other = (ThisType) obj;
    … compara cada campo significativo de
      this y other:
       return true si todos los valores iguales
       return false si algún valor distinto
}
```

“network-v8-with-equals” Sobreescribiendo *equals* en Post

```
@Override
public boolean equals(Object obj){
 if(this == obj) {
   return true; //Ambos referencian al mismo Post
 }
 if(!(obj instanceof Post)) {
   return false; //Tienen diferentes tipos
 }

 Post other = (Post) obj;

 return getUsername().equals(other.getUsername());
}
```

Decidir qué campos son significativos en *equals* dependerá de los requisitos específicos del problema que se esté tratando de resolver. En este caso hemos decidido que todos los post abstractos con un mismo nombre de usuario se consideran iguales

"network-v8-with-equals"
Sobreescribiendo *equals* en MessagePost

```
@Override
public boolean equals(Object obj){
  if(this == obj) {
    return true; // Ambos referencian al mismo MessagePost
  }
  if(!(obj instanceof MessagePost)) {
    return false; // Tienen diferentes tipos
  }

 MessagePost other = (MessagePost) obj;

  return super.equals(other) &&
      getText().equals(other.getText());}
```

Decidir qué campos son significativos en *equals* dependerá de los requisitos específicos del problema que se esté tratando de resolver. En este caso hemos decidido que además de lo indicado en Post, todos los messagePost que tengan el mismo texto serán iguales

"network-v8-with-equals" Sobreescribiendo *equals* en PhotoPost

```
@Override
public boolean equals(Object obj){
  if(this == obj) {
    return true; // Ambos referencian al mismo PhotoPost
  }
  if(!(obj instanceof PhotoPost)) {
    return false; // Tienen diferentes tipos
  }

PhotoPost other = (PhotoPost) obj;

  return super.equals(other) &&
      getFilename().equals(other.getFilename());}
```

Decidir qué campos son significativos en *equals* dependerá de los requisitos específicos del problema que se esté tratando de resolver. En este caso hemos decidido que además de lo indicado en Post, todos los photoPost que tengan el mismo nombre de archivo serán iguales

Conceptos fundamentales

- Herencia
  - Constructores
- Polimorfismo
  - Subtipos y Sustitución
  - La Clase Object
  - Colecciones Polimórficas
  - Casting
  - Tipo Estático y Dinámico
  - Sobreescritura
  - En Métodos
- StringBuilder
- Clases y métodos abstractos
- Métodos de la clase Object
  - toString
  - equals
  - hashCode
- Extendiendo jerarquías de clases
- Herencia múltiple con Interfaces
- Acceso Protegido (protected)
- La clase “Class”

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Método hashCode

- Básicamente, devuelve un valor entero que debería representar de forma inequívoca a un objeto.
- Es usado por colecciones como **HashMap** y **HashSet** (las veremos en una unidad posterior) para proporcionar una ubicación y una búsqueda de objetos eficientes en dichas colecciones.
  - El método **contains** de estas estructuras trabaja con **hashCode** en lugar de **equals** porque es mucho más rapido comparando objetos
- Con la implementación por defecto en **Object**, distintos objetos tienen distintos valores de **hashCode**.

Método hashCode

- Hay una relación importante entre los métodos **equals** y **hashCode**:
  - Cada vez que se sobreescribe el método **equals**, el método **hashCode** también debería ser sobreescrito.
  - dos objetos que son devueltos como iguales por el método **equals** deben devolver valores idénticos de **hashCode**
    - por tanto, deben usarse para su cálculo los mismos campos significativos utilizado en el método **equals**

Método hashCode

- Reglas para construir un `hashCode`:
  - inicializar con un número primo
  - si existen campos con valores numéricos:
    - usarlo para ese campo convirtiéndolo a int
  - si existen campos con su propio `hashCode`:
    - usarlo para ese campo
  - multiplicar los valores obtenidos de cada campo por números primos
  - sumar todos los valores
  - Ej en una clase Persona con campos peso y altura:
    - `hashCode`=17 + (getPeso() * 7) + (getAltura() * 13);

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

“network-v9-with-equals-and-hashcode” Sobreescribiendo `hashCode` en `Post`

```
/**
 * Hashcode technique taken from
 * Effective Java by Joshua Bloch.
 */
@Override
public int hashCode()
{
 int result = 7;
 result = 3 * result + getUsername().hashCode();
 return result;
}
```

Se usa el nombre de usuario para realizar el cálculo pues es el campo significativo utilizado en método *equals*

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

"network-v9-with-equals-and-hashcode" Sobreescribiendo `hashCode` en `MessagePost`

```
/**
 * Hashcode technique taken from
 * Effective Java by Joshua Bloch.
 */
@Override
public int hashCode()
{
    int result = 7;
    result = 3 * result + super.hashCode();
    result = 5 * result + getText().hashCode();
    return result;
}
```

Junto con el cálculo realizado en la superclase para realizar el cálculo, se usa también el contenido del texto del messagePost pues es el campo significativo utilizado en su método *equals*

"network-v9-with-equals-and-hashcode" Sobreescribiendo `hashCode` en `PhotoPost`

```
/**
 * Hashcode technique taken from
 * Effective Java by Joshua Bloch.
 */
@Override
public int hashCode()
{
    int result = 7;
    result = 3 * result + super.hashCode();
    result = 5 * result + getFilename().hashCode();
    return result;
}
```

Junto con el cálculo realizado en la superclase para realizar el cálculo, se usa también el nombre del ficherp del photoPost pues es el campo significativo utilizado en su método *equals*

Conceptos fundamentales

- Herencia
  - Constructores
- Polimorfismo
  - Subtipos y Sustitución
  - La Clase Object
  - Colecciones Polimórficas
  - Casting
  - Tipo Estático y Dinámico
  - Sobreescritura
  - En Métodos
- StringBuilder
- Clases y métodos abstractos
- Métodos de la clase Object
  - toString
  - equals
  - hashCode
- Extendiendo jerarquías de clases
- Herencia múltiple con Interfaces
- Acceso Protegido (protected)
- La clase “Class”

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

“network-v10-adding-new-types-and-deeper-hierarchies”
Extendiendo jerarquías de clases

Jerarquías de clases se pueden extender indefinidamente:

- Nuestra red social a partir de ahora tiene que gestionar también un nuevo tipo de publicación llamada evento (`EventPost`) que extiende al tipo mensaje (`MessagePost`)
  - Esta clase tendrá como campos propios:
    - Lugar del evento (location)
    - Fecha del evento (date)

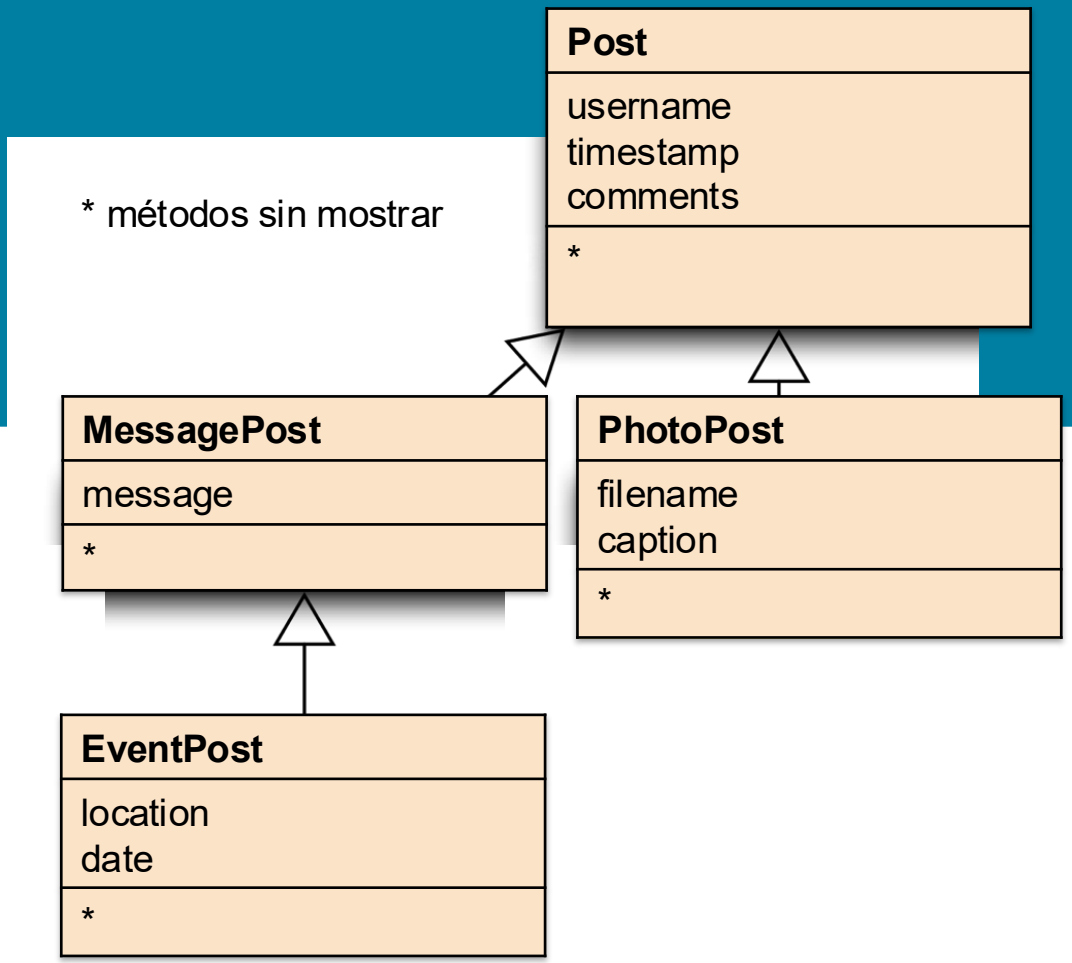
Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

“network-v10-adding-new-types-and-deeper-hierarchies”

Extendiendo jerarquías de clases

Añadimos un nuevo tipo de publicación llamado EventPost



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

"network-v10-adding-new-types-and-deeper-hierarchies"

Extendiendo jerarquías de clases

Añadimos un nuevo tipo de publicación llamado EventPost

```
public class EventPost extends MessagePost
{
    private String location;
    private String date;

    /**
     * Constructor for objects of class EventPost
     */
    public EventPost(String author, String text,
                     String location, String date)
    {
       super(author, text);// DEBE ser la primera sentencia
       this.location = location;
       this.date = date;
    }
    // methods omitted
}
```

“network-v10-adding-new-types-and-deeper-hierarchies”

Extendiendo jerarquías de clases

Añadimos un nuevo tipo de publicación llamado EventPost

```
public class EventPost extends MessagePost
{ …
    public String getLocation (){
        return location;
    }
    public String getDate (){
        return date;
    }
…
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

“network-v10-adding-new-types-and-deeper-hierarchies”

Extendiendo jerarquías de clases

Añadimos un nuevo tipo de publicación llamado EventPost

```
public class EventPost extends MessagePost
{…
/**
* A method that allow changing the details of a post.
* In this case it will return the text of a eventPost.
*/
@Override
public String getFormattedContent(){
                StringBuilder builder = new StringBuilder();
                builder.append(super.getFormattedContent());
                builder.append("Location : ");
                builder.append(getLocation ());
                builder.append("\n");
                builder.append(“Date : ");
                builder.append(getDate());
                return builder.toString();
        }
}
```

“network-v10-adding-new-types-and-deeper-hierarchies”

Extendiendo jerarquías de clases

Añadimos un nuevo tipo de publicación llamado EventPost

```
public class EventPost extends MessagePost
{…
@Override
public boolean equals(Object obj){
  if(this == obj) {
    return true; // Ambos referencian al mismo EventPost
  }
  if(!(obj instanceof EventPost)) {
    return false; // Tienen diferentes tipos
  }
 EventPost other = (EventPost) obj;
 return super.equals(other) &&
        getLocation().equals(other.getLocation()) &&
        getDate().equals(other.getDate());
 }
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

“network-v10-adding-new-types-and-deeper-hierarchies”

Extendiendo jerarquías de clases

Añadimos un nuevo tipo de publicación llamado EventPost

```
public class EventPost extends MessagePost
{…
/**
 * Hashcode technique taken from
 * Effective Java by Joshua Bloch.
 */
@Override
public int hashCode()
{
    int result = 7;
    result = 3 * result + super.hashCode();
    result = 5 * result + getLocation().hashCode();
    result = 7 * result + getDate().hashCode();
    return result;
}
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

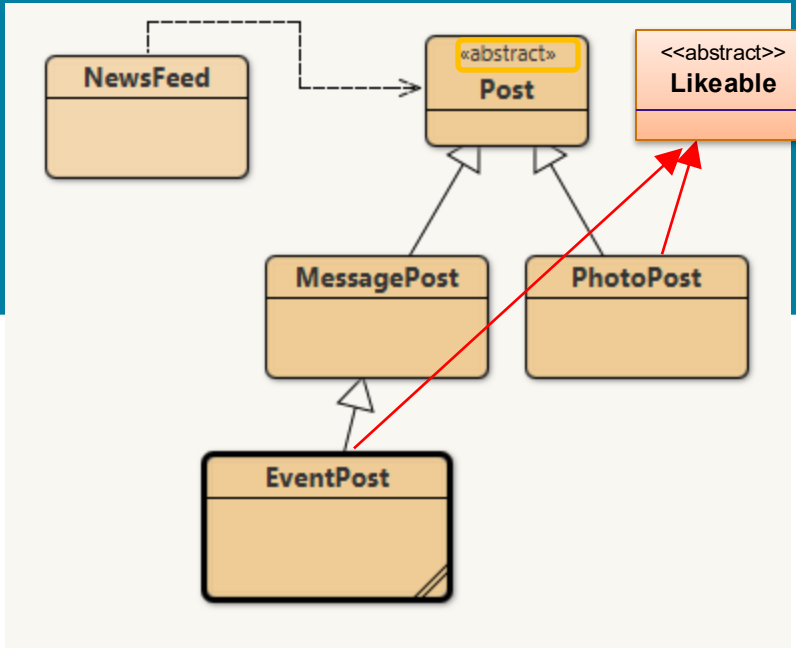
Conceptos fundamentales

- Herencia
  - Constructores
- Polimorfismo
  - Subtipos y Sustitución
  - La Clase Object
  - Colecciones Polimórficas
  - Casting
  - Tipo Estático y Dinámico
  - Sobreescritura
  - En Métodos
- StringBuilder
- Clases y métodos abstractos
- Métodos de la clase Object
  - toString
  - equals
  - hashCode
- Extendiendo jerarquías de clases
- Herencia múltiple con Interfaces
- Acceso Protegido (protected)
- La clase “Class”

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

“network-v10-adding-new-types-and-deeper-hierarchies”
Extendiendo jerarquías de clases

¿Qué ocurre si nos piden que solo los tipos PhotoPost y EventPost implementen funcionalidad que permita hacer “Me gusta”?



Podríamos pensar en añadir clase Likeable con funcionalidad común para hacer “Me gusta” y que PhotoPost y EventPost hereden de esa nueva clase

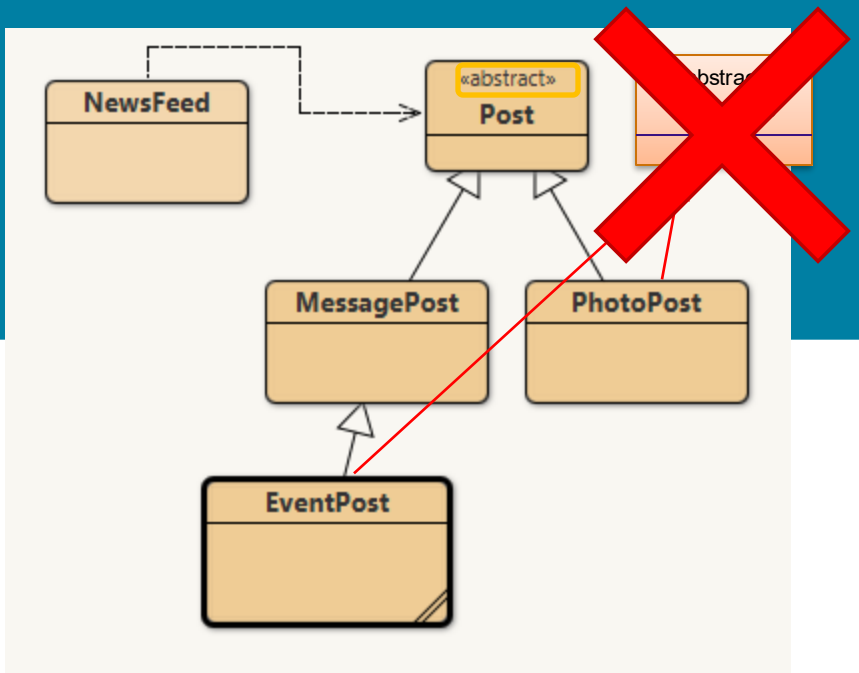
Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

“network-v10-adding-new-types-and-deeper-hierarchies”

Extendiendo jerarquías de clases

¿Qué ocurre si nos piden que solo los tipos PhotoPost y EventPost implementen funcionalidad que permita hacer “Me gusta”?



ERROR:

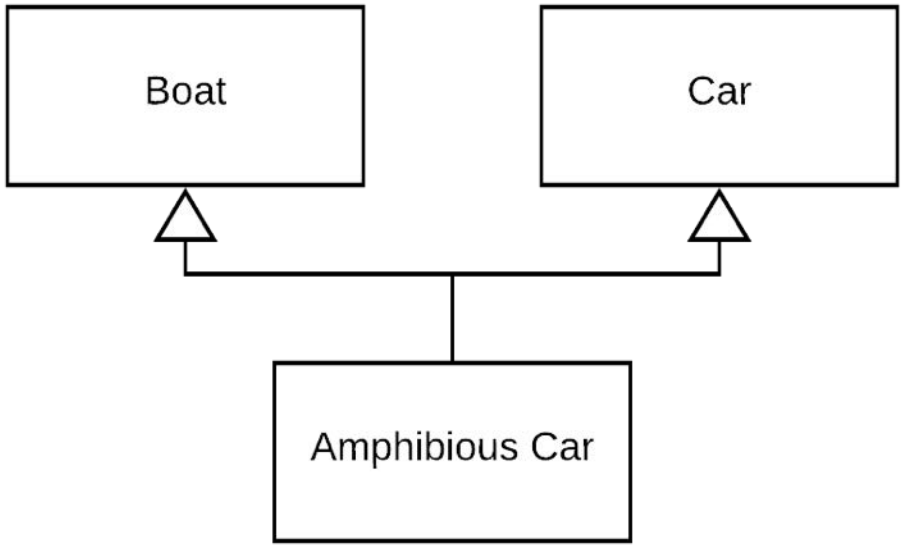
¡¡Java NO soporta herencia multiple!!

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Herencia Múltiple

- Es cuando una subclase hereda directamente de múltiples clases.
- De entrada puede parecer una buena idea pero…

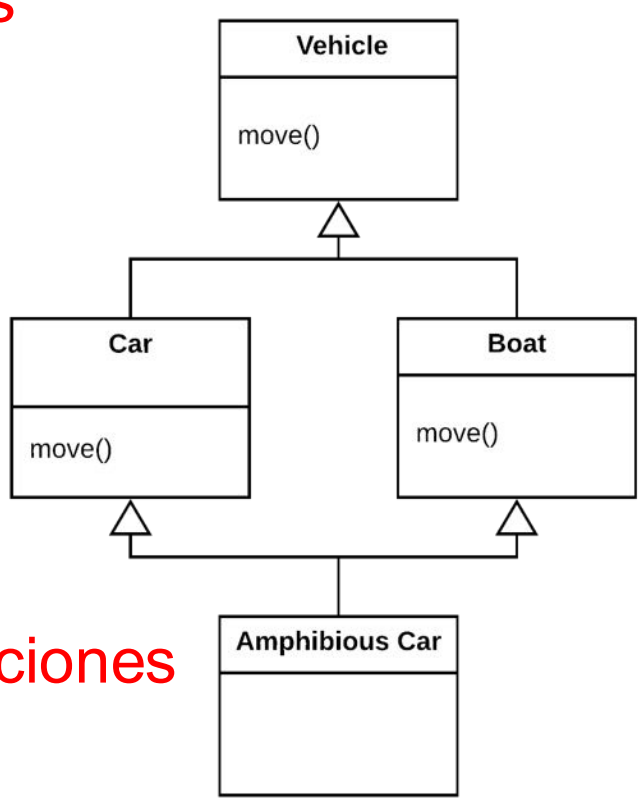


Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Herencia Múltiple

- De entrada puede parecer una buena idea pero…
  - puede causar muchos problemas

`Boat` y `Car` definen el método move()



¿Cuál de las dos implementaciones hereda `AmphibiousCar` ?

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Herencia Múltiple

- Cada lenguaje tiene sus propias reglas.
  - ¿Cómo resolver herencia de definiciones iguales?
- **Java** prohibe herencia múltiple de clases.
- **Java** permite herencia múltiple de **`interfaces`**.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Interfaces

- Proporcionan un "contrato" donde se declaran los métodos.
- Cada vez que una **clase no abstracta** implementa un `interface` debe implementar **todos** los métodos declarados en dicho `interface`.
- Todas las **clases** que implementan un `interface` tendrán por tanto su propia definición de los métodos declarados en dicho `interface`.
- Se **usan** `interfaces` para **asegurar** que **clases** que **no están en la misma jerarquía** **implementan un conjunto común de métodos**

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Características de los Interfaces

- Usan `interface` en lugar de `class` en su declaración.
- No definen constructores.
- Todos los métodos son `public` y `abstract`*.
  - Se pueden omitir estas keywords en su declaración.
- Todos los campos son `public`, `static` y `final`.
  - Se pueden omitir estas keywords en su declaración.
- Las clases que los implementan son subtipos del tipo `interface`.
- Por tanto, temenos **polimorfismo** con `interfaces` al igual que con clases.

* Desde Java 8 se permite definir métodos por defecto pero en esta asignatura obviaremos esa característica porque complican la herencia desde multiples interfaces

Interfaces

```
public interface <<Identificador>> {
    public abstract void <<identificadorMétodo>>();
    void <<otroIdentificadorMétodo>>();
}
```



Identificadores de interfaces, por convención, suelen acabar en sufijo -able

```
1  public interface Colorable {
2    public abstract String getColor();
3    void setColor(int r, int g, int b, int a);
4    void setColor(String hex);
5  }
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Interfaces

```
1  public interface Colorable {
2     public abstract String getColor();
3     [...]
4  }
```

```
1  class Car implements Colorable{
2     @Override
3     public String getColor(){
4         return "#00FF00" //green
4     }
5     [...]
6  }
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Interfaces

```
1  public interface Colorable {
2     public abstract String getColor();
3     [...]
4  }
```

```
1  class Car implements Colorable {
2     String color;
3     @Override
4     public String getColor(){
5         return this.color;
6     }
7     [...]
8     }
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Interfaces

```
1  public interface Colorable {
2     public abstract String getColor();
3     [...]
4  }
```

Clases puede extender una **superclases** e implementar varios **interfaces**

Primero se declara la **superclase**, luego los **interfaces** (separados por ,)

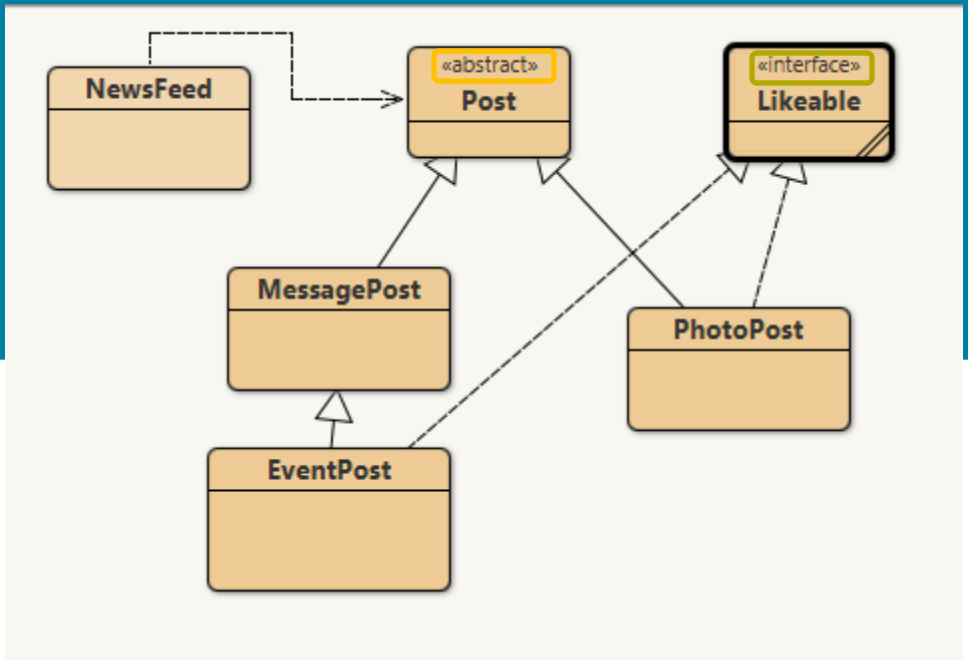
```
1  class Car extends Automobile implements Colorable, Flyable {
2     @Override
3     public String getColor(){
4          [...]
5     }
6   [...]
7  }
```

- 1º: definición de **clase**
- 2º: `extends` **superclase**
- 3º: `implements` **interface(s)**

“network-v11-adding-multiple-inheritance-with-interfaces”

Herencia múltiple usando Interface

Añadimos un interface llamado Likeable



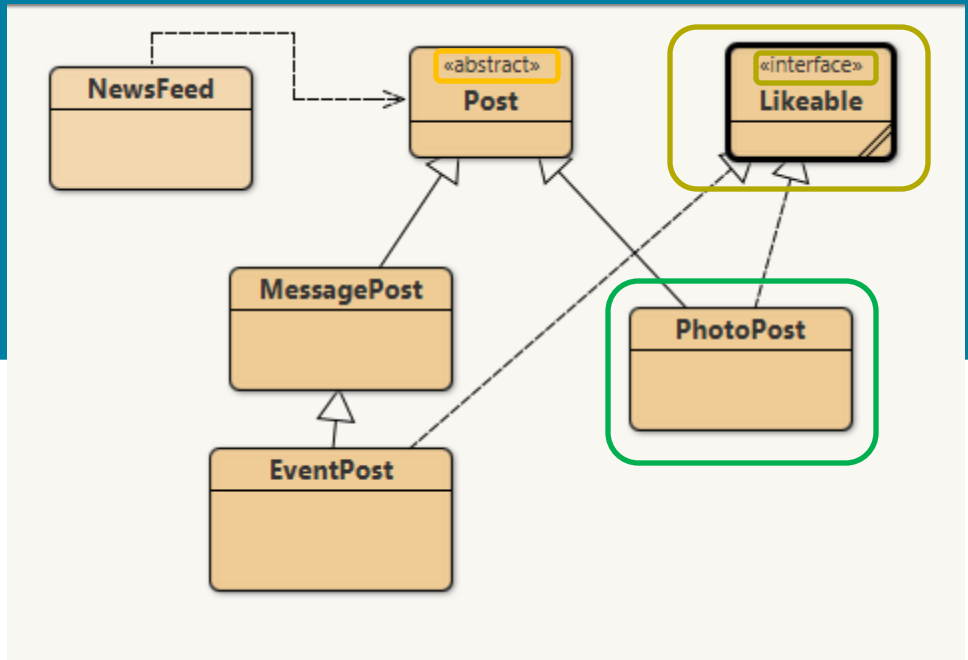
Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

“network-v11-adding-multiple-inheritance-with-interfaces”

Herencia múltiple usando Interface

Añadimos un interface llamado Likeable



PhotoPost es una publicación que ofrece hacer “Me gusta”

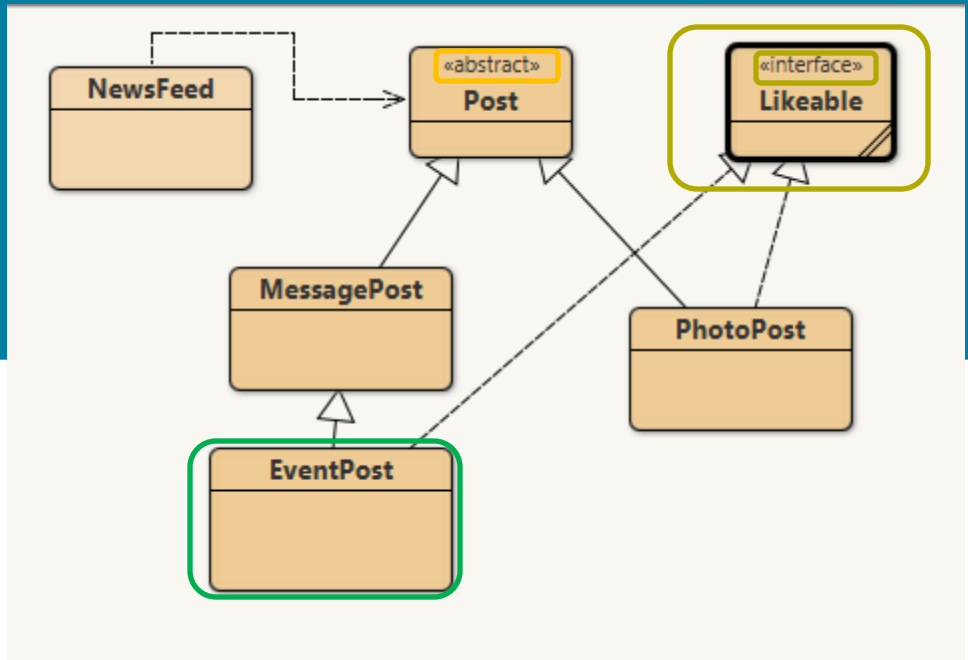
Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

“network-v11-adding-multiple-inheritance-with-interfaces”

Herencia múltiple usando Interface

Añadimos un interface llamado Likeable



EventPost es una publicación que ofrece hacer “Me gusta”

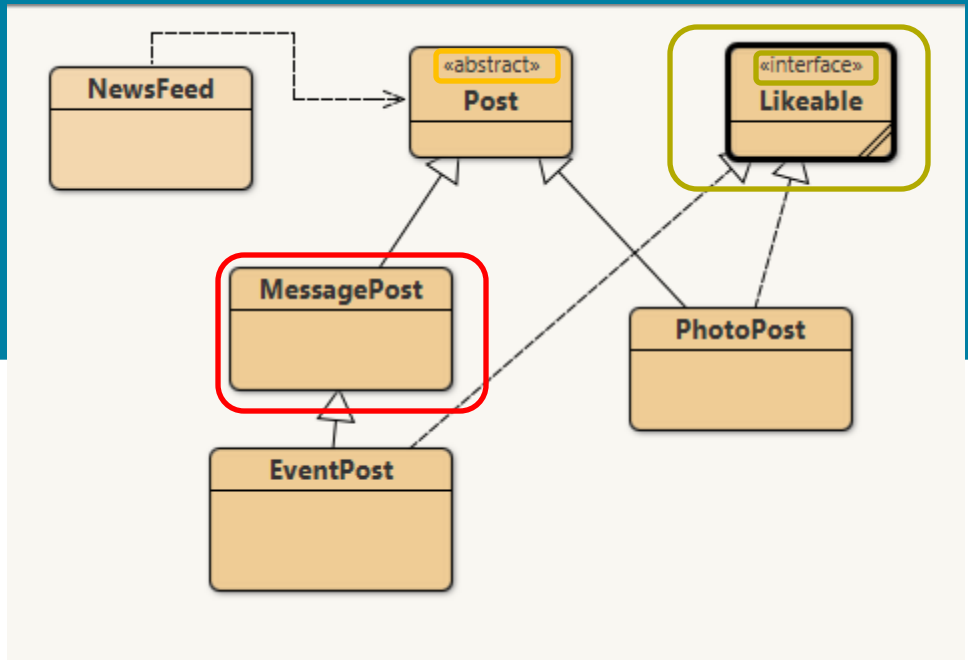
Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

“network-v11-adding-multiple-inheritance-with-interfaces”

Herencia múltiple usando Interface

Añadimos un interface llamado Likeable



MessagePost es una publicación que NO ofrece hacer “Me gusta”

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

"network-v11-adding-multiple-inheritance-with-interfaces"

Herencia múltiple usando Interface

```
/**
 * Interface for those post that allow likes.
 */
public interface Likeable{
  /**
  * Record one more 'Like' indication from a user.
  */
  public abstract void like();
  /**
  * Record that a user has withdrawn his/her 'Like' vote.
  */
  void unlike();
  /**
  * Return the number of likes of this class.
  * @return The class' number of likes.
  */
  int getLikes();
}
```

"network-v11-adding-multiple-inheritance-with-interfaces"

Herencia múltiple usando Interface

Las clases implementan el interface

```
public class EventPost extends MessagePost implements Likeable{
    private String description;
    private String location;
    private String date;
    private int likes;
    ...
}
```

Interfaces se implementan,
no se extienden

```
public class PhotoPost extends Post implements Likeable{
    private String filename;
    private String caption;
    private int likes;
    ...
}
```

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

“network-v11-adding-multiple-inheritance-with-interfaces”

Herencia múltiple usando Interface

```
public class EventPost extends MessagePost implements Likeable{
    public EventPost(...)
    {
      ...
     likes = 0;
    }
  ...
}
```

Cada clase inicializa la variable likes, no tienen por qué inicializar con el mismo valor

```
public class PhotoPost extends Post implements Likeable{
 public PhotoPost(...)
    {
      ...
     likes = 0;
    }
  ...
}
```

eserved

"network-v11-adding-multiple-inheritance-with-interfaces"

Herencia múltiple usando Interface

```
public class EventPost extends MessagePost implements Likeable{
/**Record one more 'Like' indication from a user.*/
@Override
public void like()
     {
     likes++;
     }
  ...
}
```

Cada clase implementa los métodos declarados en Likeable,

no tienen por qué implementarlos de la misma manera

```
public class PhotoPost extends Post implements Likeable{
/**Record one more 'Like' indication from a user.*/
@Override
public void like()
     {
     likes++;
     }
  ...
}
```

served

"network-v11-adding-multiple-inheritance-with-interfaces"

Herencia múltiple usando Interface

```
public class EventPost extends MessagePost implements
Likeable{
/**Return the number of likes of this post.
   @return The post's number of likes.*/
@Override
public int getLikes()
    {
    return likes;
    }
 ...
```

Cada clase implementa los métodos declarados en Likeable,

no tienen por qué implementarlos de la misma manera

```
public class PhotoPost extends Post implements Likeable{
/**Return the number of likes of this post.
   @return The post's number of likes.*/
@Override
public int getLikes()
    {
    return likes;
    }
 ...
}
```

“network-v11-adding-multiple-inheritance-with-interfaces”

Herencia múltiple usando Interface

```
public class EventPost extends MessagePost implements Likeable{
/** Record that a user has withdrawn his/her 'Like' vote.
    It doesn´t allow having a negative number of likes.*/
@Override
public void unlike()
     {
     if (likes > 0) likes--;
     }
  ...
}
```

Cada clase implementa los métodos declarados en Likeable,

no tienen por qué implementarlos de la misma manera

```
public class PhotoPost extends Post implements Likeable{
/**Record that a user has withdrawn his/her 'Like' vote.
 * It does allow having a negative number of likes.*/
@Override
public void unlike()
     {
     likes--;
     }
  ...
}
```

Interfaces como Especificaciones

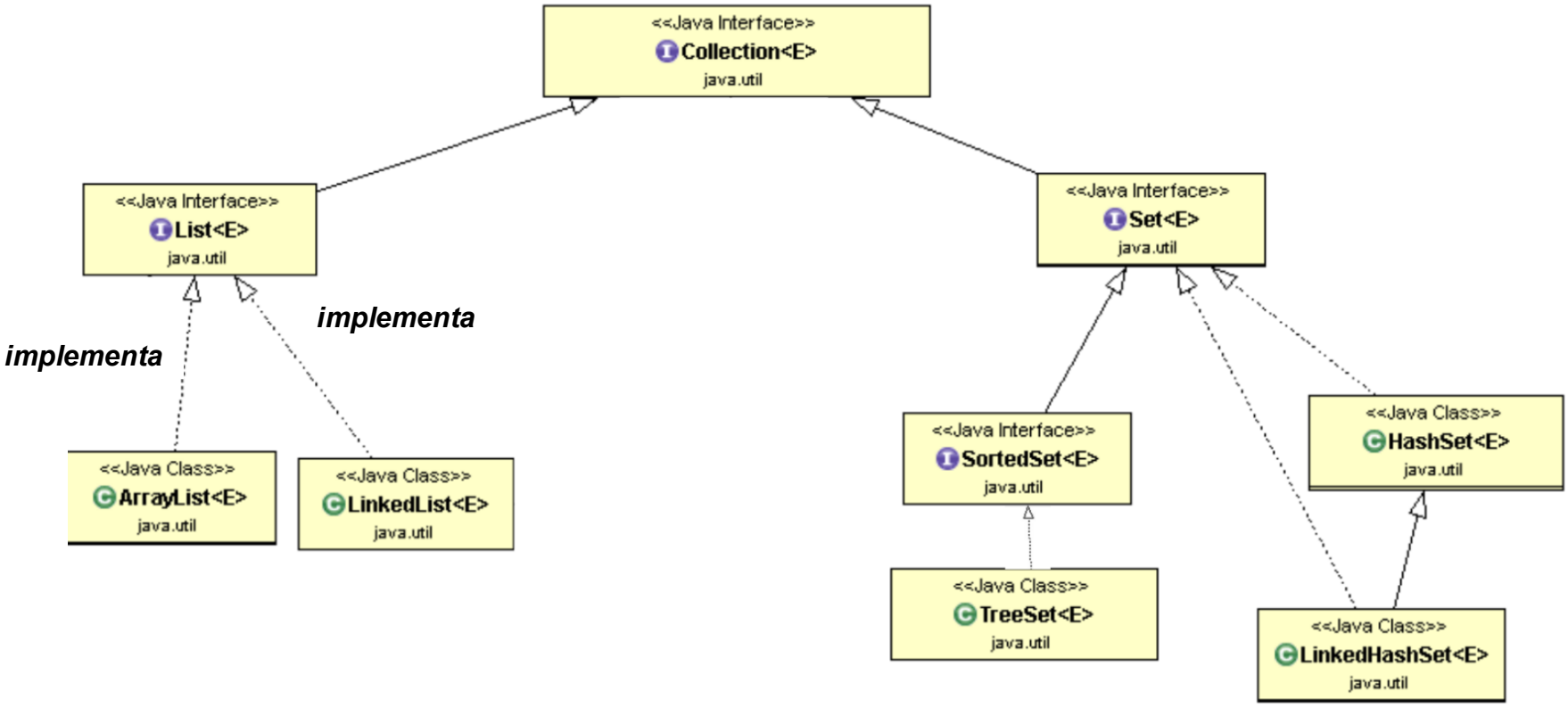
- El uso de `interface` proporciona múltiples ventajas:
  - Fuerte separación de funcionalidad proporcionada con respecto a la implementación.
    - Solo los tipos de los parámetros de los métodos y su tipo de retorno son obligatorios.
  - Clientes interactúan independientemente de la implementación.
    - Además, los clientes pueden elegir entre implementaciones alternativas.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Interfaces como Especificaciones

- Las colecciones **`List`**, **`Map`** y **`Set`** que veremos con más detalle en una unidad posterior son ejemplo de estas ventajas



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales

- Herencia
  - Constructores
- Polimorfismo
  - Subtipos y Sustitución
  - La Clase Object
  - Colecciones Polimórficas
  - Casting
  - Tipo Estático y Dinámico
  - Sobreescritura
  - En Métodos
- StringBuilder
- Clases y métodos abstractos
- Métodos de la clase Object
  - toString
  - equals
  - hashCode
- Extendiendo jerarquías de clases
- Herencia múltiple con Interfaces
- Acceso Protegido (protected)
- La clase “Class”

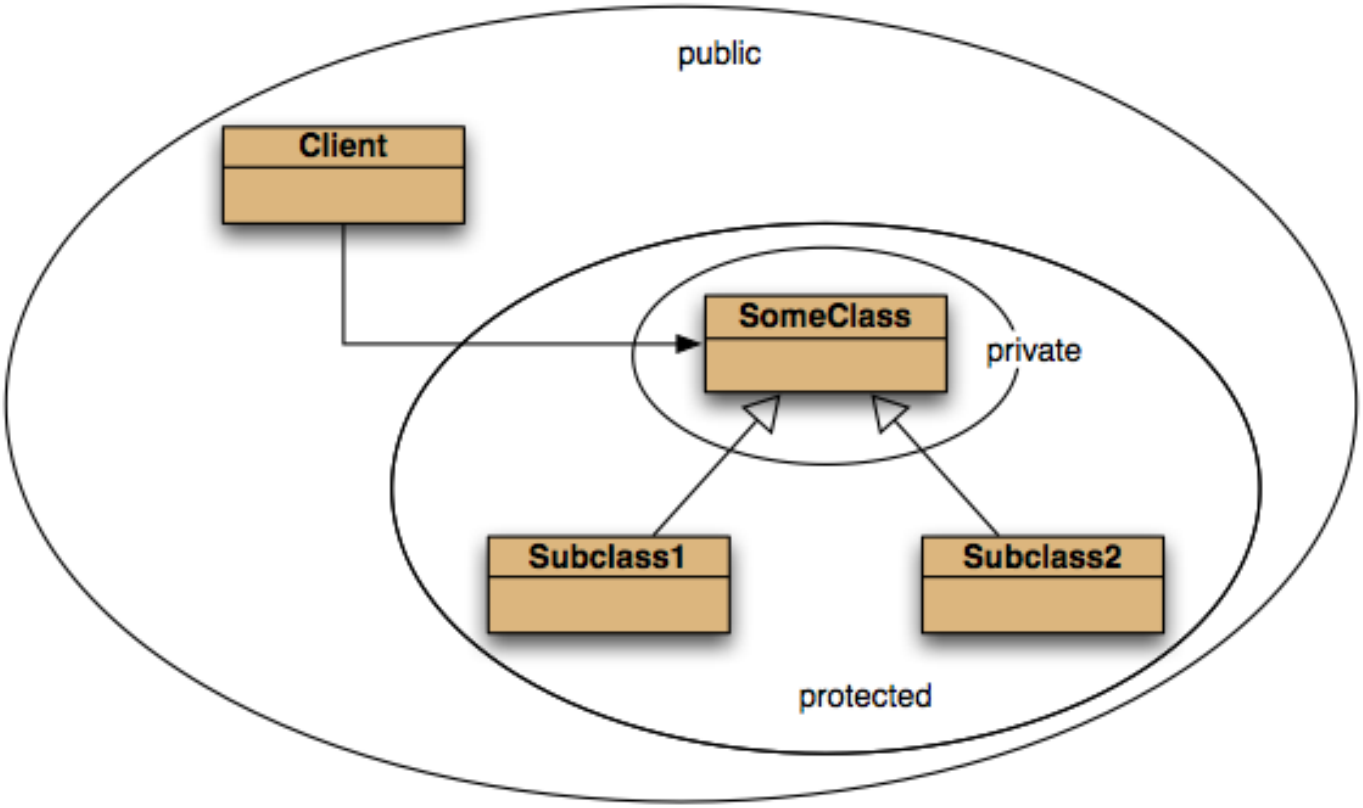
Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Acceso Protegido

- Hay casos en que tener un acceso private en los campos de la superclase puede ser demasiado restrictivo para una subclase.
- Para la relación más estrecha que representa la herencia existe el acceso protegido (**protected**).
- **Protected** es más restrictivo que **public**.
- **Aún así recomendamos mantener campos private**.
  - **Y definir como protected los métodos de acceso y mutadores**.
    - P.ej, podríamos hacer este cambio en Post si queremos que sus subclases usen este método:

~~private~~ protected String timeString(long time)

Niveles de acceso



Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Niveles de acceso

	Class	Paquete	Subclase (mismo Paquete)	Subclase (diferente Paquete)	Mundo (Otras Clases)
private	accesible	No accesible	No accesible	No accesible	No accesible
sin modificador (package-private)	accesible	accesible	accesible	No accesible	No accesible
protected	accesible	accesible	accesible	accesible	No accesible
public	accesible	accesible	accesible	accesible	accesible

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales

- Herencia
  - Constructores
- Polimorfismo
  - Subtipos y Sustitución
  - La Clase Object
  - Colecciones Polimórficas
  - Casting
  - Tipo Estático y Dinámico
  - Sobreescritura
  - En Métodos
- StringBuilder
- Clases y métodos abstractos
- Métodos de la clase Object
  - toString
  - equals
  - hashCode
- Extendiendo jerarquías de clases
- Herencia múltiple con Interfaces
- Acceso Protegido (protected)
- La clase “Class”

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

La clase `Class`

- Un objeto de tipo **`Class`** es devuelto por el método **`getClass()`** de **`Object`**.
  - Ej: Class classPhotoPost = photoPos1.getClass();
- Tiene algunos métodos que pueden ser útiles:
  - **`String getName()`** devuelve el nombre de la clase.
    - Ej: system.out.println(photoPos1.getClass().getName());
  - **`Field[] getDeclaredFields()`**
  - **`Method[] getDeclaredMethods()`**

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Resumen (1 de 7)

- Herencia permite la definición de clases como extensiones de otras clases.
- Herencia
  - evita duplicación de código
  - permite reutilización de código
  - simplifica el código
  - simplifica mantenimiento y ampliación
- Las variables pueden mantener objetos de subtipos.
- Los subtipos pueden sustituir a objetos esperados del supertipo.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Resumen (2 de 7)

- El tipo declarado de una variable es su Tipo Estático.
  - Compiladores comprueban Tipos Estáticos.
- El tipo de un objeto es su Tipo Dinámico.
  - Tipos Dinámicos son los usados en tiempo de ejecución.
- Los Métodos pueden ser sobreescritos en las subclases.
- La búsqueda del método a ejecutar comienza con el Tipo Dinámico.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Resumen (3 de 7)

- Herencia proporciona implementación compartida.
  - Clases Concretas y abstractas.
- Herencia proporciona información de tipos compartida.
  - Clases e interfaces.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Resumen (4 de 7)

- Métodos abstractos permiten comprobación estática de tipo sin requerir implementación.
- Clases abstractas funcionan como clases incompletas.
  - Sin instancias.
- Clases abstractas soportan **polimorfismo**.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Resumen (5 de 7)

- Interfaces proporcionan especificación sin implementación.
- Interfaces soportan **polimorfismo**.
- Interfaces proporcionan **herencia múltiple**.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Resumen (6 de 7)

- Se usa una clase cuando:
  - Hay que definir el estado y el comportamiento de un conjunto de objetos
  - Ninguna de las clases existentes encaja con el caso de uso de ese conjunto de objetos
- Se usa una subclase cuando:
  - Se necesita una versión más específica de una clase existente
  - Se quiere sobreescribir métodos o añadir nuevo comportamiento

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Resumen (7 de 7)

- Se usa una clase abstracta cuando:
  - Se quiere definir una plantilla para un conjunto de subclases (para una jerarquía de herencia)
  - Proporcionar implementación que todas las subclases podrían usar
  - Se quiere asegurar que nunca se instancia una clase
- Se usa un interface cuando:
  - Se quiere proporcionar funcionalidad común para un conjunto de clases, independientemente de su jerarquía de herencia
  - La definición de la funcionalidad es proporcionada por las clases que lo implementan

Otra bibliografía:

- Tony Gauvin - Introduction to Programming. Presentations. University of Maine. Fort Kent.
- Dr. Becka Morgan- Introduction to Computer Science. Presentations. University of Western Oregon.
- Christiane Hagedorn, Ann Katrin Kuessner, Thomas Staubitz, Ralf Teusner. Object-Oriented Programming in Java. OpenSAP Courses.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved